

Temporal Complex Event Recognition In Streaming Data

a diploma Thesis by

Athanasios Krevvatas

Konstantinos Koutsidis

Advisor:

Nikolaos Mamoulis, Professor

Ioannina, July 2026



ΤΜΗΜΑ ΜΗΧ. Η/Υ & ΠΛΗΡΟΦΟΡΙΚΗΣ

ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

UNIVERSITY OF IOANNINA

DEDICATION

To our beloved families, for supporting us every step of the way.

ACKNOWLEDGMENTS

Special thanks to Panos Simatis for his invaluable assistance and contribution to our work.

We are also profoundly grateful to our supervisor, Professor Nikos Mamoulis, for his insightful mentorship and guidance.

TABLE OF CONTENTS

ΠΕΡΙΛΗΨΗ	vi
ABSTRACT.....	vii
CHAPTER 1.....	1
INTRODUCTION	1
1.1 Complex Event Recognition Systems	1
1.2 Related Work	2
1.3 Objectives.....	5
1.4 Structure.....	6
CHAPTER 2.....	8
CORE	8
2.1 CORE System Overview	8
2.2 Fundamental Data Units and Stream Processing	9
2.3 CEQL: Complex Event Query Language	11
2.3.1 Mandatory Components.....	11
2.3.2 Optional Components.....	12
2.3.3 CEQL Example	13
2.4 Complex Event Automata (CEA)	13
2.4.1 CEA Evaluation.....	15
2.5 Quarantine	15
2.5.1 Drops	16
2.5.2 DIRECT policy	16
2.5.3 FIXED TIME policy	17
2.5.4 Example of CORE Quarantine Policies.....	18
2.6 Limitations of CORE's Quarantine Policies	21
CHAPTER 3.....	23
IMPROVING EVENT REORDERING IN CORE	23
3.1 New Temporal Attributes	23
3.1.1 Event Time	23

3.1.2	Arrival Time	24
3.1.3	System Arrival Delay	24
3.1.4	Detection Delay.....	24
3.2	New Fixed Time Policy	25
3.2.1	Example of New Fixed Time Policy	26
3.2.2	Limitations of New Fixed Time Policy.....	28
3.3	Dynamic Quarantine Policies	29
3.3.1	Dynamic Average Policy (AVG).....	30
3.3.2	Dynamic p99-EMA Policy (p99-EMA)	32
3.3.3	Dynamic MAX-EMA Policy (MAX-EMA)	34
3.3.4	Dynamic MAX Policy (MAX)	37
3.3.5	Dynamic Jump-and-Decay Policy (Jump/Decay).....	39
3.3.6	Dynamic Per-Event Policy (Per-Event)	41
CHAPTER 4.....		44
DATASET ANALYSIS		44
4.1 Dataset Overview.....		44
4.2 Detailed Dataset Analysis		45
4.2.1	Fires Dataset.....	45
4.2.2	Aviation Dataset	48
4.2.3	Crypto Dataset.....	50
CHAPTER 5.....		53
EVALUATION OF QUARANTINE POLICIES		53
5.1 Overview of Experiments.....		53
5.2 Detailed Analysis of the Fires Dataset		54
5.2.1	Fires Dataset: Medium-Complexity Query Analysis	54
5.2.1.1	Dropped Events	55
5.2.1.2	Complex Events.....	57
5.2.1.3	Execution Time	57
5.2.1.4	Detection Delay	58
5.2.1.5	Comparison with ideal results	61
5.2.1.6	Maximum Events in Quarantine and Quarantine Size	62
5.2.2	Fires Dataset: Simple Query Analysis	63
5.2.2.1	Dropped Events	63

5.2.2.2 Complex Events.....	64
5.2.2.3 Execution Time	65
5.2.2.4 Detection Delay	65
5.2.2.5 Comparison with ideal results	66
5.2.3 Fires Dataset: Complex Query Analysis	66
5.2.3.1 Dropped Events	67
5.2.3.2 Complex Events.....	68
5.2.3.3 Execution Time	69
5.2.3.4 Detection Delay	69
5.2.3.5 Comparison with ideal results	70
5.3 Detailed Analysis of the Aviation Dataset	71
5.3.1 Aviation Dataset: Medium-Complexity Query Analysis	71
5.3.1.1 Dropped Events	71
5.3.1.2 Complex Events.....	73
5.3.1.3 Execution Time	74
5.3.1.4 Detection Delay	75
5.3.1.5 Comparison with ideal results	75
5.3.1.6 Maximum Events in Quarantine and Quarantine Size	76
5.3.2 Aviation Dataset: Simple Query Analysis	77
5.3.2.1 Dropped Events	78
5.3.2.2 Complex Events.....	78
5.3.2.3 Execution Time	79
5.3.2.4 Detection Delay	80
5.3.2.5 Comparison with ideal results	80
5.3.3 Aviation Dataset: Complex Query Analysis	81
5.3.3.1 Dropped Events	82
5.3.3.2 Complex Events.....	82
5.3.3.3 Execution Time	83
5.3.3.4 Detection Delay	84
5.3.3.5 Comparison with ideal results	84
5.4 Detailed Analysis of the Crypto Dataset	85
5.4.1 Crypto Dataset: Medium-Complexity Query Analysis	85
5.4.1.1 Dropped Events	86
5.4.1.2 Complex Events.....	87
5.4.1.3 Execution Time	88
5.4.1.4 Detection Delay	89
5.4.1.5 Comparison with ideal results	90

5.4.1.6 Maximum Events in Quarantine and Quarantine Size	91
5.4.2 Crypto Dataset: Simple Query Analysis	92
5.4.2.1 Dropped Events	92
5.4.2.2 Complex Events.....	93
5.4.2.3 Execution Time	94
5.4.2.4 Detection Delay	94
5.4.2.5 Comparison with ideal results	95
5.4.3 Crypto Dataset: Complex Query Analysis	96
5.4.3.1 Dropped Events	96
5.4.3.2 Complex Events.....	97
5.4.3.3 Execution Time	98
5.4.3.4 Detection Delay	98
5.4.3.5 Comparison with ideal results	99
CHAPTER 6.....	100
CONCLUSION	100
6.1 Summary of the Project.....	100
6.2 Overall Conclusion and Policy Characteristics	100
6.3 Future Work & Extensions.....	101
REFERENCES.....	102

ΠΕΡΙΛΗΨΗ

Η διπλωματική μας εργασία επικεντρώθηκε στη βελτίωση του CORE+ [1], ενός συστήματος για την αναγνώριση σύνθετων γεγονότων. Μια σημαντική πρόκληση σε τέτοια συστήματα είναι η διαχείριση των ετεροχρονισμένων γεγονότων, δηλαδή δεδομένων που φτάνουν στο σύστημα με καθυστέρηση και εκτός χρονολογικής σειράς. Η αρχική υλοποίηση του CORE+ διαχειριζόταν αυτές τις καθυστερήσεις με έναν απλό μηχανισμό «καραντίνας», ο οποίος αποδείχθηκε αναποτελεσματικός. Η προσέγγιση αυτή οδηγούσε συχνά στην απώλεια κρίσιμων γεγονότων που θα μπορούσαν να συμβάλουν στην παραγωγή ενός τελικού αποτελέσματος, μειώνοντας έτσι τη συνολική ακρίβεια του συστήματος. Για την αντιμετώπιση αυτού του προβλήματος, αναπτύξαμε και υλοποιήσαμε νέες προσεγγίσεις για την αποδοτικότερη διαχείριση των ετεροχρονισμένων ροών δεδομένων. Η επέκτασή μας διασφαλίζει ότι τα καθυστερημένα γεγονότα ενσωματώνονται σωστά στη ροή επεξεργασίας του συστήματος αντί να απορρίπτονται. Ως αποτέλεσμα, η υλοποίησή μας επιτυγχάνει βελτιωμένες επιδόσεις και μεγαλύτερη αξιοπιστία σε σχέση με τις προηγούμενες μεθόδους, προσφέροντας μια πιο εύρωστη λύση για την επεξεργασία δεδομένων σε πραγματικό χρόνο χωρίς την απώλεια σημαντικών πληροφοριών.

ABSTRACT

Our thesis focuses on enhancing CORE+ [1], an open-source Complex Event Recognition system. A significant challenge in such systems is managing "late events" data that arrives out of chronological order due to various delays. The original implementation of CORE+ handled these late arrivals using a basic "quarantine" mechanism, which proved to be inefficient. This approach often led to the loss of critical events that could have contributed to a final complex result, thereby reducing the system's overall accuracy. To address this, we developed and implemented new approaches designed to manage delayed data flows more effectively. Our extension ensures that late events are correctly integrated into the processing stream rather than being discarded. As a result, our implementation achieves improved performance and higher reliability compared to the previous methods, providing a more robust solution for handling real-time data streams without sacrificing important information.

CHAPTER 1

INTRODUCTION

This opening chapter provides an overview of the thesis topic and outlines the structure of the work, including a description of each chapter.

1.1 Complex Event Recognition Systems

Complex Event Recognition (CER) systems are software frameworks designed to detect meaningful patterns within continuous streams of incoming events. An event is an atomic data record representing a single occurrence or state change at a specific point in time (e.g., a single sensor reading or a stock price update). Instead of analyzing each event in isolation, CER systems focus on identifying combinations of events that together represent a higher-level situation of interest. These situations often called complex events may involve temporal ordering, filtering conditions, correlations between attributes, or constraints on the time interval in which events occur. A complex event is a composite result produced by the combination of multiple simple events that match a specific pattern.

In practice, CER systems operate in environments where data arrives rapidly, and decisions must be made in real time. Examples include financial markets, where systems monitor trading activity for unusual patterns; cybersecurity, where sequences of login attempts or network packets may indicate an intrusion; industrial monitoring, where sensor readings must be interpreted to detect equipment failures; and transportation or maritime tracking, where movement patterns reveal abnormal behavior. In all these domains, the ability to recognize complex events quickly and accurately is essential for timely responses.

To perform this task, CER systems continuously maintain and update partial matches that represent patterns that have been partially satisfied by the events seen so far. As new events arrive, these partial matches may be extended, combined, or discarded depending on the pattern being evaluated. This process allows the system to detect complex events as soon as all required conditions are met.

However, the design of CER systems becomes considerably more demanding when events arrive late or out of order, a situation that frequently arises in distributed or high-latency environments. In such cases, events may appear long after other related events have already been processed, or in a sequence that does not reflect their actual temporal order. To cope with this behavior, the system must temporarily retain these delayed events until their correct logical position within the pattern can be determined. This temporary retention increases the number of intermediate states the system must manage, as a late event may still need to be matched with patterns that began earlier in the stream. When combined with expressive constructs such as iteration, disjunction, and time windows, the presence of late arrivals can significantly expand the number of partial matches the system must consider. High event throughput further amplifies this effect, making the efficient management of late and out-of-order events a crucial requirement for any practical CER system.

1.2 Related Work

Complex Event Recognition (CER) systems are generally classified into three main categories: automata-based, tree-based, and logic-based, with several engines adopting hybrid combinations of these approaches. A central challenge across all categories is the management of partial matches intermediate, incomplete event combinations maintained before a full pattern is detected. Traditional engines either store large volumes of partial-match state or repeatedly recompute it, and although many systems employ compact shared data structures to mitigate these costs, their efficiency and performance guarantees vary significantly.

To establish a clear evaluation framework, the core architectural families can be summarized as follows:

- Automata-based systems model event patterns as state-transition graphs, typically using Non-deterministic Finite Automata (NFAs). Each incoming event is evaluated against transition predicates, incrementally advancing active instances toward accepting states that represent full pattern matches.
- Tree-based systems adopt relational-style optimization, organizing operators such as filters, joins, and windows into hierarchical evaluation trees. Events enter through leaf nodes and propagate upward, progressively constructing composite events while pruning irrelevant branches.

- Hybrid systems combine both paradigms, often using tree-based filtering and indexing for rapid event discrimination, followed by automata-like state tracking to enforce strict temporal sequencing.
- Logic-based systems rely on declarative rule languages and formal logic programming. They treat event streams as evolving knowledge bases, deriving complex events through logical unification, backtracking, and temporal reasoning mechanisms.

Bridging these theoretical models with practical deployments, the literature includes several representative state-of-the-art frameworks. In automata-based execution, SASE [2] and Cayuga [16] pioneered real-time sequence detection over sequential streams, while modern distributed engines such as Apache Flink CEP [5] scale NFA-based pattern matching to large, fault-tolerant cloud environments. In contrast, tree-based systems like ZStream [17] apply cost-based operator reordering to minimize intermediate results, whereas imperative frameworks such as OpenCEP [15] implement hierarchical routing through custom object-oriented structures. Hybrid engines such as Esper [8] combine shared discrimination trees with dynamic state trees for memory-efficient sequence tracking. Finally, logic-based systems including RTEC [18] and Wayeb [19] employ formal semantics to support highly expressive temporal queries, robust handling of out-of-order data, and reasoning over background knowledge.

The following sections examine these representative systems in detail, highlighting how their architectural choices address the fundamental challenges of partial-match management, memory efficiency, and real-time performance.

SASE

SASE (Stream-based and Sequence-based Event Processing Engine) is an automata-based Complex Event Recognition (CER) system designed for detecting sequence patterns over event streams [2]. It provides a declarative query language with sequence operators, Kleene closure, predicates, and time-window constraints [2]. Queries are compiled into nondeterministic finite automata (NFAs), where states represent partial progress toward a match and transitions are triggered by incoming events satisfying the query conditions [3]. SASE maintains and incrementally extends partial matches as new events arrive, enabling real-time stream processing [3]. However, like most automata-based systems, SASE may materialize a large number of partial matches under high event rates, without

offering formal guarantees on memory usage or enumeration delay [4]. This limitation motivates approaches such as CORE, which provide optimal performance guarantees for partial match representation and enumeration.

Apache Flink CEP

Apache Flink CEP is a Complex Event Processing library built on top of Apache Flink's distributed stream-processing engine. Apache Flink is a powerful distributed stream-processing engine designed for real-time, high-throughput, and low-latency processing of continuous data streams [5]. Apache Flink CEP uses a high-level API for defining and detecting complex event patterns over continuous data streams. Flink CEP supports event-time semantics, enabling events to be processed according to their original timestamps rather than their arrival order. It uses watermarks together with configurable lateness thresholds to buffer and reorder out-of-order events before pattern evaluation [5,6]. Internally, Flink CEP follows an automata-based approach: each user-defined pattern is translated into a Non-Deterministic Finite Automaton (NFA), which maintains and updates partial matches as new events arrive [7]. The CEP API provides an expressive pattern language that includes sequences, iterations, disjunctions, time windows, and custom conditions [5]. It takes advantage of Flink's ability to run on many systems, handle large amounts of data, and recover from failures [7]. Its watermark-based mechanism handles late events, but events arriving after the allowed lateness are discarded, which limits the system's ability to recover very late events or detect the maximum number of patterns [6].

Esper

Esper is a hybrid Complex Event Processing (CEP) system that combines a tree-based filtering architecture with dynamic state management for real-time pattern detection [8]. It utilizes a Shared Filter Index (a discrimination tree) to route incoming events to relevant queries in nearly constant time. For its execution model, Esper employs dynamic evaluation trees to track multiple overlapping partial matches as events arrive, while employing automata-like logic to define the state transitions and sequencing rules required by complex event patterns [9]. Its main limitation, however, is the potential for state explosion, where high event rates or complex overlapping patterns can lead to significant memory consumption and unpredictable delays due to the large number of intermediate matches held in memory [10].

OpenCEP

OpenCEP is an imperative, tree-based Complex Event Processing (CEP) system designed to detect complex patterns by processing user-defined detection rules via a structured object-oriented execution model [4,15]. It belongs to the category of structural CEP systems, written in Java, where event streams are filtered and routed through hierarchical evaluation structures. During execution, tracks incoming event occurrences against active rules, progressively constructing evaluation paths to detect composite patterns in real-time [4,15]. This structural approach allows for clean integration with object-oriented applications and provides deterministic execution flows for standard pattern matching. However, its main limitation stems from the management of overlapping partial matches. Under high-throughput workloads or heavily nested patterns, the system faces significant computational overhead due to recursive path traversal and object instantiation, which can lead to increased processing latency compared to highly optimized, rigid finite-state machine implementations.

1.3 Objectives

The goal of this thesis is to improve CORE, a Complex Event Recognition system capable of efficiently processing high-volume event streams that contain late and out-of-order events. A central objective is to introduce and evaluate new policies for temporarily storing and managing delayed events, ensuring that they can be correctly integrated into the pattern-matching process once their logical position becomes clear.

We aim to improve a CER engine that maintains partial matches effectively, reduces the number of events that are discarded due to late arrival, and increases the total number of correctly detected complex events. The system focuses on expressive event patterns with temporal constraints, iteration, and disjunction, while providing mechanisms to handle delayed events without excessive memory usage or processing cost. As part of this work, multiple quarantine strategies were designed, implemented, and experimentally tested to determine their impact on accuracy, efficiency, latency, and scalability.

This thesis presents the methodology and design decisions that shaped the current version of the system. The following chapters will:

- Present the extension of CORE's temporal model, introducing an expanded set of time attributes that enable more precise reasoning about event disorder and late arrivals.

- Describe the improved Fixed-Time policy, outlining how the redesigned version addresses the limitations of the original static approach and provides a stronger baseline for comparison.
- Detail the design and implementation of new dynamic quarantine policies, including AVG, p99-EMA, MAX-EMA, MAX, Jump-and-Decay, and Per-Event, and explain how they adapt to the observed lateness characteristics of each stream.
- Report the experimental evaluation, analyzing correctness, memory behavior, detection delay, and runtime performance across three real-world datasets under diverse disorder conditions.
- Identify the most effective policy, demonstrating that the Per-Event Dynamic Policy achieves the best balance between correctness, adaptability, and computational efficiency.
- Propose potential extensions and optimizations for future development.

1.4 Structure

This thesis is organized into six chapters, each addressing a different aspect of the system and its evaluation. Their contents are briefly summarized below:

- **Chapter 1** introduces the topic of Complex Event Recognition, outlines the motivation behind this work, reviews related systems, and presents the objectives and overall structure of the thesis.
- **Chapter 2** provides a detailed description of the CORE system. It explains the fundamental data units, the CEQL query language, the Complex Event Automaton (CEA), and the original quarantine mechanism, including its limitations.
- **Chapter 3** presents the extensions introduced in this thesis to improve event reordering in CORE. It describes the new temporal attributes, the redesigned Fixed-Time policy, and the full suite of dynamic quarantine policies, along with their design rationale and implementation details.
- **Chapter 4** analyzes the three real-world datasets used in the experimental evaluation. It examines their temporal characteristics, disorder patterns, and structural properties, providing the necessary context for interpreting the results.
- **Chapter 5** contains the experimental evaluation of all quarantine policies across the Fires, Aviation, and Crypto datasets. It presents detailed analyses for each query type and metric, comparing efficiency, execution time, detection delay, and quarantine behavior.

- **Chapter 6** concludes the thesis by summarizing the main findings, discussing the comparative strengths and weaknesses of the examined policies, and outlining potential directions for future research and system extensions.

CHAPTER 2

CORE

This chapter describes the functional design of the CORE system, specifically focusing on how it processes incoming information, applies evaluation logic, and generates complex event outputs.

2.1 CORE System Overview

CORE is Complex Event Recognition (CER) engine built to detect specific patterns within massive, fast-moving data streams in real time. While CER systems are essential for critical tasks like maritime safety and fraud detection, most existing systems are held back by a challenge known as the "super-linear partial match problem". In typical setups, as the volume of data grows, the number of potentials matches the system must track increases so rapidly that it creates significant delays. CORE overcomes this by using a specialized algorithm that bypasses these typical performance constraints, ensuring that performance remains fast and reliable regardless of the data's volume or complexity.

Architecturally, CORE is an automata-based system that builds upon established principles of finite state machines but introduces a specialized execution model. The engine translates high-level queries into a specialized structure called a Complex Event Automaton (CEA). Unlike traditional models, a CEA is built to handle endless streams of data and uses "marking" actions to precisely identify which individual events belong in a final result. During operation, the system records these patterns using a compact graph structure that compactly represents partial and complete matches.

The defining innovation of CORE lies in its formal, proven, and optimal performance guarantees. It is the first CER system to guarantee constant-time updates per input event, meaning the time required to maintain its internal data structures does not increase as more data is processed. Additionally, CORE provides "output-linear delay" for enumeration. Once a match is identified, the system generates results in time proportional only to the size of the output itself. By combining these mathematical

foundations with practical features like time-window management and event partitioning, CORE achieves a throughput that can outperform the other systems.

2.2 Fundamental Data Units and Stream Processing

To understand the internal mechanics of the CORE engine, it is essential to first define the foundational data units it processes and how they are structured within the system.

Simple Event

A **simple event** is the fundamental atomic record that describes a specific real-world occurrence at a single point in time. Each event is a data-tuple containing all the relevant attributes required to process it independently, such as event types and specific data values.



Figure 2.1 A Simple Event

As illustrated in Figure 2.1 above, a single event represents a complete record of an occurrence, such as a flight arrival. In this example, the event contains attributes like the carrier's name (e.g., Delta Air Lines), flight number, origin (SFO), destination (MSP), and timestamp that indicates the exact moment when the event occurred. These attributes provide the necessary context for the system to evaluate individual events against broader patterns.

The Stream Model

The CORE system processes events as part of a stream, defined as a potentially infinite sequence of data-tuples. In typical setups, these streams are handled through structured formats such as CSV files, where each row represents a simple event that the system reads sequentially. Crucially, each event in the CSV includes a timestamp attribute as one of its

columns (shown in Figure 2.1). This timestamp records the exact moment when the event occurred in the real world, allowing the system to evaluate events in their correct temporal order.

Complex Event

A **complex event** is a higher-level occurrence derived from correlating, aggregating, or analyzing multiple simple events over time to identify a meaningful operational pattern or condition. Unlike a simple event, which captures an isolated moment, a complex event is formally defined by a time interval marking its duration and by the specific subset of simple events from the stream that satisfy the user-defined query.

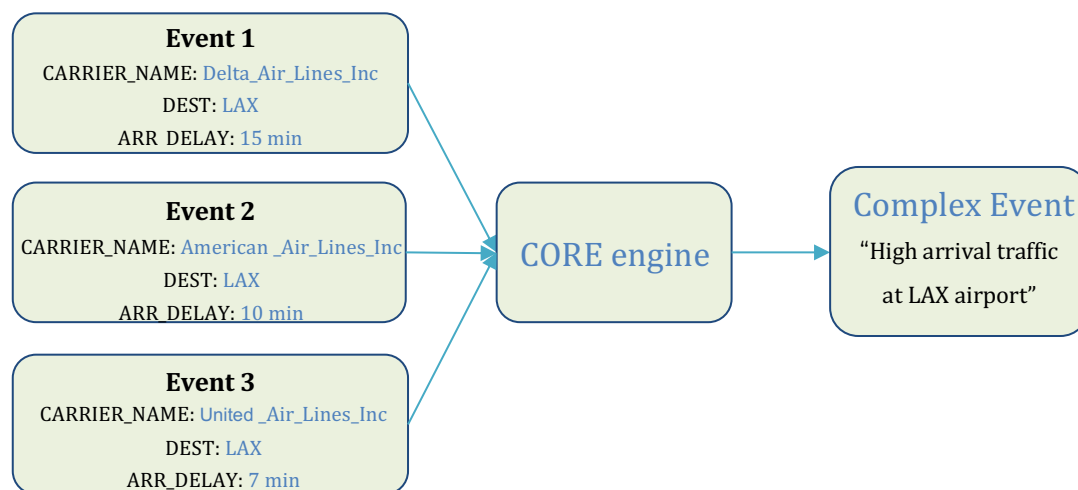


Figure 2.2 Composing a Complex Event

As illustrated in Figure 2.2, the CORE engine acts as the central processor that analyzes a continuous flow of data, such as individual flight arrivals from different carriers (Events 1, 2, and 3). Rather than treating each flight as an isolated data point, the system correlates them based on shared attributes, such as destination and arrival delays. When this sequence of events satisfies the specified logic within a defined timeframe, CORE produces a complex event as its final output, for example the “High arrival traffic at LAX airport” shown in the figure. Through this mechanism, the engine transforms raw data-tuples into structured, actionable insights.

2.3 CEQL: Complex Event Query Language

To define the patterns that the CORE engine must recognize, the system utilizes CEQL (Complex Event Query Language). This language provides a structured way to describe relationships between individual events, allowing users to express complex temporal logic through a syntax that is similar to traditional SQL but specifically tailored for streaming data. CEQL is a functional language built upon the theoretical foundations of Complex Event Logic (CEL), which provides the formal framework and logical operators necessary for pattern matching over data streams.

The structure of a CEQL query is organized into several clauses that define how data is gathered, filtered, and correlated. As shown in Figure 2.3a, the core logic of any query is expressed as a CEL-formula within the mandatory WHERE clause, which dictates the formal logical relationship between the events in the stream. In this implementation, the requirements are color-coded for clarity: the blue clauses are mandatory for the query to function, while those in brackets are optional enhancements that provide more specific control over evaluation. Syntactically, a CEQL query has this form shown below:

CEQL query's form:

```
SELECT < list-of-variables >  
FROM < list-of-streams >  
WHERE < CEL-formula >  
[FILTER <list-of-filters>]  
[PARTITION BY < list-of-attributes >]  
WITHIN < time-value >  
CONSUME BY NONE/ANY  
[LIMIT < number >]
```

Figure 2.3a CEQL query's form

CEQL Example:

```
SELECT * FROM S  
WHERE (BTC as btc; BNB as bnb;  
DOGE as doge; NEAR as near)  
PARTITION BY [Exchange]  
WITHIN 10 [Timestamp]  
CONSUME BY ANY
```

Figure 2.3b CEQL Example

2.3.1 Mandatory Components

Every functional query in CORE must include these five elements to establish its baseline logic:

- **SELECT:** Defines the output by specifying which variables or attributes should be returned once a pattern is successfully matched. As shown in the example (Figure

2.3b), the use of the asterisk (*) indicates that the query should return all captured data and variables associated with the match.

- **FROM:** Specifies the origin and provenance of the data to be analyzed. It identifies the input streams from CSV dataset and allows the engine to logically unify multiple sources into a single, cohesive chronological sequence for evaluation.
- **WHERE:** This is the most critical part of the query, as it contains the CEL-formula. It specifies the sequence of events using the ;(sequence) operator, which enforces that the listed events must occur in the given order. In addition, it defines the logical constraints that must be satisfied for a complex event to be recognized.
- **WITHIN:** Establishes a mandatory time window requiring the entire pattern (the CEL-formula) to be completed within a specified duration to be considered a valid match.
- **CONSUME BY:** Defines the selection strategy of the engine, determining how simple events are handled after a match is found:
 - **NONE:** This strategy allows events to be reused, enabling the system to identify all possible overlapping sequences that satisfy the pattern.
 - **ANY:** This strategy prevents event reuse. Once an event contributes to a result, it is consumed and cannot be used again for the same pattern.

2.3.2 Optional Components

While the mandatory components establish the foundational logic, CEQL offers additional optional parameters shown in brackets in Figure 2.3(a) that facilitate the construction of more complex queries:

- **FILTER:** Applies specific logical conditions to individual events, such as requiring a numerical attribute to exceed a certain threshold.
- **PARTITION BY:** Groups the stream by shared attributes to ensure the engine only correlates events related to the same entity. In practical terms, this attribute effectively maps to a specific column within the CSV dataset (e.g., "Exchange" or

"Carrier"), allowing the system to isolate and track patterns for each unique value in that column independently.

- **LIMIT:** Restricts the total number of recognized complex events generated for a specific query.

2.3.3 CEQL Example

To demonstrate how CEQL can express more complex multi-asset patterns, consider a scenario where a user wants to detect a specific sequence of cryptocurrency events across several trading pairs. The query shown in Figure 2.3b monitors the stream *S* and looks for four consecutive events corresponding to BTC, BNB, DOGE, and NEAR. By applying `PARTITION BY [Exchange]`, the system ensures that the pattern is evaluated independently for each exchange, preventing cross-exchange interference. The `WITHIN 10 [Timestamp]` clause restricts the entire sequence to occur within a 10-unit time window, enforcing temporal coherence among the four events. Finally, the query uses `CONSUME BY ANY`, meaning that once a matching sequence is detected, the contributing events are consumed and cannot participate in another match.

2.4 Complex Event Automata (CEA)

A Complex Event Automaton (CEA) is the computational model that CORE uses to evaluate CEQL queries efficiently. Conceptually, a CEA resembles a finite-state automaton, but instead of consuming symbols from a finite alphabet, it processes a stream of data-tuples. Each transition in a CEA is labeled with a predicate and an action, allowing the automaton to decide whether an incoming event should advance the computation and whether that event should be marked as part of the resulting complex event. Formally, a CEA is defined as a tuple:

$$A = (Q, \Delta, q_0, F)$$

where:

- **Q** is a finite set of states,
- **Δ** is a transition relation that defines how the automaton moves between states
- **$q_0 \in Q$** is the initial state, and
- **$F \subseteq Q$** is the set of final states.

A transition happens when an incoming event satisfies a condition and an associated action determines whether this event is included in the output. This method effectively represents CEQL operators such as sequencing, filtering, disjunction, and iteration, making CEA the core execution framework for CORE.

CEA runs represent all possible ways in which a CEQL pattern can match a stream. A run begins at some position in the stream, follows transitions whose predicates match the incoming tuples, and ends at a final state. The marked positions along the run form the data component of the resulting complex event. Because CEQL allows non-contiguous sequencing and iteration, a single stream may induce many overlapping runs, each corresponding to a different complex event.

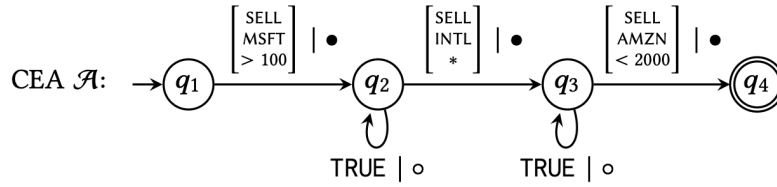


Figure 2.4 Example of a CEA

The automaton in Figure 2.4 [1] is a Complex Event Automaton (CEA) designed to detect a specific sequence of stock-market events. It consists of four states, starting from q_1 and ending at the accepting state q_4 . Each transition between states is labeled with a predicate that must be satisfied by the incoming event and an action indicating whether the event should be included in the output complex event. A marked transition ($\bullet$) means the event is part of the detected pattern, while an unmarked transition ($\circ$) means the event is ignored but still allows the automaton to continue. The pattern encoded here corresponds to three SELL events in order: a Microsoft sale above 100 USD, an Intel sale of any price, and an Amazon sale below 2000 USD. The self-loops on q_2 and q_3 allow the automaton to skip irrelevant events without breaking the sequence, enabling non-contiguous matching. A run reaches the final state q_4 only when all three required events have been observed in the correct order, and the marked events along that run form the resulting complex event.

2.4.1 CEA Evaluation

CORE uses an innovative evaluation algorithm that overcomes the classical rapid growth of partial matches by maintaining a compact graph representation of all active CEA runs. Instead of storing each partial match explicitly, CORE encodes them in a shared structure whose size is linear in the number of processed events. When a new event arrives, the graph is updated in constant time, regardless of how many partial matches exist. This is achieved by carefully organizing runs so that shared prefixes and suffixes are represented only once, and by maintaining a ranked ordering of runs to enable removal of outdated or irrelevant runs based on time windows.

The evaluation process proceeds as follows. Each incoming tuple is checked against the predicates of outgoing transitions from all currently active states. Matching transitions extend the graph with new edges representing the updated runs. The action associated with each transition determines whether the tuple becomes part of the complex event. Marked positions are stored in the graph so that they can be retrieved during enumeration. CORE integrates time windows directly into the evaluation algorithm. Runs are ranked by their start time, allowing the system to remove all runs that have exceeded the time window in a single pass without scanning the entire structure. When a run reaches a final state, CORE can enumerate the corresponding complex events with output-linear delay, meaning the time to produce each result is proportional only to its size, not to the number of partial matches in the system.

This evaluation strategy provides formal guarantees of constant-time updates per event and linear-size memory usage under data complexity. As a result, CORE remains stable even for long time windows and highly expressive CEQL queries, outperforming existing CER engines by several orders of magnitude.

2.5 Quarantine

CORE uses a quarantine mechanism to manage the order of events in its data stream processing. Quarantine is a buffering technique that temporarily holds and reorders incoming events before they are processed. Its primary purpose is to handle out-of-order events, which occur when events arrive in a sequence different from their actual occurrence time, a common challenge in real-world data streams. By applying quarantine, the system ensures that events are processed in the correct logical order required by CEQL queries, improving accuracy and consistency.

In CORE, quarantine policies decide whether an event should be reordered or dropped before entering the main evaluation engine. The initial version of CORE includes two baseline quarantine mechanisms: the **Direct policy** and the **Fixed Time policy**.

2.5.1 Drops

In CORE's quarantine mechanism, drops refer to events that arrive too late to be reordered safely. When an event's timestamp is earlier than the latest event already sent to the system from the quarantine buffer, it is considered late and is dropped to maintain the accuracy of the event processing. Dropping such events prevents incorrect matches or evaluations caused by out-of-order arrivals that cannot be corrected. Events with timestamps earlier than the latest event already processed cannot be reordered safely and are therefore dropped. While drops reduce the number of events in the processed stream, they are necessary to ensure the accuracy and consistency of query results by avoiding the processing of disordered events.

2.5.2 DIRECT policy

In the DIRECT policy, CORE does not use any quarantine mechanism. Every event is forwarded to the engine immediately in the exact order it arrives. This means that the system does not check for network delays or any other disorder in the stream. The DIRECT policy therefore offers the lowest possible latency, since events are processed as soon as they appear, but it also provides the weakest guarantees regarding accuracy. Any event that arrives late, meaning its timestamp is earlier than previously processed events, cannot be placed in its proper temporal position and may cause missed matches or incorrect window evaluations. Such events are dropped and not processed by the system. DIRECT policy is suitable only when the input stream is already well-ordered or when extremely low latency is more important than accuracy.

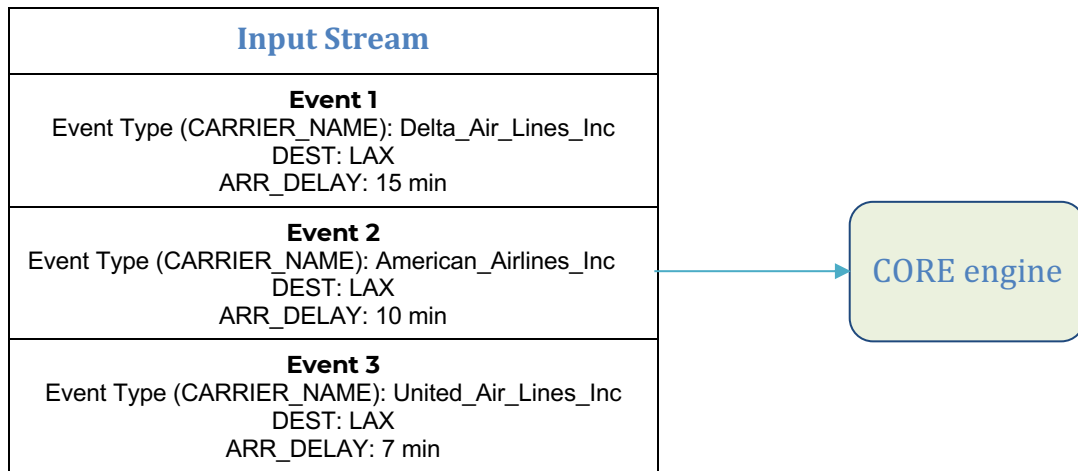


Figure 2.5: Diagram of the Direct Quarantine Policy

Based on Figure 2.5, the diagram visually demonstrates how the DIRECT policy sends events to the CORE engine immediately in their arrival order, without buffering or reordering. As shown, events are processed as soon as they appear in the input stream, and any event that arrives late, meaning its timestamp is earlier than the last processed event, is dropped because it cannot be placed in the correct temporal position.

2.5.3 FIXED TIME policy

The Fixed Time policy introduces a quarantine buffer where each event is held for a fixed amount of wall-clock time before being released to CORE. When an event arrives, it is inserted into the buffer in the correct position according to its timestamp. The event remains there until its quarantine time expires. Once the waiting period has passed, the event with the smallest timestamp in the buffer is released, along with any other events whose quarantine time has also expired. This mechanism allows CORE to reorder events and manage the disorder in the stream. However, if an event arrives after the system has already released events with later timestamps, it is considered too late to be safely reordered and it is dropped. The Fixed Time policy therefore improves accuracy compared to DIRECT but introduces additional latency and still struggles when delays exceed the configured waiting threshold.

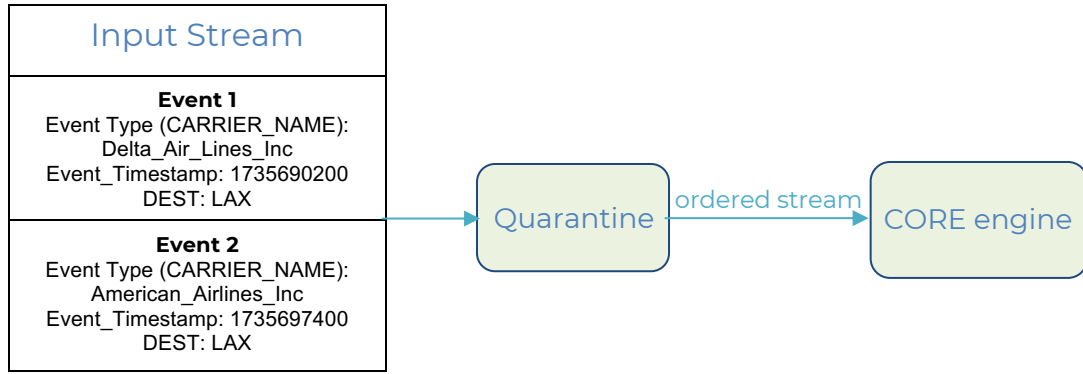


Figure 2.6: Diagram of the Fixed Time Quarantine Policy

Based on Figure 2.6, the diagram shows how the Fixed Time policy temporarily holds incoming events in a quarantine buffer before sending them to the CORE engine. As shown, events are inserted into the buffer according to their timestamps and remain there for a fixed wall-clock time before release. Once this time expires, the event with the smallest timestamp is released for processing. This visual diagram highlights how the policy reorders out-of-order events, while late events are still dropped if they arrive after the system has already released later-timestamped events from quarantine.

2.5.4 Example of CORE Quarantine Policies

This subsection provides a concrete example that demonstrates how the DIRECT and Fixed Time quarantine policies process a simple input stream. By applying both policies to the same set of events, we can observe how they differ in their ability to maintain temporal order and under which conditions events are dropped because they cannot be placed in a valid position in the event-timestamp sequence.

Input stream	
Event	Timestamp
Event A	17
Event B	8
Event C	30
Event D	10

Table 2.1: Example input stream of events with their timestamp

The input stream in Table 2.1 contains four events, each associated with a timestamp that reflects when it occurred. Each event is assumed to arrive in the order listed in the table, allowing to examine how the policies react when this arrival order does not match the underlying event-timestamp sequence. The timestamps are unordered, creating a scenario where some events may appear late relative to others. This structure allows us to demonstrate how the DIRECT and Fixed Time policies differ in their ability to reorder events, tolerate disorder, and decide when an event must be dropped because it can no longer be placed in a valid temporal position.

The following table presents the evaluation of the DIRECT policy, which is examined first because it does not use any quarantine mechanism and sends events immediately upon arrival:

Input Stream Description	Drop check	System Action	Last Sent Event
Arrive Event A (ET:17)	No drop	Event is sent	A (ET:17)
Arrive Event B (ET:8)	Drop ($8 < 17$)	Event is dropped	A (ET:17)
Arrive Event C (ET:30)	No drop ($30 > 17$)	Event is sent	C (ET:30)
Arrive Event D (ET:10)	Drop ($10 < 30$)	Event is dropped	C (ET:30)

Events sent to the system: [Event A, Event C]

Dropped events: [Event B, Event D]

Table 2.2: Step-by-step evaluation of the DIRECT quarantine policy over the example input stream.

Table 2.2 presents the behavior of the DIRECT quarantine policy when applied to the example input stream of Table 2.1. For each arriving event, the table shows whether the event is dropped, what action the system takes, and which event is currently the last one sent to the system for query evaluation. The key rule of DIRECT is that an event is dropped if its timestamp is smaller than the timestamp of the last sent event, otherwise it is immediately forwarded.

When Event A (ET:17) arrives, there is no previously sent event, so it is sent to the CORE system for further processing and A becomes the last sent event. When Event B (ET:8) arrives, its timestamp is earlier than that of A ($8 < 17$), so it is considered late and is dropped. The last sent event remains A. Event C (ET:30) arrives with a later timestamp

than A ($30 > 17$), so it is sent and becomes the new last sent event. Finally, Event D (ET:10) arrives with a timestamp earlier than C ($10 < 30$) and is therefore dropped, leaving C as the last sent event. Overall, because the DIRECT policy sends events immediately upon arrival and no reorders them, it consequently drops many of events that arrive out of order.

The following table presents the evaluation of the Fixed-Time policy, which introduces a quarantine mechanism that temporarily buffers events before releasing them:

Quarantine time: 5 sec.

Input Stream Description	Drop check	Quarantine before insertion	System Action	Quarantine after insertion	Last Sent Event
Arrive Event A (ET:17)	No drop	[]	Buffer A	[A]	-
Arrive Event B (ET:8)	No drop	[A]	Buffer B	[B, A]	-
Arrive Event C (ET:30)	No drop	[B, A]	Buffer C	[B, A, C]	-
Arrive Event D (ET:10)	No drop	[B, A, C]	Buffer D	[B, D, A, C]	-
End of stream	-	[B, D, A, C]	Release all events from quarantine	[]	C (ET:30)

Events sent to the system: [Event B, Event D, Event A, Event C]

Dropped events: []

Table 2.3 Step-by-step evaluation of the Fixed-Time quarantine policy (5-second window) over the example input stream

Table 2.3 presents the behavior of the Fixed-Time quarantine policy when applied to the example input stream of Table 2.1. For each arriving event, the table shows whether the event is dropped, how the quarantine buffer evolves before and after insertion, what action the system takes, and which event is currently the last one sent in CORE system for query evaluation. In the Fixed-Time policy, every event is placed into a quarantine buffer for a fixed time of 5 seconds after its arrival. This 5-second interval corresponds to a

real-time window maintained by the system, during which the event is temporarily held and inserted into the buffer in increasing event-timestamp order. During this 5-second window, each arriving event is added to the quarantine buffer in the correct temporal position, ensuring that the buffer always remains sorted by event timestamp.

When Event A (ET:17) arrives, the quarantine is initially empty, so A is inserted as the only buffered event. Event B (ET:8) arrives next and is inserted before A because its timestamp is earlier, so the sorted quarantine buffer at that time is [B, A]. Event C (ET:30) is appended at the end of the buffer, and Event D (ET:10) is inserted between B and A, producing the final sorted quarantine [B, D, A, C]. The system performs these ordering operations well within the 5-second real-time window, meaning that all events are fully sorted before their quarantine expires. As a result, no events are dropped. After the end of the stream, and once all quarantine windows have expired, the system releases the buffered events in increasing event-timestamp order, B first, followed by D, A, and finally C, having a fully ordered output stream. Overall, the Fixed-Time policy successfully reorders the out-of-order input without dropping any events, in contrast to DIRECT, which immediately sends events and therefore drops those that arrive late.

2.6 Limitations of CORE's Quarantine Policies

Although the Direct and Fixed-Time policies are two distinct strategies for handling events within CORE, both approaches have limitations that restrict their effectiveness in realistic streaming environments.

DIRECT emphasizes low latency by sending each event as soon as it arrives, without any buffering or reordering. This design minimizes latency but makes the system extremely sensitive to temporal disorder: any event whose timestamp is earlier than the last sent event is permanently dropped. As a result, DIRECT is unsuitable for applications with network delays and disorder events, since it sacrifices accuracy for minimal delay.

The Fixed-Time policy addresses some of these issues by introducing a real-time quarantine window during which events are temporarily held and ordered before being released. However, its fixed-time nature introduces a different set of challenges. Selecting an appropriate quarantine time is difficult because a short window may not be able to fix the disorder stream, leading to unnecessary drops and loss of results, while a long window increases latency even when the stream is nearly ordered. Moreover, the policy cannot adapt to network latencies. It applies the same waiting time to all events regardless of whether the stream is well ordered or highly unordered, failing to account

network delays and unordered arrival of events, which can further decrease accuracy and increase latency.

These limitations reveal that CORE's existing quarantine mechanisms lack the flexibility needed to respond to dynamic and evolving stream characteristics, motivating the exploration of more adaptive approaches.

CHAPTER 3

IMPROVING EVENT REORDERING IN CORE

The previous chapter mentioned the limitations of CORE’s existing quarantine policies and their inability to adapt to disorder in real event streams. Based on these observations, this chapter introduces new quarantine policies that allow CORE to adapt dynamically according to observed disorder and network delays, by improving accuracy. We integrate additional temporal attributes (event time, arrival time, system arrival delay and detection delay) to better capture the structure of incoming data. Based on these concepts, we present an improved version of the Fixed-Time policy and introduce new dynamic quarantine mechanisms that adjust their behavior according to the observed characteristics of the stream. These enhancements aim to provide more flexible and robust handling of out-of-order events within CORE.

3.1 New Temporal Attributes

To support more accurate reasoning about disorder in event streams, CORE is extended with a set of additional time attributes that describe different temporal aspects of each event. These attributes allow the system to distinguish between when an event occurred, when it was observed by the system, and how much was its overall delay. The input stream used in this work (provided as a CSV file) includes dedicated columns for these time values, enabling precise measurement and analysis of temporal behavior.

3.1.1 Event Time

The **Event Time** represents the moment when the event actually occurred in the real world. It reflects the timestamp associated with the event’s generation, independent of when it is received by the system. Event time is crucial for defining the correct temporal order of events and for evaluating time-based patterns and windows. In the CSV input stream, the `event_time` column records this value for each event.

3.1.2 Arrival Time

The **Arrival Time** represents the moment when the event arrives at the system. Arrival time is affected by network delays, meaning it may differ significantly from when the event occurred. This attribute is essential for understanding how events differ from their ideal event-time order. In the CSV input stream, the arrival_time column records this value for each event.

3.1.3 System Arrival Delay

The **System Arrival Delay** represents the temporal difference between when an event occurred and when it was received by the system. It reflects the delay introduced by network latency, that cause events to arrive later than their occurrence time. This delay is essential for understanding the degree of disorder present in the input stream and for selecting the appropriate quarantine mechanism that can adapt to varying levels of lateness. Formally, system arrival delay is computed as the difference between arrival time and event time:

$$\text{System Arrival Delay} = \text{Arrival Time} - \text{Event Time (1)}$$

3.1.4 Detection Delay

The detection delay represents the total end-to-end latency experienced by an event, capturing the full duration from its original generation to the moment the system produces the corresponding output (the complex event). It combines both the external delay introduced before the event reaches CORE and the internal delay required for the system to process it. Formally, detection delay is defined as the sum of the system arrival delay and the processing delay:

$$\text{Detection Delay} = \text{System Arrival Delay} + \text{Processing Delay}$$

where system arrival delay (1) reflects the difference between arrival time and event time, and processing delay measures the time CORE needs to execute its logic after the event has arrived. Processing Delay is computed using the following formula:

$$\text{Processing Delay} = \text{Output Result Time} - \text{Arrival Time}$$

where output result time represent the time, the system outputs the result. Detection Delay provides a comprehensive view of how long an event takes to be fully handled by the system.

3.2 New Fixed Time Policy

The initial Fixed-Time quarantine policy of CORE, introduced in *Chapter 2.5.3*, operates by buffering incoming events for a pre-defined waiting time before releasing them for evaluation. This mechanism assumes that events may arrive out of order and uses a fixed delay, measured in real wall-clock time, to fix disorder. This approach does not account for varying arrival patterns and network conditions, which can lead to either unnecessary waiting or early release of events before all relevant data has arrived. This can also increase the number of events that are dropped in the system.

To address these limitations, this work introduces an enhanced version of the Fixed-Time policy, called New Fixed-Time Policy. Unlike the original approach, which relies exclusively on wall-clock time, the new policy operates directly on the timestamps of the events. This change allows the system to rely on the temporal characteristics of the stream itself rather than on external timing.

At the New Fixed-Time Policy, events are placed into quarantine upon arrival, and the system continuously monitors the arrival time of the most recently received event. This arrival time acts as a reference point for determining how far the stream has progressed. An event stored in quarantine is released only when the difference between the latest arrival time and the event's own arrival time exceeds a configured waiting interval. Additionally, the policy handles extremely late events. If an event arrives with an event time smaller than the event time of the last event already released to CORE, it is considered too late and it is therefore dropped. This prevents outdated events from corrupting the evaluation state or producing invalid results.

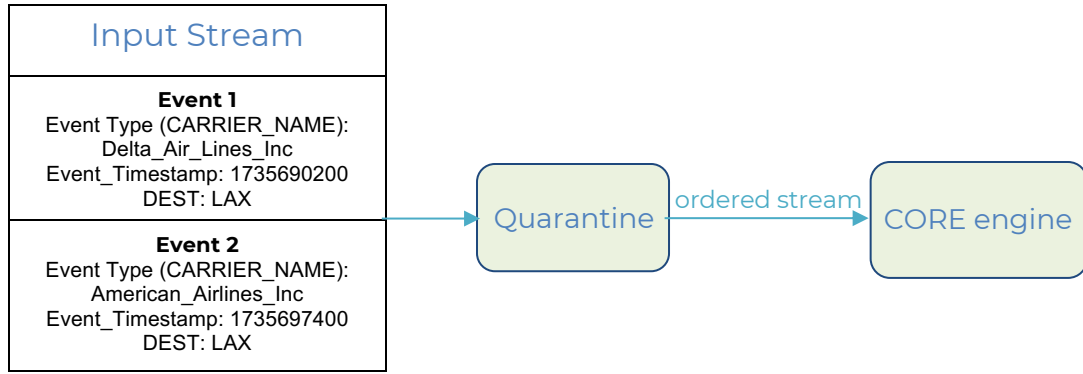


Figure 3.1: Diagram of the New Fixed Time Quarantine Policy

Based on the Figure 3.1, the New Fixed-Time Policy extends the original approach by using event timestamps rather than wall-clock time to manage disorder. Incoming events are placed into a quarantine buffer, and their release is determined by comparing their arrival time to the arrival time of the most recently received event. An event is forwarded to the CORE engine only when this difference exceeds a configured waiting time. This mechanism allows the policy to reorder out-of-order events while still dropping extremely late ones whose event times are smaller than the last event already released.

3.2.1 Example of New Fixed Time Policy

This subsection provides a concrete example that demonstrates how the New Fixed-Time Policy processes an input stream using both event times and arrival times. By applying the policy to this set of events, we can observe how it leverages arrival-time progression to reorder out-of-order events and identify the conditions under which late events must be dropped because they can no longer be placed in a valid temporal position.

Input stream		
Event	Event Time	Arrival Time
Event A	20	22
Event B	10	40
Event C	45	50
Event D	15	55

Table 3.1: Example input stream of events with their event time and arrival time

The input stream in Table 3.1 contains four events, each associated with both an event time and an arrival time. As in the previous examples, the events are assumed to arrive in the order listed, allowing us to examine how the New Fixed-Time Policy behaves when the arrival order does not align with the event-time sequence. The combination of unordered event times and varying arrival times creates a realistic scenario where some events may appear late relative to others. This example shows how the New Fixed-Time Policy uses arrival-time progression to tolerate disorder, reorder events within the quarantine buffer, and determine when an event must be dropped because it can no longer be placed in a valid temporal position.

The following table presents the evaluation of the New Fixed-Time Policy, which uses a quarantine buffer that temporarily stores events and releases them based on the arrival times of the events and a pre-defined waiting time:

Quarantine time: 5 sec.

Arrival Time	Input Stream Description	Drop check	Quarantine before insertion	System Action	Quarantine after insertion	Last Sent Event
22	Arrive Event A (ET:20)	No drop	[]	Buffer A	[A]	-
40	Arrive Event B (ET:10)	No drop	[A]	Hold A, Buffer B	[B, A]	-
50	Arrive Event C (ET:45)	No drop	[B, A]	Release B, Buffer C	[A, C]	B (ET:10)
50			[A, C]	Release A	[C]	A (ET:20)
55	Arrive Event D (ET:15)	DROP D	[C]	-	[C]	A (ET:20)
	End of stream	-	[C]	Release all events from quarantine	[]	C (ET:45)

Events sent to the system: [Event B, Event A, Event C]

Dropped events: [Event D]

Table 3.2: Step-by-step evaluation of the New Fixed-Time quarantine policy (5-second window) over the example input stream

Table 3.2 presents the behavior of the New Fixed-Time Policy when applied to the example input stream of Table 3.1. For each arriving event, the table shows whether the event is dropped, how the quarantine buffer evolves before and after insertion, what action the system takes, and which event is currently the last one sent to the CORE system for query evaluation. In contrast to the original Fixed-Time policy, which relied on a fixed wall-clock delay, the New Fixed-Time Policy bases its decisions on the arrival-time progression of the stream. Each event is temporarily stored in the quarantine buffer, and its release is determined by comparing its arrival time with the arrival time of the most recently received event. If this difference exceeds the pre-defined waiting interval of 5 seconds, the event is released, otherwise, it remains in quarantine. This mechanism allows the system to tolerate with the disorder.

When Event A (ET:20) arrives at time 22, the quarantine is empty, so A is simply buffered. Event B (ET:10) arrives next at time 40. Because its event time is earlier than A's, the sorted quarantine places B at the front of the queue, producing the buffer [B, A]. The policy evaluates the first event in the queue (B) and calculates its waiting duration as 0 seconds (Current Arrival Time 40 - B's Arrival Time 40). Since this is less than the 5-second waiting interval, the evaluation stops immediately. This holds B and forces A to remain in quarantine, even though A has already been waiting for 18 seconds.

When Event C (ET:45) arrives at time 50, the system checks the buffer again. The waiting duration for B is now 10 seconds (50 - 40), exceeding the waiting interval, so B is released. The system then evaluates the next event, A, and finds its duration is 28 seconds (50 - 22), which also exceeds the interval, so A is released as well. Event C is evaluated, but its duration is 0 seconds (50 - 50), so it remains held, updating the buffer to [C].

Event D (ET:15) arrives afterward at time 55. Before it can even be inserted into the buffer, the system checks its event time (15) against the last event already released (A, with ET:20). Since 15 is smaller than 20, Event D is immediately dropped. At the end of the stream, the remaining event in quarantine, C, is released.

3.2.2 Limitations of New Fixed Time Policy

Although the New Fixed-Time Policy improves disorder by using arrival time of each event, it still relies on a pre-defined and static waiting interval. This fixed window does not adapt to the stream behavior, such as sudden delays or periods of highly ordered arrivals. As a result, the policy may wait longer than necessary during stable conditions or release events earlier when network delays increase. This limitation highlights the

need for more flexible mechanisms that adjust their waiting time based on the characteristics of the incoming stream, motivating the introduction of dynamic quarantine policies in the following chapter.

3.3 Dynamic Quarantine Policies

To overcome the limitations of static waiting times, this chapter introduces a new set of Dynamic Quarantine Policies designed to adapt their behavior to the characteristics of the incoming stream. Unlike the New Fixed-Time Policy, which relies on a pre-defined and unchanging waiting window, these dynamic policies continuously adjust their quarantine time based on recent arrival-time patterns. Their goal is to reduce unnecessary drops, improve temporal ordering, and increase the number of complex events detected, while maintaining execution times as stable as the existing quarantine policies.

The dynamic quarantine approach includes six distinct policies, each implementing a different strategy for estimating the appropriate waiting interval: **Dynamic Average (AVG)**, **Dynamic p99-EMA**, **Dynamic MAX-EMA**, **Dynamic MAX**, **Dynamic Jump-and-Decay**, and **Dynamic Per-Event**. Despite their differences, all dynamic policies share a common implementation structure. Each maintains a quarantine buffer where events are temporarily held before being forwarded to the CORE engine. They also rely on a sliding window of the most recent 500 events, a continuously updated subset of the stream that captures the latest arrival behavior. This window is used to observe system arrival delays, estimate current network conditions, and adapt the quarantine time accordingly. All dynamic policies use a safety margin of 1.5, which is a multiplier that slightly increases the calculated quarantine time so the system can handle sudden spikes of delay. In practice, it prevents the policy from overreacting to small lateness values and from releasing events too early, while keeping the system adaptive and responsive. Out-of-order events that violate temporal correctness are still dropped, and each policy enforces both a user-defined minimum quarantine time and a maximum limit of one million seconds to prevent unbounded waiting. Time is defined in terms of CSV timestamp differences, not actual wall-clock time.

The following sections describe each policy in detail and evaluate how their adaptive behavior improves event-stream handling under varying network conditions.

3.3.1 Dynamic Average Policy (AVG)

The Dynamic Average Policy computes its quarantine time by continuously tracking the recent behavior of the stream and adapting to it. Specifically, the policy maintains a sliding window of the 500 most recent events and records their *System Arrival Delay*, which represents how late each event arrived at the system relative to its event time. At every new arrival, the policy calculates the average of these delays and uses this value as the current quarantine time. This means that if the stream experiences higher latency or increased disorder, the average delay grows, causing the quarantine window to expand and allowing more time for earlier events to arrive. Conversely, when the stream becomes stable and delays decrease, the average shrinks, reducing the quarantine time and allowing events to be released more quickly.

To illustrate how the Dynamic Average Policy works, we apply it to the input stream of Table 3.1:

Min quarantine time: 5 sec.

Arrival Time	Input Stream Description	New Quarantine Time	Drop check	Quarantine before insertion	System Action	Quarantine after insertion	Last Sent Event
22	Arrive A (ET:20)	5 sec.	No drop	[]	Buffer A	[A]	-
40	Arrive B (ET:10)	24 sec.	No drop	[A]	Hold A, Buffer B	[B, A]	-
50	Arrive C (ET:45)	18.5 sec.	No drop	[B, A]	Hold B, Hold A, Buffer C	[B, A, C]	-
55	Arrive D (ET:15)	28.8 sec.	No drop	[B, A, C]	Hold B, Hold A, Hold C, Buffer D	[B, D, A, C]	-
	End of stream		-	[B, D, A, C]	Release all	[]	C (ET:45)

Events sent to the system: [Event B, Event D, Event A, Event C]

Dropped events: []

Table 3.3: Step-by-step evaluation of the Dynamic Average Policy over the example input stream

Table 3.3 presents the behavior of the Dynamic Average Policy (AVG) when applied to the example input stream of Table 3.1. As each event arrives, the policy computes its quarantine time based on the average arrival delay of the most recent events, multiplied by a safety margin, while always takes account the minimum waiting interval of 5 seconds. The table shows how the sliding window of latencies evolves, how the dynamic quarantine time is updated, and how the system decides whether to buffer, release, hold, or drop each event.

When Event A (ET:20) arrives at time 22, its lateness is 2 seconds, so the sliding window contains only this value. The average lateness is 2 seconds, and after applying the 1.5 safety margin, the calculated time is 3 seconds. Because this is below the minimum allowed value, the dynamic quarantine time becomes 5 seconds. Since the quarantine is initially empty, A is simply inserted into the buffer. The system checks if A can be released, but its waiting duration is 0 seconds ($22 - 22$), which is less than the 5-second threshold. Therefore, A is held in quarantine.

When Event B (ET:10) arrives at time 40, its lateness is 30 seconds, so the sliding window becomes {2, 30}. The average lateness is now 16 seconds, and after applying the safety margin, the dynamic quarantine time increases to 24 seconds. Because its event time is earlier than A's, B is sorted to the front of the queue, producing the buffer [B, A]. The system evaluates the first event (B), but its waiting duration is 0 seconds ($40 - 40$). Since this is below the new 24-second threshold, the evaluation stops immediately, holding both B and A in quarantine.

When Event C (ET:45) arrives at time 50, its lateness is 5 seconds, updating the sliding window to {2, 30, 5}. The average lateness drops to 12.3 seconds, and the dynamic quarantine time adjusts to 18.5 seconds. C is inserted into the buffer based on its event time, resulting in [B, A, C]. The system checks the first event in the queue, B. Its waiting duration is now 10 seconds ($50 - 40$), but this is still less than the adjusted 18.5-second threshold. Consequently, the system holds all events in quarantine.

When Event D (ET:15) arrives at time 55, its lateness is 40 seconds, expanding the sliding window to {2, 30, 5, 40}. The average lateness rises to 19.25 seconds, and the dynamic quarantine time increases to 28.8 seconds. D is sorted into its correct chronological position, updating the buffer to [B, D, A, C]. The system evaluates B again. Its waiting duration is 15 seconds ($55 - 40$), which remains below the 28.8-second threshold. Thus, all events continue to be held.

Finally, at the end of the stream, the policy releases all remaining events in the quarantine buffer. The system empties the queue, forwarding Event B, Event D, Event A, and Event C to the system in correct chronological order.

Overall, the Average Dynamic Policy successfully adapts its waiting window to manage severe latency spikes, preventing the drop of Event D that would have occurred using a fixed-time policy.

3.3.2 Dynamic p99-EMA Policy (p99-EMA)

The Dynamic p99-EMA Policy builds on the idea that the quarantine time should reflect not only the average behavior of the stream but also its extreme delays. Instead of relying on the average arrival delay, this policy computes the 99th percentile (p99) of the arrival delays observed in the sliding window of the most recent 500 events. The p99 value captures the extreme values in the delay distribution, representing the worst delays that occur in the stream but excluding extremely rare outliers. This makes the policy more sensitive to sudden spikes of disorder than the AVG policy, which tends to smooth such spikes away.

To avoid sudden changes in the quarantine time, the p99 value is then smoothed using an Exponential Moving Average (EMA) with a smoothing factor $\alpha = 0.1$. The EMA ensures that the quarantine time reacts to changes in the stream but does it gradually, preventing sudden jumps that could destabilize the system. The update rule is:

$$Q_t = (1 - \alpha) * Q_{t-1} + \alpha * P_{99},$$

where:

Q_t : is the new quarantine time,

Q_{t-1} : is the previous quarantine time,

P_{99} : is the 99th percentile of the latencies in the sliding window,

$\alpha = 0.1$ controls how quickly the policy adapts.

A lower α (like 0.1) means the quarantine time changes slowly, giving stability, while a higher α would make it more reactive but also more unstable. As with all dynamic policies, the p99-EMA policy applies a safety margin of 1.5, slightly increasing the computed quarantine time to protect against sudden spikes. It also enforces the user-defined minimum quarantine time and the global maximum of one million seconds. Out-of-order

events that violate temporal correctness are still dropped, ensuring that the output stream remains strictly ordered.

The p99-EMA policy is more robust than the AVG policy because it focuses on the high-delay of the stream rather than mean value. This allows it to better handle bursty or irregular delay patterns while still maintaining smooth and stable adaptation through the EMA.

To illustrate how the Dynamic p99-EMA Policy works, we apply it to the input stream of Table 3.1:

Min quarantine time: 5 sec.

Arrival Time	Input Stream Description	New Quarantine Time	Drop check	Quarantine before insertion	System Action	Quarantine after insertion	Last Sent Event
22	Arrive A (ET:20)	5 sec.	No drop	[]	Buffer A	[A]	-
40	Arrive B (ET:10)	5 sec.	No drop	[A]	Hold A, Buffer B	[B, A]	-
50	Arrive C (ET:45)	8.79 sec.	No drop	[B, A]	Release B, Buffer C	[A, C]	B (ET:10)
50				[A, C]	Release A	[C]	A (ET:20)
55	Arrive D (ET:15)		DROP D	[C]		[C]	A (ET:20)
	End of stream		-	[C]	Release C	[]	C (ET:45)

Events sent to the system: [Event B, Event A, Event C]

Dropped events: [Event D]

Table 3.4: Step-by-step evaluation of Dynamic p99-EMA Policy over the example input stream

When Event A (ET:20) arrives at time 22, its lateness is 2 seconds, which becomes the 99th percentile of the sliding window. The system applies the exponential moving average (EMA) with a 0.1 learning rate, resulting in an overall lateness of 0.2 seconds. After applying the 1.5 safety margin, the calculated quarantine time is 0.3 seconds. Because this is below the minimum allowed value, the dynamic quarantine time becomes 5 seconds.

Since the quarantine is initially empty, A is simply inserted into the buffer, and because its waiting duration is 0 seconds, it is held into the quarantine.

When Event B (ET:10) arrives at time 40, its lateness is 30 seconds. The sliding window sorts the latencies to {2, 30}, and the 99th percentile is 30. However, the EMA uses only the 10% of this spike, raising the smoothed overall lateness to 3.18 seconds. Multiplied by the 1.5 margin, the calculated threshold is 4.77 seconds, which still clamps to the 5-second minimum. B is sorted to the front of the queue, producing the buffer [B, A]. The system evaluates B, finds its waiting duration is 0 seconds, and holds all events.

When Event C (ET:45) arrives at time 50, its lateness is 5 seconds. The sorted latencies are {2, 5, 30}, and the 99th percentile remains 30. The EMA uses another 10% of the P99 value, raising the smoothed overall lateness to 5.86 seconds. The dynamic quarantine time finally exceeds the minimum, updating to 8.79 seconds. C is inserted into the buffer to form [B, A, C]. The system checks the first event, B. Its waiting duration is now 10 seconds ($50 - 40$), which exceeds the new 8.79-second threshold, so B is released. The system then evaluates A, whose duration is 28 seconds ($50 - 22$), also exceeding the threshold, so A is released as well. Event C is evaluated but held, leaving the buffer as [C].

When Event D (ET:15) arrives at time 55, the system first checks whether it violates temporal correctness. Because Event A (ET:20) was just released, the last event time is 20. Since D's event time (15) is smaller than 20, D is considered too late to be placed in the correct order and is immediately dropped. Because dropped events bypass the main logic, the sliding window and quarantine time remain unchanged.

Finally, at the end of the stream, the policy releases all remaining events in the quarantine buffer. The system empties the queue, forwarding C to the system for query evaluation.

3.3.3 Dynamic MAX-EMA Policy (MAX-EMA)

The Dynamic MAX-EMA Policy computes the quarantine time based on the maximum arrival delay observed in the sliding window of the most recent 500 events. Instead of using an average (AVG) or a high percentile (p99), this policy focuses entirely on the single worst delay in the window. The idea is that if the system recently observed a very late event, the quarantine time should increase enough to cover delays of that size and prevent early releases.

To avoid large jumps caused by sudden spikes in the maximum delay, the policy applies an Exponential Moving Average (EMA) with a smoothing factor $\alpha = 0.1$. The update rule is:

$$Q_t = (1 - \alpha) * Q_{t-1} + \alpha * \max(L_t),$$

where:

Q_t : is the updated quarantine time,

Q_{t-1} : is the previous quarantine time,

L_t : is the list of lateness values in the sliding window,

$\max(L_t)$: is the maximum lateness in the window,

$\alpha = 0.1$ controls how quickly the quarantine time adapts.

After the EMA update, the policy applies the same safety margin of 1.5 used in the other dynamic policies, ensuring that the quarantine time remains slightly larger than the observed maximum delay. It also enforces the user-defined minimum quarantine time and the global maximum of one million seconds. As with all policies, events that violate temporal correctness (events whose event time is smaller than the last event sent) are dropped immediately.

The MAX-EMA policy is the least aggressive among the approaches because it reacts to the single worst delay rather than a statistical measure like the mean or p99. This makes it highly robust to disorder but also tends to maintain large quarantine times for extended periods, especially if even one event in the window arrives extremely late. The EMA smoothing helps moderate this effect, but the policy still focuses to grow the quarantine time quickly and shrink it slowly.

To illustrate how the MAX-EMA Policy works, we apply it to the input stream of Table 3.1:

Min quarantine time: 5 sec.

Arrival Time	Input Stream Description	New Quarantine Time	Drop check	Quarantine before insertion	System Action	Quarantine after insertion	Last Sent Event
22	Arrive A (ET:20)	5 sec.	No drop	[]	Buffer A	[A]	-
40	Arrive B (ET:10)	5 sec.	No drop	[A]	Hold A, Buffer B	[B, A]	-

50	Arrive C (ET:45)	8.79 sec.	No drop	[B, A]	Release B, Buffer C	[A, C]	B (ET:10)
50				[A, C]	Release A	[C]	A (ET:20)
55	Arrive D (ET:15)		DROP D	[C]		[C]	A (ET:20)
	End of stream		-	[C]	Release C	[]	C (ET:45)

Events sent to the system: [Event B, Event A, Event C]

Dropped events: [Event D]

Table 3.5: Step-by-step evaluation of MAX-EMA Policy over the example input stream

When Event A (ET:20) arrives at time 22, its lateness is 2 seconds. Since it is the only event, the maximum lateness in the sliding window is 2. The system applies the exponential moving average (EMA) with a 0.1 learning rate, calculating an overall lateness of 0.2 seconds. After applying the 1.5 safety margin, the threshold is 0.3 seconds, which is clamped to the 5-second minimum. The quarantine is empty, so A is buffered and held because its waiting duration is 0 seconds.

When Event B (ET:10) arrives at time 40, its lateness is 30 seconds, which becomes the new maximum in the sliding window. However, the EMA uses the 10% of the new maximum, raising the smoothed overall lateness to 3.18 seconds. Multiplied by the 1.5 margin, the calculated threshold is 4.77 seconds, which still clamps to the 5-second minimum. Event B is sorted to the front of the queue, producing the buffer [B, A]. The system evaluates B, finds its waiting duration is 0 seconds, and holds all events.

When Event C (ET:45) arrives at time 50, its lateness is 5 seconds. The maximum in the window remains 30. The EMA uses the 10% of this maximum, raising the smoothed overall lateness to 5.86 seconds. The dynamic quarantine time now exceeds the minimum, updating to 8.79 seconds after applying the 1.5 safety margin. Event C is inserted, updating the buffer to [B, A, C]. The system checks the first event, B. Its waiting duration is 10 seconds (50 – 40), which exceeds the new 8.79-second threshold, so B is released. The system then evaluates A, whose duration is 28 seconds (50 – 22), also exceeding the threshold, triggering its release. Event C is evaluated but held, leaving the buffer as [C].

When Event D (ET:15) arrives at time 55, the system checks the last event time, which is now 20 (from Event A). Since D's event time (15) is smaller than 20, it is

immediately dropped to preserve temporal ordering. Because dropped events bypass the EMA calculation, the window and quarantine time remain unchanged.

Finally, at the end of the stream, the policy releases all remaining events, forwarding C to the system for query evaluation.

3.3.4 Dynamic MAX Policy (MAX)

The Dynamic MAX Policy determines the quarantine time by tracking the maximum observed lateness within a sliding window of recent events. For every incoming event, the policy computes its lateness as the difference between its arrival time and event time and inserts this value into the window. The quarantine time is then derived by selecting the maximum lateness in the window, multiplying it by the safety margin, and finally enforcing a minimum quarantine threshold to prevent overly aggressive behavior when delays are small. Because the policy reacts directly to the worst-case lateness rather than an average or percentile, it is highly sensitive to extreme delays: a single event with a large lateness is enough to significantly increase the quarantine time. This design makes the Dynamic MAX Policy highly responsive to worst-case delays, but it also causes the policy to expand the quarantine time whenever even a single extreme lateness value appears in the workload.

To illustrate the behavior of the Dynamic MAX Policy, we apply it to the same input stream used in previous examples. Each event arrives with a specific event time and arrival time, allowing us to compute its lateness and update the sliding window accordingly. At every step, the policy determines the quarantine time by selecting the maximum lateness observed in the window and multiplying it by the safety margin, while also enforcing the minimum quarantine threshold of 5 seconds. Based on this quarantine time, the policy decides whether buffered events should be released or held. The table below summarizes how the buffer changes, how the quarantine time evolves, and how the system acts as each event arrives:

Min quarantine time: 5 sec.

Arrival Time	Input Stream Description	New Quarantine Time	Drop check	Quarantine before insertion	System Action	Quarantine after insertion	Last Sent Event
22	Arrive A (ET:20)	5 sec.	No drop	[]	Buffer A	[A]	-

40	Arrive B (ET:10)	45 sec.	No drop	[A]	Hold A, Buffer B	[B, A]	-
50	Arrive C (ET:45)	45 sec.	No drop	[B, A]	Hold B, Hold A, Buffer C	[B, A, C]	-
55	Arrive D (ET:15)	60 sec.	No drop	[B, A, C]	Hold B, Hold A, Hold C, Buffer D	[B, D, A, C]	-
	End of stream		-	[B, D, A, C]	Release all	[]	C (ET:45)

Events sent to the system: [Event B, Event D, Event A, Event C]

Dropped events: []

Table 3.6: Step-by-step evaluation of MAX Policy over the example input stream

When Event A (ET:20) arrives at time 22, its lateness is 2 seconds, which becomes the maximum value in the sliding window. After applying the 1.5 safety margin, the calculated threshold is 3 seconds. Because this is below the minimum allowed value, the dynamic quarantine time becomes 5 seconds. Since the quarantine is initially empty, A is simply inserted into the buffer, and because its waiting duration is 0 seconds, it is held.

When Event B (ET:10) arrives at time 40, its lateness is 30 seconds. The sliding window becomes {2, 30}, and the new maximum lateness is now 30. Multiplied by the 1.5 margin, the dynamic quarantine time spikes instantly to 45 seconds. B is sorted to the front of the queue, producing the buffer [B, A]. The system evaluates B, finds its waiting duration is 0 seconds, and holds all events.

When Event C (ET:45) arrives at time 50, its lateness is 5 seconds. The latencies are {2, 30, 5}, and the maximum remains 30. Therefore, the dynamic quarantine time stays at 45 seconds. Event C is inserted into the buffer to form [B, A, C]. The system checks the first event, B. Its waiting duration is 10 seconds (50 – 40), which is below the 45-second threshold. Consequently, the system holds all events in quarantine.

When Event D (ET:15) arrives at time 55, its lateness is 40 seconds (55 – 15), expanding the sliding window to {2, 30, 5, 40}. The new maximum becomes 40, increasing the dynamic quarantine time even higher to 60 seconds. D is inserted into its correct

chronological position, updating the buffer to [B, D, A, C]. The system evaluates B again. Its waiting duration is 15 seconds, which remains below the 60-second threshold, so all events continue to be held.

Finally, at the end of the stream, the policy releases all remaining events in the quarantine buffer. The system empties the queue, forwarding Event B, Event D, Event A, and Event C in correct chronological order.

3.3.5 Dynamic Jump-and-Decay Policy (Jump/Decay)

The Dynamic Jump-and-Decay Policy is designed to react quickly to sudden spikes in system latency while gradually reducing the quarantine time when conditions improve. For each incoming event, the policy computes its lateness and inserts it into a sliding window of recent latencies. It then determines the maximum lateness in the window, representing the worst-case delay observed. If this maximum lateness exceeds the current quarantine time, the policy performs a jump, immediately raising the quarantine time to match the new maximum. Conversely, if the maximum lateness is lower than the current quarantine time, the policy applies a decay step, reducing the quarantine time smoothly using an exponential moving average with a fixed learning rate. This update rule allows the policy to respond aggressively to sudden disorder while avoiding sudden decreases that could drop out-of-order events. After computing the updated lateness estimate, the policy multiplies it by the safety margin and enforces a minimum quarantine threshold, ensuring that the system remains robust even when recent delays are small. The update rule is:

$$Q_t = \begin{cases} \max(L_t), & \text{if } \max(L_t) > Q_{t-1} \text{ (jump)} \\ (1 - \alpha)Q_{t-1} + \alpha \cdot \max(L_t), & \text{otherwise (decay)} \end{cases}$$

where,

L_t : is the list of lateness values in the sliding window,

$\max(L_t)$: is the maximum lateness observed in the sliding window,

Q_{t-1} : is the previous quarantine time,

Q_t : is the updated quarantine time,

α : is the decay learning rate that controls how quickly the quarantine time decreases when recent lateness values are smaller than the previous estimate. In the implementation: $\alpha = 0.1$.

To illustrate how the Jump-and-Decay Policy works, we apply it to the input stream of Table 3.1:

Min quarantine time: 5 sec.

Arrival Time	Input Stream Description	New Quarantine Time	Drop check	Quarantine before insertion	System Action	Quarantine after insertion	Last Sent Event
22	Arrive A (ET:20)	5 sec.	No drop	[]	Buffer A	[A]	-
40	Arrive B (ET:10)	45 sec.	No drop	[A]	Hold A, Buffer B	[B, A]	-
50	Arrive C (ET:45)	45 sec.	No drop	[B, A]	Hold B, Hold A, Buffer C	[B, A, C]	-
55	Arrive D (ET:15)	60 sec.	No drop	[B, A, C]	Hold B, Hold A, Hold C, Buffer D	[B, D, A, C]	-
	End of stream		-	[B, D, A, C]	Release all	[]	C (ET:45)

Events sent to the system: [Event B, Event D, Event A, Event C]

Dropped events: []

Table 3.7: Step-by-step evaluation of Jump-and-Decay Policy over the example input stream

When Event A (ET:20) arrives at time 22, its lateness is 2 seconds. Because this maximum (2) is greater than the initial overall lateness (0), the policy triggers a "Jump", instantly setting the overall lateness to 2. After applying the 1.5 safety margin, the 3-second threshold is clamped to the 5-second minimum. The quarantine is empty, so A is buffered and held because its waiting duration is 0 seconds.

When Event B (ET:10) arrives at time 40, its lateness is 30 seconds. This lateness is identified as the new window maximum. Since 30 is greater than the current overall lateness (2), the policy triggers another Jump, bypassing the EMA smoothing and sets the overall lateness to 30. Multiplied by the 1.5 margin, the dynamic quarantine time spikes

immediately to 45 seconds. B is sorted to the front of the queue, producing the buffer [B, A]. The system evaluates B, finds its waiting duration is 0 seconds, and holds all events.

When Event C (ET:45) arrives at time 50, its lateness is 5 seconds. The maximum in the window remains 30. Because the maximum (30) is not greater than the current overall lateness (30), the policy triggers a "Decay" using the EMA formula. However, because the max is still 30, the decay calculation $((1 - 0.1) * 30) + (0.1 * 30)$ evaluates exactly to 30. The quarantine time stays at 45 seconds. Event C is inserted, updating the buffer to [B, A, C]. The system checks the first event, B. Its waiting duration is 10 seconds, which is below the 45-second threshold, so all events are held.

When Event D (ET:15) arrives at time 55, its lateness is 40 seconds (55 – 15). The new window maximum becomes 40. Because 40 is greater than the current overall lateness (30), the policy Jumps again and increase the dynamic quarantine time to 60 seconds. D is sorted into its correct chronological position, updating the buffer to [B, D, A, C]. The system evaluates B again. Its waiting duration is 15 seconds, which remains below the 60-second threshold, so it is not released like all the other events in the buffer.

Finally, at the end of the stream, the policy releases all remaining events, forwarding Event B, Event D, Event A, and Event C in correct chronological order.

3.3.6 Dynamic Per-Event Policy (Per-Event)

The Dynamic Per-Event Policy extends the adaptive behavior of the Jump-and-Decay mechanism by computing a separate quarantine time for each event type independently. Instead of maintaining a single quarantine time for the entire stream, the policy keeps a dedicated sliding window of recent latencies for each event type and updates the corresponding quarantine time individually. This allows the system to adapt to workloads in which different event types may experience different network delays, arrival rates, or characteristics. For each incoming event, the policy updates the lateness estimate associated with its type using the Jump-and-Decay rule and then computes a type-specific quarantine time by applying the safety margin and minimum threshold. During release, each buffered event is compared against the quarantine time corresponding to its own type, enabling satisfactory control over disorder handling and reducing unnecessary waiting for event types that have low or stable lateness.

To illustrate the behavior of the Dynamic Per-Event Policy, we extend the original input stream by assigning each event to one of two event types. Events A and D belong to Event

Type 1, while events B and C belong to Event Type 2. Each type maintains its own lateness window and computes its own dynamic quarantine time independently.

Input stream			
Event Type	Event	Event Time	Arrival Time
1	Event A	20	22
2	Event B	10	40
2	Event C	45	50
1	Event D	15	55

Table 3.8: Example input stream of events with their event type, event time and arrival time

Min quarantine time: 5 sec.

Arrival Time	Input Stream Description	New Quarantine Time	Drop check	Quarantine before insertion	System Action	Quarantine after insertion	Last Sent Event
22	Arrive A (T1, ET:20)	T1: 5 sec. T2: 5 sec.	No drop	[]	Buffer A	[A]	-
40	Arrive B (T2, ET:10)	T1: 5 sec. T2: 45 sec.	No drop	[A]	Hold A, Buffer B	[B, A]	-
50	Arrive C (T2, ET:45)	T1: 5 sec. T2: 45 sec.	No drop	[B, A]	Hold B, Hold A, Buffer C	[B, A, C]	-
55	Arrive D (T1, ET:15)	T1: 60 sec. T2: 45 sec.	No drop	[B, A, C]	Hold B, Hold A, Hold C, Buffer D	[B, D, A, C]	-
	End of stream		-	[B, D, A, C]	Release all	[]	C (ET:45)

Events sent to the system: [Event B, Event D, Event A, Event C]

Dropped events: []

Table 3.9: Step-by-step evaluation of Dynamic Per-Event Policy over the example input stream

When Event A (Type 1, ET:20) arrives at time 22, its lateness is 2 seconds. The policy updates the overall lateness for Type 1, triggering a "Jump" to 2. After applying the 1.5 safety margin, Type 1's threshold is clamped to the 5-second minimum. The quarantine is empty, so A is buffered and held.

When Event B (Type 2, ET:10) arrives at time 40, its lateness is 30 seconds. The policy updates the overall lateness for Type 2 and jumps to 30. Multiplied by the safety margin, Type 2's threshold increases to 45 seconds, while Type 1's threshold remains at 5 seconds. Because B's event time (10) is older than A's (20), B is sorted to the front of the global queue, producing the buffer [B, A]. The system evaluates B using Type 2's threshold. Since its waiting duration is 0 seconds, event B is held in quarantine buffer like all the other events.

When Event C (Type 2, ET:45) arrives at time 50, its lateness is 5 seconds. The Type 2 window maximum remains 30, so its threshold stays at 45 seconds using the Jump and Decay formula. Event C is inserted into the buffer. The system checks the first event, B. Its waiting duration is 10 seconds, which is below the 45-second threshold, so all events remain at the quarantine.

When Event D (Type 1, ET:15) arrives at time 55, its lateness is 40 seconds (55 – 15). The policy updates the Type 1 overall lateness, triggering a Jump that increases Type 1's threshold to 60 seconds. Event D is sorted into its chronological position, resulting in [B, D, A, C]. The system evaluates B again. Its 15-second duration is still below the 45-second threshold, keeping the entire queue unchanged.

Finally, at the end of the stream, the policy releases all remaining events, forwarding Event B, Event D, Event A, and Event C in chronological order.

This chapter introduced a complete set of quarantine policies, starting from simple static thresholds to advanced adaptive mechanisms, closing with the Per-Event Dynamic Policy. By isolating lateness behavior per event type and adapting independently to the characteristics of each stream, this final policy provides the most precise and effective handling of disorder among all approaches presented. In the next chapter, all policies, both existing and newly proposed, will be evaluated under diverse workload conditions to demonstrate how they behave in practice and to highlight the performance advantages of the Per-Event Dynamic Policy across heterogeneous and dynamic environments.

CHAPTER 4

DATASET ANALYSIS

To thoroughly evaluate the quarantine policies presented in the previous chapters, we must first understand the data streams used in our experiments. This chapter introduces and analyzes the three workloads selected for our evaluation. We explain the domain specific characteristics of each dataset, their structural properties, and how we modified them to simulate realistic stream conditions. By examining their unique lateness patterns, event-type distributions, and disorder profiles, we establish a clear baseline. This comprehensive analysis provides the necessary context for the experimental evaluation that follows in Chapter 5.

4.1 Dataset Overview

To test and evaluate the new dynamic quarantine policies under different conditions, we chose three different datasets. These datasets come from three completely different areas: fire department incidents, commercial flights, and cryptocurrency transactions. This variety helps us understand how the algorithms perform with different data distributions, event speeds, and arrival delays.

Every event in these datasets has two main timestamps: the *Event Time* when the event actually happened in the real world and the *Arrival Time* when the event reached our system. Apart from these two timestamps, each dataset contains other columns with specific information and data for its domain. These extra columns are used for filtering and partitioning the data.

It is very important to note that all three CSV files are sorted by their *Arrival Time*. This is because in real-world stream processing, a system like CORE receives data sequentially as it arrives at the infrastructure. Due to network delays, internet problems, or system bottlenecks, events often arrive out of chronological order.

In this context, we classify an event as *disordered* when its Event Time is smaller than the Event Time of the previous event in the stream. For example, if we check a row with Event Time = 5 and the previous row had Event Time = 6, then this event is

disordered because it normally should have arrived before the event with timestamp 6. The "Mean Disorder Delay" shows how far back in time these late events usually are. Finally, the term *Event Types* means the different categories or classes of events inside the dataset (such as different fire departments, airline companies, or crypto tokens). These event types show how the data can be split up and processed at the same time by the CORE engine.

It is important to highlight that all three original datasets were modified in various ways to fit the specific requirements of our evaluation framework. In their original form, these datasets did not contain a system arrival timestamp. Therefore, we modified the Arrival Time attribute into every dataset based on realistic network latency distributions. This allowed us to control and simulate different levels of stream disorder for our experiments. Additionally, we performed further data modifications and formatting adjustments—such as cleaning missing values, structuring the event types properly, and aligning column names—to ensure that all files could be parsed and processed correctly by the CORE engine.

Dataset	Number of Events	Types of Events	Disordered Events (%)	Mean Disorder Delay
Fires Incidents	563.683	4	12.84%	3.95 min
Aviation	1.005.411	14	23.13%	42.39 min
Crypto	1.756.833	8	41.08%	39.16 min

Table 4.1: Structural and Disorder Characteristics of the Evaluation Datasets

4.2 Detailed Dataset Analysis

4.2.1 Fires Dataset

The first dataset analyzed in our evaluation is the "Fires" dataset, which contains records of fire department incidents and emergency calls [12]. This dataset is used as our baseline workload because it represents a scenario with moderate data disorder and relatively small time delays. Apart from the temporal fields, the dataset includes several columns that describe each incident in detail. These columns include the *Fire_Department_Name* (which identifies which of the 4 departments reported the event), *Incident_Street_Address*,

Incident_City, and *Incident_Zip_Code*. As mentioned in the overview, we modified the *Arrival_Timestamp* column alongside the original *Event_Timestamp*. This modification allows us to simulate realistic network latencies and evaluate how effectively the CORE engine can reorder the out-of-order stream using different quarantine policies.

To get an initial understanding of the dataset's size and total disorder, we look at the high-level summary metrics.

Metric	Value
Total Events	563.683
Disordered Events	72.396
Disordered Percentage	12,84%
Mean Disorder Delay	237,12 sec (3,95 min)

Table 4.2: Summary metrics table for the Fires dataset

As shown in Table 4.2, the dataset consists of a total of 563,683 events. Out of these, 72,396 events are classified as disordered because they arrived at the system out of chronological order. This creates a total disorder percentage of 12.84%. When an event arrives late, the average time distance from its original occurrence (Mean Disorder Delay) is 237.12 seconds, which is equal to 3.95 minutes.

Next, we examine how these out-of-order events are divided among the 4 different fire departments (event types) in the stream.

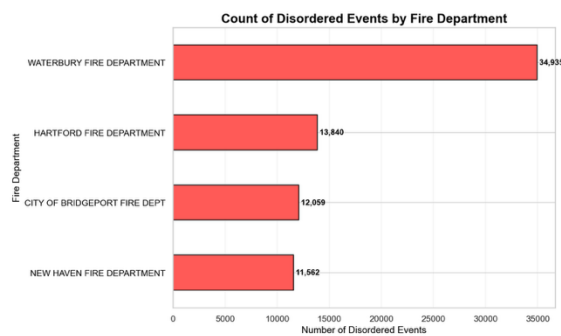


Figure 4.1: Count of disordered events for each fire department

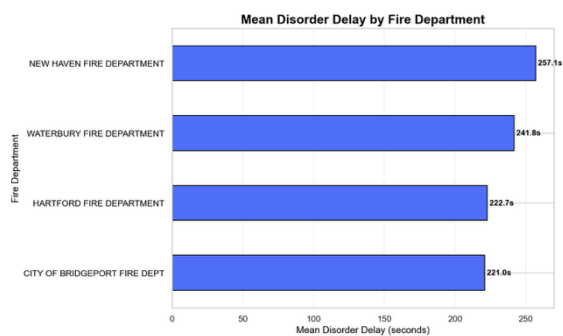


Figure 4.2: Mean disorder delay in seconds for each fire department

Figure 4.1 demonstrates that data disorder is not spread evenly across the workload. The *Waterbury Fire Department* is responsible for most out-of-order data, showing 34,935 disordered events. The other three departments show much lower and very similar levels of disorder, ranging closely between 11,562 and 13,840 events.

Interestingly, Figure 4.2 reveals that the average delay time is quite stable across all departments, sitting between 221.0 and 257.1 seconds. An important observation here is that the *New Haven Fire Department* has the lowest number of out-of-order events (11,562) but exhibits the highest average delay per event (257.1 seconds). Conversely, *City of Bridgeport Fire Dept* features the lowest average delay at 221.0 seconds.

To analyze how much time passes between when an event happens in the real world and when it arrives at our system, we look at the system arrival delay distribution.

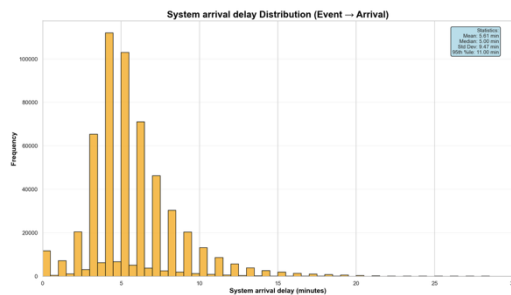


Figure 4.3: System arrival delay distribution histogram for the Fires dataset

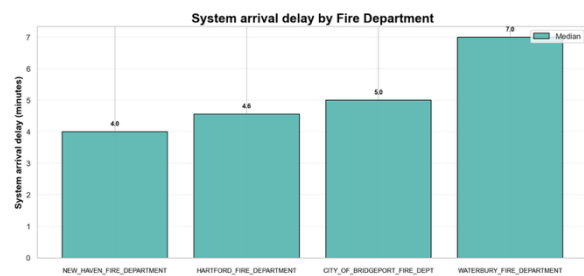


Figure 4.4: Median system arrival delay comparison across the four fire departments

As illustrated in Figure 4.3, the distribution graph is right-skewed. This shows that for most events, the delay is short and stable. Most events cluster between 4 and 7 minutes, and the median system arrival delay is exactly 5.00 minutes. However, there is a long tail of slower events on the right side of the chart. These larger delays increase the statistical mean to 5.61 minutes and create a 95th percentile of 11.00 minutes, meaning that 95% of all events arrive within 11 minutes.

Furthermore, Figure 4.4 confirms that different fire departments face different arrival latencies in the stream. The *Waterbury Fire Department* has the highest median delay at 7.0 minutes, which explains why it has the most disordered data. The *City of Bridgeport Fire Dept* follows with a median of 5.0 minutes. *Hartford Fire Department* has a median of 4.6 minutes, while *New Haven Fire Department* features the fastest delivery with a median delay of 4.0 minutes.

4.2.2 Aviation Dataset

The second dataset used in our evaluation is the "Aviation" dataset, which contains commercial flight operations and performance data [13]. Compared to the Fires dataset, this workload represents a significantly more challenging testing scenario characterized by a much higher volume of events, data disorder, and substantially longer time delays. Besides the timestamps, this dataset contains several columns with real flight information. The columns include the *CARRIER_NAME* (identifying the airline company), *OP_CARRIER_FL_NUM* (the flight number), *ORIGIN* and *DEST* airport codes, as well as performance metrics like *AIR_TIME*, *DISTANCE*, *ARR_DELAY*, and whether a flight was *CANCELLED*. To ensure compatibility and seamless processing by the CORE engine, the original arrival time column was renamed to *Arrival_Timestamp*, while the flight's initial departure time column was mapped to *Event_Timestamp*. We also cleaned missing entries and standardized the formatting so the records could be correctly evaluated during partitioned and filtered CEQL query execution.

To evaluate the scale and overall disorder profile of this dataset, we review its high-level summary metrics

Metric	Value
Total Events	1.005.411
Disordered Events	232.528
Disordered Percentage	23,13 %
Mean Disorder Delay	2543,64 sec (42,39 min)

Table 4.3: Summary metrics table for the Aviation dataset

As shown in Table 4.3, the dataset contains over one million records, with a total of 1,005,411 events. A substantial portion of these flights specifically 232,528 events arrived out of chronological order, leading to a high disorder percentage of 23.13%. Furthermore, late-arriving events suffer from severe latencies, featuring a Mean Disorder Delay of 2543.64 seconds, which translates to 42.39 minutes.

To understand how these out-of-order events are distributed across the 14 different airline carriers (event types), we analyze the individual stream characteristics

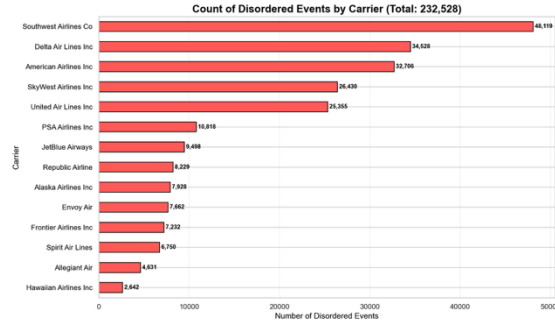


Figure 4.5: Count of disordered events for each airline carrier

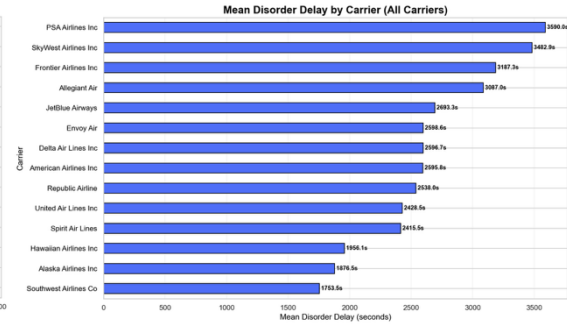


Figure 4.6: Mean disorder delay in seconds for each airline carrier

Figure 4.5 and Figure 4.6 show an interesting contrast in data disorder across the 14 carriers, which makes this dataset highly valuable for our experiments. On one hand, Figure 4.5 shows that *Southwest Airlines Co* dominates the out of order volume with 48,119 disordered events. On the other hand, Figure 4.6 reveals an inverse trend regarding the actual delay times: *Southwest* features the lowest mean delay (1753.5 seconds), while carriers with fewer disordered events, like *PSA Airlines Inc*, exhibit the longest waiting times, reaching up to 3590.0 seconds. This combination of high-frequency short delays and low-frequency long delays provides a rich diversity of behaviors. It makes the Aviation dataset an excellent and realistic dataset to see how flexible and robust the dynamic quarantine policies are when facing completely different disorder patterns within the same stream.

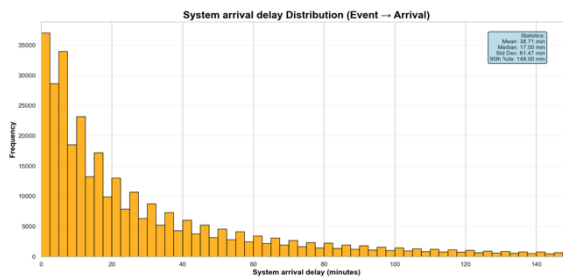


Figure 4.7: System arrival delay distribution histogram for the Aviation dataset

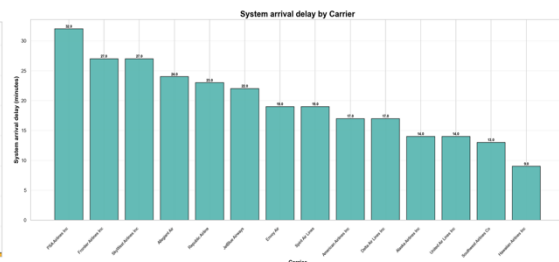


Figure 4.8: Median system arrival delay comparison across the airline carriers

Figure 4.7 and Figure 4.8 present another layer of diversity that makes this dataset a perfect for testing. While Figure 4.7 shows a heavily skewed distribution where most flights arrive with small delays (median at 17.0 minutes), it also contains extreme delays

that push the 95th percentile to 148.0 minutes. This unpredictable delay behavior varies significantly among event types, as shown in Figure 4.8, where carriers like *PSA Airlines Inc* experience high median delays (32.0 minutes), while others like *Hawaiian Airlines Inc* present very low latencies (9.0 minutes). Testing our algorithms against this combination of predictable short-term delays and sudden, extreme late arrivals is highly valuable. It allows us to verify if the dynamic quarantine policies can adapt to different delay distributions without wasting system memory or dropping critical historical events.

4.2.3 Crypto Dataset

The third and final workload examined in our evaluation is the "Crypto" dataset, which contains cryptocurrency transaction events observed across different blockchain networks and trading platforms [14]. This dataset represents the most extreme and challenging testing scenario in our evaluation framework due to its massive event volume, very high disorder percentage, and highly volatile delay characteristics. Besides the core timestamps, this dataset includes technical transaction attributes such as *TxSize* (transaction size), *TxFee* (transaction fee), and inputs/outputs (*TxIn*, *TxOut*). It also tracks blockchain-specific progress fields like *ExitBlock*, *BlocksToConfirm*, *SecSinceLB*, and *Exchange* (the platform observing the trade). Just like the previous datasets, we modified the original column *ExitTime* to a custom *Arrival_Timestamp* column and the *EntryTime* to the *Event_Timestamp* column based on network distributions, while clean-up routines were performed to map the 8 distinct crypto assets as correct event types for sequential sorting in the CORE engine.

To evaluate the sheer scale and heavy disorder profile of this network, we review its high-level summary metrics.

Metric	Value
Total Events	1.756.833
Disordered Events	721.651
Disordered Percentage	41,08 %
Mean Disorder Delay	2349,89 sec (39,16 min)

Table 4.4: Summary metrics table for the Crypto dataset

As shown in Table 4.4, the dataset comprises 1,756,833 events, making it the largest workload tested. From these records, 721,651 events arrived out of order, which is a very high disorder percentage of 41.08%. The delay is also very high, featuring a Mean Disorder Delay of 2349.89 seconds, which equals 39.16 minutes.

To analyze how out-of-order events behave across the 8 different crypto assets (event types), we inspect their individual stream statistics.

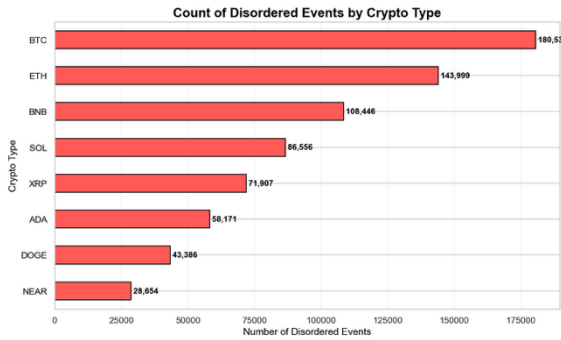


Figure 4.9: Count of disordered events for each crypto asset type

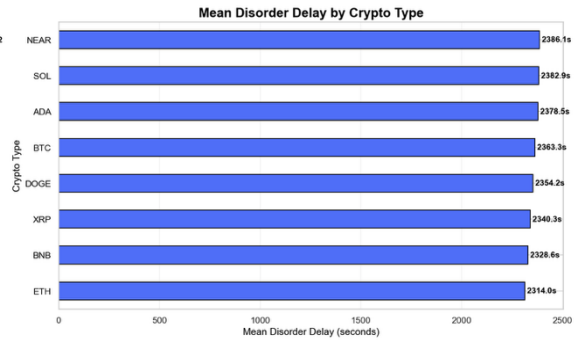


Figure 4.10: Mean disorder delay in seconds for each crypto asset type

Figure 4.9 and Figure 4.10 reveal a highly unique behavioral pattern that contrasts with the Aviation dataset, adding an excellent form of diversity to our experiments. Figure 4.9 shows that the total volume of out-of-order data is heavily unbalanced. *BTC* has the highest number of disordered events with 180,532 records, followed closely by *ETH* at 143,999, while *NEAR* has the lowest at 28,654. However, Figure 4.10 shows that the actual delay times are completely flat and uniform across all assets, tightly clustering between 2314.0 seconds (*ETH*) and 2386.1 seconds (*NEAR*). This specific combination of heavily unbalanced event volumes but highly uniform out-of-order delays makes this dataset very valuable. It allows us to test how well the algorithms handle streams where every event type feels the same constant network lag, but some types generate much heavier traffic than others.

We examine the overall system arrival delay distribution histogram alongside the median values for each asset.

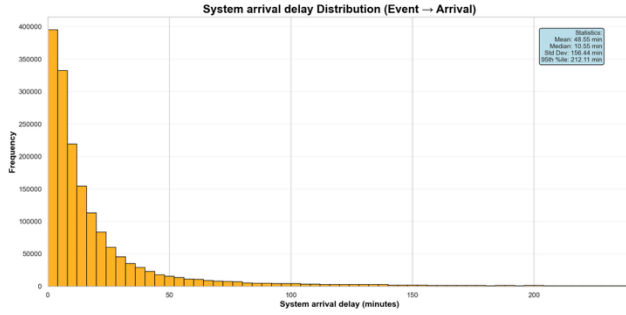


Figure 4.11: System arrival delay distribution histogram for the Crypto dataset

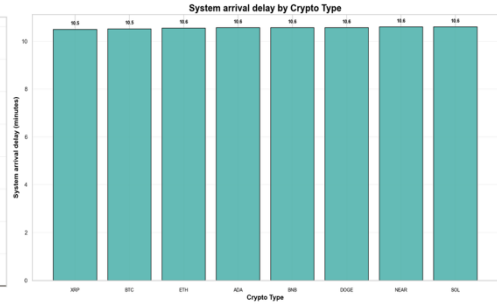


Figure 4.12: Median system arrival delay comparison across the 8 crypto asset types

Figure 4.11 and Figure 4.12 highlight the final layer of complexity in this dataset. As illustrated in Figure 4.11, the distribution is extremely right skewed and highly volatile. The vast majority of transaction messages arrive quickly, leading to a small median delay of 10.55 minutes. However, a very long tail of extreme delays stretches past 200 minutes, which inflates the mean delay to 48.55 minutes and creates a huge standard deviation of 156.44 minutes. The 95th percentile sits at 212.11 minutes, meaning that the quarantine layer must be capable of tracking data across roughly 3.5 hours of real-world latency. This extreme uniform lag is clearly proven by Figure 4.12, where all 8 crypto types share almost the exact same median arrival delay of 10.5 to 10.6 minutes.

In summary, these three datasets provide a complete and diverse testing environment that represents three different levels of difficulty for our experiments. The Fires dataset serves as our baseline scenario with a smaller event volume, low disorder (12.84%), and short, predictable delays. The Aviation dataset increases the complexity by introducing a larger volume, more out-of-order data (23.13%), and long delays that vary significantly across different airline carriers. Finally, the Crypto dataset acts as our extreme stress-test, presenting the largest overall volume of events, severe stream disorder (41.08%), and highly volatile but uniform delays. By testing our quarantine policies across these three distinct profiles, we can thoroughly evaluate their behavior and verify the correctness, memory efficiency, and processing speed of our algorithms under both typical and extreme real-world streaming conditions.

CHAPTER 5

EVALUATION OF QUARANTINE POLICIES

Having established the full set of quarantine policies in the previous chapters and analyzed the evaluation workloads in Chapter 4, this chapter evaluates their behavior under a wide range of workload conditions. The goal is to examine how policies respond to different lateness patterns, event-type distributions, and to measure their impact on latency, disorder handling, event drops, and complex-event detection. By applying all policies across these workloads, we highlight their strengths and limitations. This evaluation provides an in-depth understanding of the performance of each quarantine mechanism in realistic scenarios, emphasizing the advantages of the Per-Event Dynamic Policy across diverse conditions.

5.1 Overview of Experiments

To evaluate system performance across different workloads, each dataset was processed using a structured set of three query experiments. These experiments were designed to progressively increase query complexity, allowing us to observe how different each query affect execution time, resource usage, and overall system behavior. The three query patterns used in all datasets are summarized below:

- **Experiment 1 – Simple Query**
 - Retrieves only raw events and their timestamps
 - No partitions, filters, or additional operators
 - Serves as the baseline for measuring fundamental system performance
- **Experiment 2 – Partitioned Query**
 - Uses PARTITION BY to group events into logical subsets
 - Adds moderate structural complexity
 - Evaluates how partitioning affects processing cost and latency
- **Experiment 3 – Partitioned & Filtered Query**
 - Combines partitioning with filtering conditions
 - Represents the most realistic analytical workload
 - Measures performance under the most complex query configuration

This experimental design shows a clear comparison of how increasing query complexity effects performance across all datasets, providing a structured approach for evaluating the system's behavior.

In the analysis that follows, each dataset is evaluated across all three query types, beginning in every case with Experiment 2, the medium complexity query. This ordering is chosen because the medium query provides the most representative balance between basic and advanced query characteristics, making it the most informative for examining system behavior. For this experiment, we present a full comparison of all newly introduced dynamic quarantine policies to highlight their differences under realistic workloads. In contrast, Experiments 1 and 3 focus only on the top three performing policies, ensuring that the evaluation remains concise while still capturing the most meaningful performance details across different levels of query complexity.

In addition, each experiment compares the newly introduced dynamic quarantine policies against the existing strategies. For every query, the evaluation includes a consistent set of performance graphs:

- Dropped events
- Results (complex events)
- Execution time
- Detection delay (average, median, 95th-percentile, standard deviation)
- The percentage of results relative to the ideal case, where no events arrive out of order and no drops occur
- The total number of items stored in quarantine
- The total memory used, expressed in MB, of quarantine buffer size.

This set of metrics provides a comprehensive view of how each policy behaves under different query complexities and datasets.

5.2 Detailed Analysis of the Fires Dataset

5.2.1 Fires Dataset: Medium-Complexity Query Analysis

The medium complexity query for the Fires dataset detects pairs of events where an incident reported by the CITY_OF_BRIDGEPORT_FIRE_DEPT is followed by an incident from the NEW_HAVEN_FIRE_DEPARTMENT within a 500-second window, based on their

```
SELECT * FROM S
WHERE CITY_OF_BRIDGEPORT_FIRE_DEPT; NEW_HAVEN_FIRE_DEPARTMENT
PARTITION BY [Arrival_Timestamp]
WITHIN 500 [Event_Timestamp]
CONSUME BY NONE
```

The following figures present the full set of performance metrics for this query. Each graph highlights a different aspect of system behavior under the selected quarantine policies. These graphs provide a complete view of how the policies perform when processing this medium-complexity query on the Fires dataset.

Dropped Events - Comparison Across Policies

Y-axis: Number of Dropped Events (0 to 1600). X-axis: Quarantine Time (s) (1 to 16384).

Legend:

- MAX (window max)
- EMA on max ($\alpha=0.1$)
- Jump-and-Decay ($\alpha=0.1$)
- Mean (window average)
- Individual quarantine per event type

Quarantine Time (s)	MAX (window max)	EMA on max ($\alpha=0.1$)	Jump-and-Decay ($\alpha=0.1$)	Mean (window average)	Individual quarantine per event type
1	1659	1659	1659	1659	108
2	1659	1659	1659	1659	108
3	1659	1659	1659	1659	108
6	1659	1659	1659	1659	108
10	1659	1659	1659	1659	108
50	1659	1659	1659	1659	108
64	1659	1659	1659	1659	108
128	1659	1659	1659	1659	108
256	1659	1659	1659	1659	108
512	1550	1550	1550	1550	108
1024	480	480	480	480	108
2048	180	180	180	180	108
4096	50	50	50	50	108
8192	20	20	20	20	108
16384	24	24	24	24	24

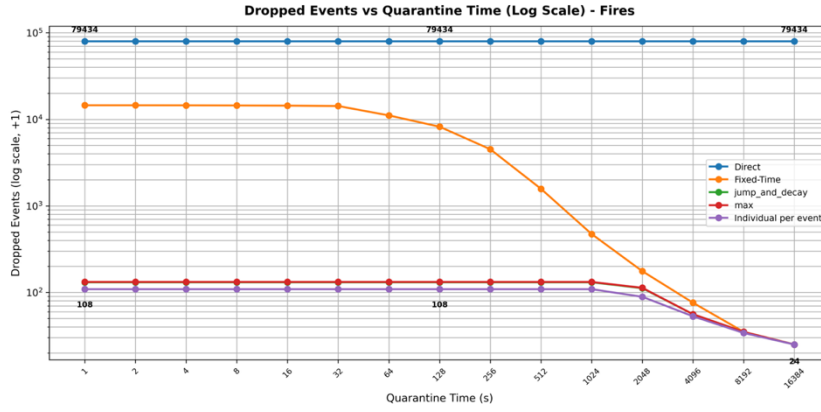
Dropped Events - Key Policies Comparison

Y-axis: Number of Dropped Events (20 to 140). X-axis: Quarantine Time (s) (1 to 16384).

Legend:

- MAX (window max)
- EMA on max ($\alpha=0.1$)
- Individual quarantine per event type

Quarantine Time (s)	MAX (window max)	EMA on max ($\alpha=0.1$)	Individual quarantine per event type
1	133	133	108
2	133	133	108
3	133	133	108
6	133	133	108
10	133	133	108
50	133	133	108
64	133	133	108
128	133	133	108
256	133	133	108
512	133	133	108
1024	133	133	108
2048	110	110	88
4096	55	55	55
8192	33	33	33
16384	24	24	24



5.2.1.2 Complex Events

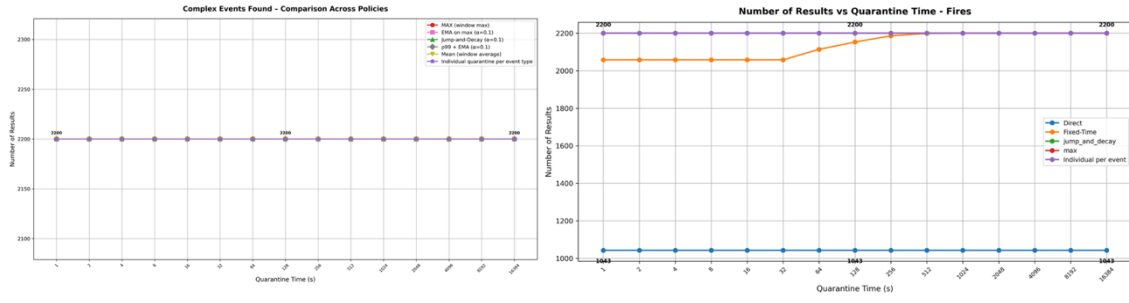


Figure 5.2: Complex events for the medium complexity Fires query. (a) All dynamic policies. (b) Comparison between the top dynamic policies and the existing baseline policies.

The impact of the dropped events is directly reflected in the system's ability to successfully detect complex events, as showed in the subfigures above. The dynamic policies demonstrate perfect reliability, consistently identifying all 2.200 complex events regardless of the selected quarantine time. In contrast, the baseline approaches severely compromise the overall system output. The Direct policy detects less than half of the expected complex events (only 1.043), while the Fixed-Time policy misses some results until the quarantine window is increased above 256 seconds. This difference occurs because complex event detection relies on the successful capture of multiple individual events. Baseline policies often drop events because they arrive in incorrect position or additional quarantine time is required. When this happens, the sequence breaks and the system miss the complex event completely. This shows the importance of dynamic policies to find these patterns.

5.2.1.3 Execution Time

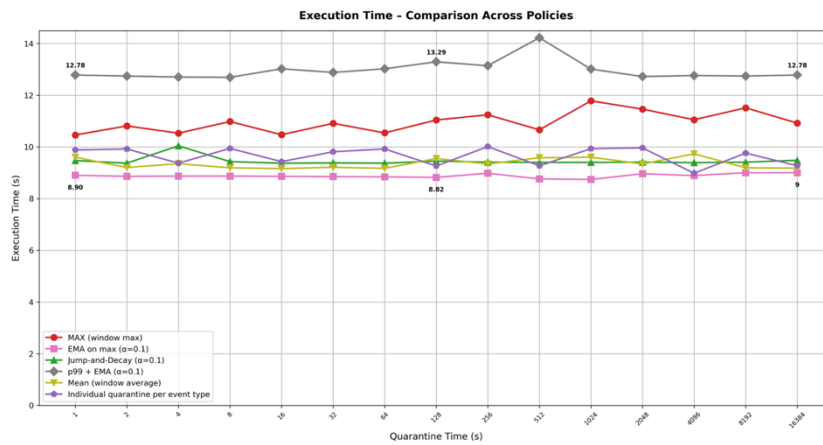




Figure 5.3: Execution time for the medium complexity Fires query. (a) All dynamic policies. (b) Top dynamic policies versus existing baseline policies.

The Execution Time results for the Fires dataset show that all policies have not significant differences, with only some exceptions as the quarantine window increases. The Direct policy remains the fastest, since it performs no buffering and processes events immediately. The Fixed-Time and Jump and Decay policies also achieve low execution times even though their buffering process. In contrast, max and p-99 EMA policies consistently exhibit the highest execution time, due to the policy's behavior of waiting for the maximum quarantine time before releasing events. The Individual per event policy is between these limits. Even though it adds additional cost from maintaining separate per type buffers, it still remains within the overall 9–11.5 s range. Overall, the Fires dataset demonstrates that even the more complex buffering strategies add only moderate computational overhead, with all policies maintaining stable execution times across all quarantine windows.

5.2.1.4 Detection Delay

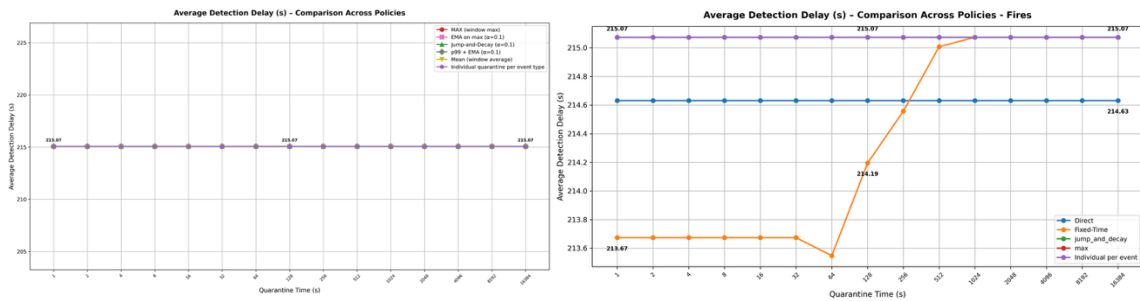


Figure 5.4: Average detection delay for the medium complexity Fires query. (a) All dynamic policies. (b) Top dynamic policies versus existing baseline policies.

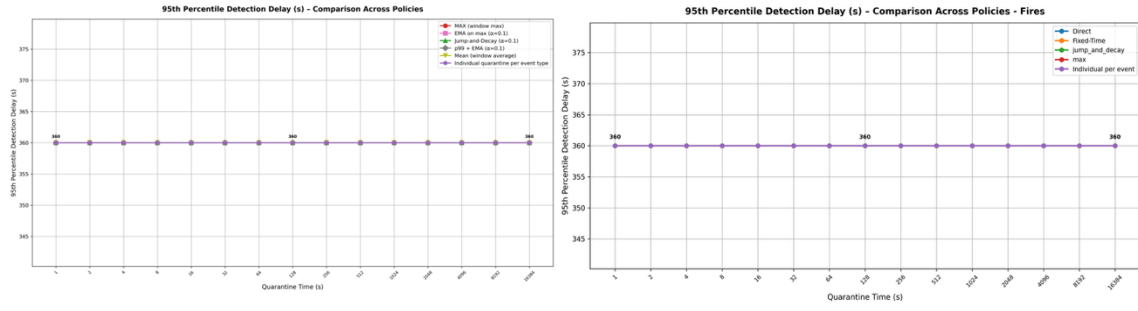


Figure 5.6: 95th-percentile detection delay for the medium complexity *Fires* query. (a) All dynamic policies. (b) Top dynamic policies versus existing baseline policies.

The 95th-percentile detection delay figures show that all policies produce the same high-percentile delay across every quarantine time, with all remaining at 360 seconds. This indicates that the 95% of the detections are completely unaffected by the choice of quarantine policy or the size of the quarantine window. The reason this happens is that even the slowest detections are determined by the temporal structure of the *Fires* query, rather than by buffering or reordering effects introduced by the policies. Although different policies may reorder or delay a few events internally, these differences are too small to affect the 95th percentile. As a result, the figures clearly show that the detection delay is invariant across policies, meaning that quarantine mechanisms do not impact detection performance.

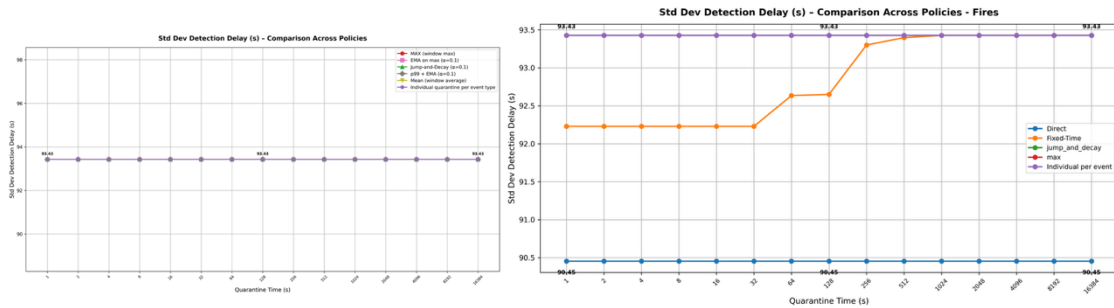


Figure 5.7: Standard deviation detection delay for the medium complexity *Fires* query. (a) All dynamic policies. (b) Top dynamic policies versus existing baseline policies.

The standard deviation detection delay figures show that almost all policies maintain stable variability across all quarantine times. The only noticeable deviation comes from the Fixed-Time policy, which gradually increases as the quarantine window grows. This difference occurs because most policies apply similar buffering and event release behavior, so the spread of detection delays remains unchanged. Direct stays constant because it processes events immediately and introduces no additional timing. Fixed-Time increases as larger quarantine windows naturally cause events to remain in quarantine for longer periods. Overall, the figures show that the internal mechanics of each policy,

have minimal impact on the standard deviation of detection delay, with most policies producing nearly identical delays.

For the next experiments, we focus on the 95th percentile detection delay, since it is the most informative of the detection delay metrics. The 95th percentile captures the behavior of the slowest detections, making it the most sensitive metric for revealing meaningful differences between policies when they occur. Because it represents the upper tail of the delay distribution, it is the most appropriate metric for evaluating worst case performance, and it provides a clearer picture of how each policy behaves under more demanding conditions.

5.2.1.5 Comparison with ideal results

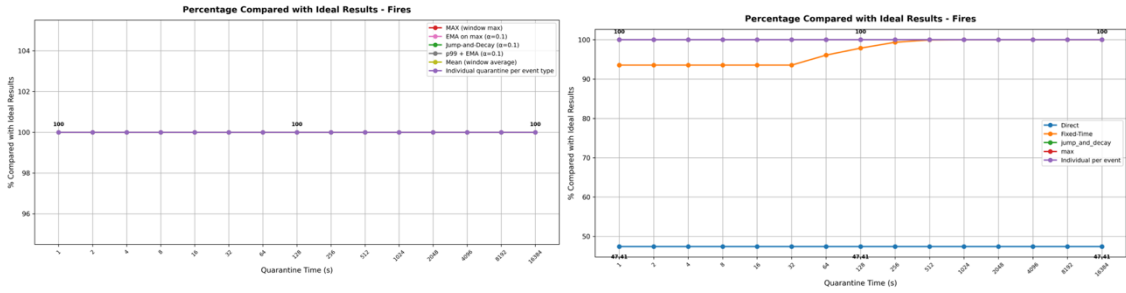


Figure 5.8: Percentage of results recovered compared with the ideal output for the medium complexity Fires query. (a) All dynamic policies. (b) Top dynamic policies versus existing baseline policies.

The percentage of ideal results figures show that all dynamic policies achieve perfect correctness at all quarantine times. Dynamic policies remain at 100% for the entire range, demonstrating that they fully recover the ideal number of results regardless of buffering duration. In contrast, Direct policy performs significantly worse, recovering less than half of the ideal results, while Fixed-Time policy gradually improves from 90% to 100% as the quarantine window increases. Overall, the figures clearly show that dynamic policies are highly robust and maintain ideal correctness, while baseline policies lose results when the quarantine times are too small.

5.2.1.6 Maximum Events in Quarantine and Quarantine Size

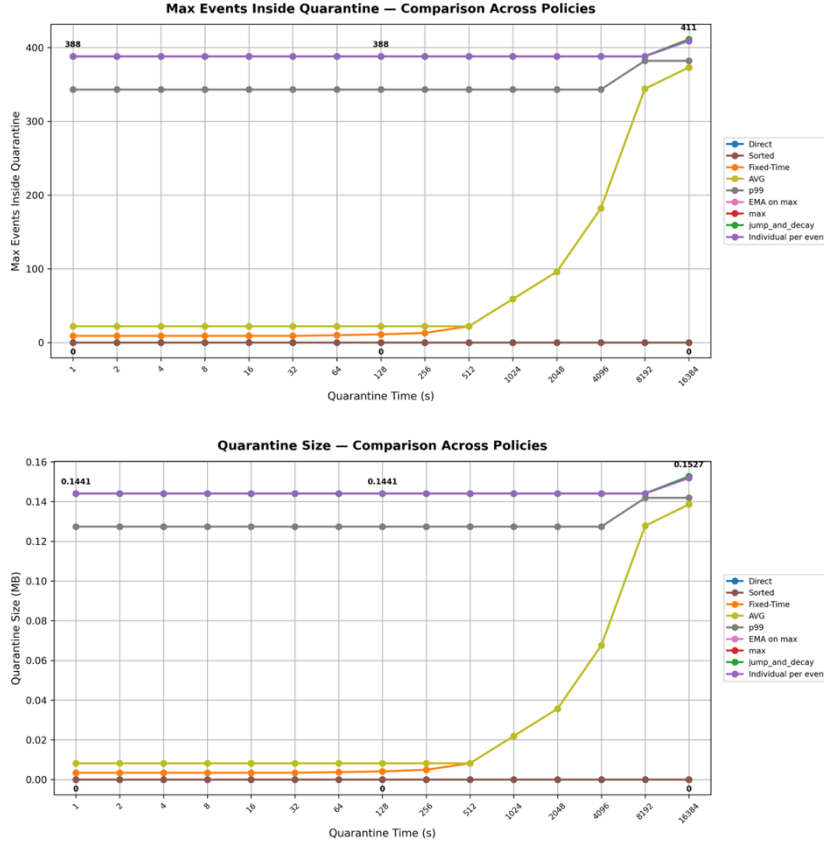


Figure 5.9: (a) Max events inside quarantine across all policies. (b) Quarantine size (MB) across all policies.

The Max Events Inside Quarantine and Quarantine Size figures highlight clear differences in memory behavior across the evaluated policies. Direct and Sorted¹ maintain zero buffered events for all quarantine times, as they never store events and therefore, they do not add memory overhead. In contrast, policies such as max, p99, and EMA on max store many events, resulting in consistently high quarantine sizes across all window durations. The AVG policy also shows a noticeable increase for large windows, reflecting the fact that averaging over longer intervals leads to greater quarantine sizes. The Individual per event policy exhibits similarly high memory usage, since maintaining a separate quarantine per event type leads to large buffers in the Fires dataset. Fixed-Time increases with larger windows because it holds every event for the full duration of the quarantine interval. Overall, the figures show that only a few policies remain memory efficient, while others add substantial buffering overhead as quarantine time increases, but those policies also achieve better results.

¹ This refers to the stream fully ordered by event timestamp used to compute the ideal results.

5.2.2 Fires Dataset: Simple Query Analysis

The low-complexity query for the Fires dataset detects pairs of events where an incident reported by the CITY_OF_BRIDGEPORT_FIRE_DEPT is followed by an incident from the NEW_HAVEN_FIRE_DEPARTMENT within a 500-second window, based on their event timestamps. Unlike the medium-complexity query, this query does not apply any partitioning, meaning that the entire event stream is evaluated sequentially as a single timeline. The query uses CONSUME BY NONE, meaning that all events remain available for matching, allowing multiple pairs to be detected.

```
SELECT * FROM S
WHERE CITY_OF_BRIDGEPORT_FIRE_DEPT; NEW_HAVEN_FIRE_DEPARTMENT
WITHIN 500 [Event_Timestamp]
CONSUME BY NONE
```

Query 5.2: Simple complexity Fires query

The following figures present the performance metrics for this query for the top three quarantine policies compared with the existing quarantine policies. Each graph highlights a different aspect of system behavior under the selected quarantine policies. These graphs provide a complete view of how the policies perform when processing this simple-complexity query on the Fires dataset.

5.2.2.1 Dropped Events

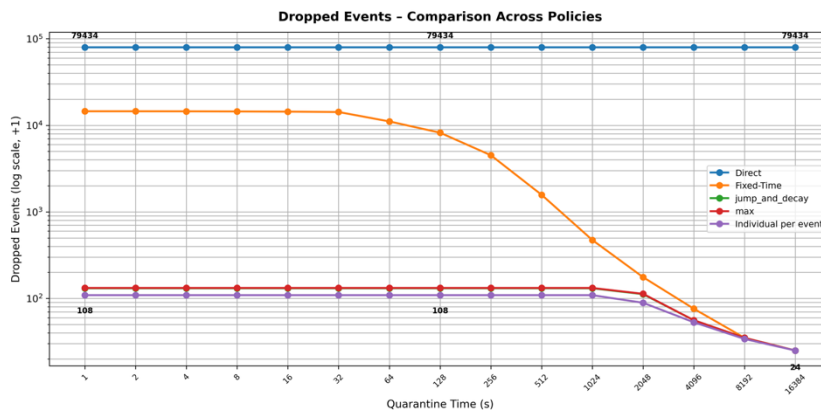


Figure 5.10: Dropped Events - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

The graph shows how the number of dropped events varies under different quarantine policies for the low-complexity Fires query, revealing the limitations of baseline approaches compared to dynamic policies. As shown most clearly, the Direct policy has many data losses dropping 79,434 events regardless of the quarantine window, while the Fixed-Time policy also has many losses at smaller quarantine times. Conversely, the dynamic policies, specifically Individual quarantine per event type, Jump-and-Decay and MAX, demonstrate excellent results. They maintain a stable, low drop rate between 108 and 132 events across most quarantine times, with the Individual per event type policy consistently outperforming the rest.

5.2.2.2 Complex Events

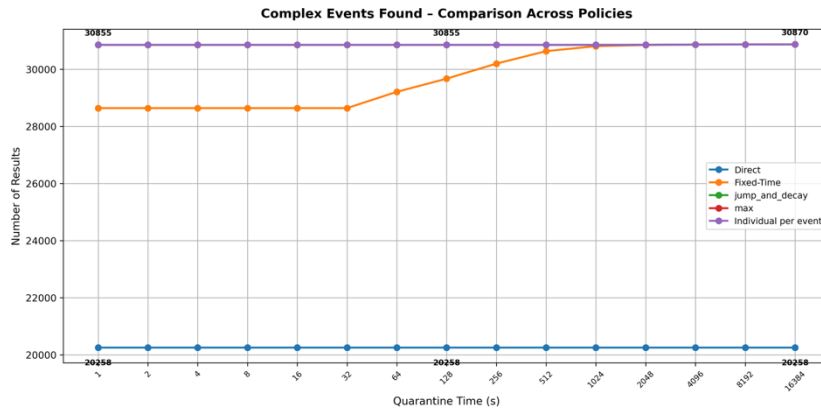


Figure 5.11: Complex Events - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

The graph shows how the number of detected complex events changes under different quarantine policies, highlighting a direct link between dropped events and system accuracy. The Direct policy performs the worst by far, finding only 20,258 results. Because it does not wait for late events at all, it misses more than 10,000 valid matches. The standard Fixed-Time policy performs better but is still slow to recover results at lower quarantine windows. In contrast, all the advanced dynamic policies (MAX, Jump-and-Decay, and Individual per event) perform exceptionally well for lower quarantine times. Their lines overlap perfectly at the top of the chart, capturing 30,855 results even with a tiny 1-second quarantine window. Because these policies adapt to the stream latency immediately, they secure almost 100% accuracy right away, and they all eventually converge to the absolute maximum of 30,870 results at the highest quarantine times.

5.2.2.3 Execution Time

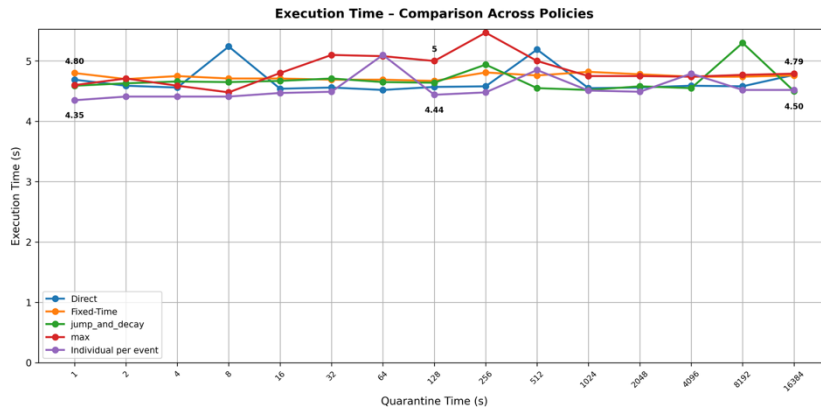


Figure 5.12: Execution Time - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

As shown, the computational overhead remains remarkably stable across all evaluated strategies. All policies from the simplest Direct policy to the most advanced Individual per event policy cluster between 4.35 and 5.5 seconds. The small fluctuations are minor. This confirms that implementing smarter quarantine logic or holding events longer does not affect the performance of the system, proving that our dynamic solutions are highly efficient.

5.2.2.4 Detection Delay

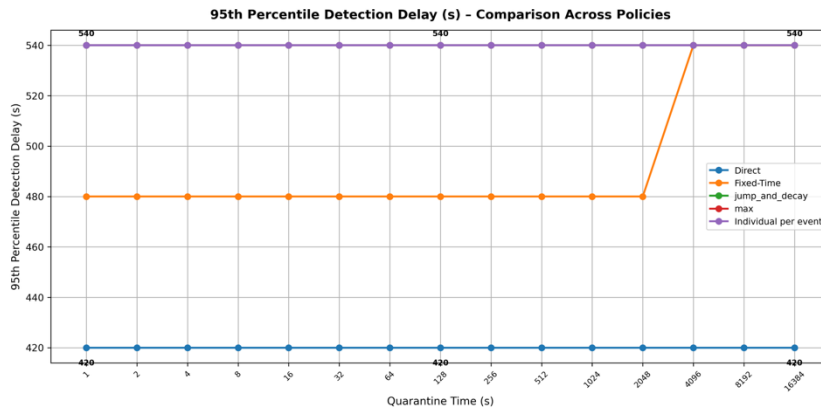


Figure 5.13: 95th Percentile Detection Delay - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

As shown, the Direct policy maintains the lowest delay at exactly 420 seconds because it processes events instantly without waiting, though this causes high data loss. The Fixed-Time policy stays at 480 seconds for smaller quarantine windows and jumps to 540 seconds once the quarantine time reaches 4096 seconds or more. In contrast, all the

advanced dynamic policies overlap perfectly and maintain a stable delay of 540 seconds across all time windows. Although these dynamic policies have a slightly higher delay around 60 seconds more than Fixed-Time at lower windows. This stable delay allows them to immediately achieve near-perfect accuracy and capture almost all correct results.

5.2.2.5 Comparison with ideal results

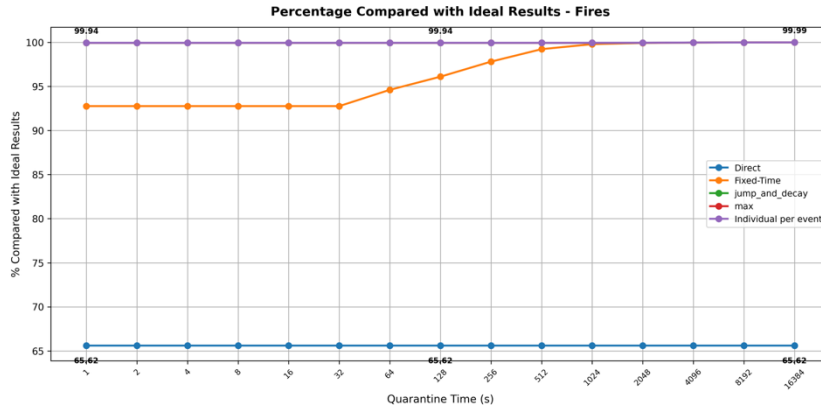


Figure 5.14: Percentage Compared with Ideal Results - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

As shown, the Direct policy achieves only 65.62% accuracy because it ignores late-arriving data. The standard Fixed-Time policy starts higher at 92.72% but needs a static quarantine window of 1024 seconds or more to reach 100% correctness. In contrast, all dynamic secure an immediate 99.94% accuracy from a 1-second window and eventually reaching 99.99%.

5.2.3 Fires Dataset: Complex Query Analysis

The complex query for the Fires dataset detects correlated incidents between the CITY_OF_BRIDGEPORT_FIRE_DEPT and the NEW_HAVEN_FIRE_DEPARTMENT, but only when each department reports an event in a specific ZIP code (06608 for Bridgeport and 06513 for New Haven). The evaluation is partitioned by arrival timestamp, and the query requires both events to occur within a 500-second window based on their event timestamps. As with the medium complexity query, the use of *CONSUME BY NONE* ensures that all events remain available for matching, allowing multiple overlapping pairs to be identified. However, the additional filtering and the need to correlate two distinct

event types under temporal constraints increase the query's complexity and sensitivity to disorder.

```
SELECT * FROM S
WHERE (CITY_OF_BRIDGEPORT_FIRE_DEPT as COBFD;
NEW_HAVEN_FIRE_DEPARTMENT as NHFD)
FILTER COBFD[Incident_Zip_Code='06608'] AND NHFD[Incident_Zip_Code='06513']
PARTITION BY [Arrival_Timestamp]
WITHIN 500 [Event_Timestamp]
CONSUME BY NONE
```

Query 5.3: Complex Fires query

The following figures present the full set of performance metrics for this complex query. Each graph highlights a different aspect of system behavior under the selected quarantine policies. These visualizations provide a complete view of how the policies perform when processing this complex query on the Fires dataset.

5.2.3.1 Dropped Events

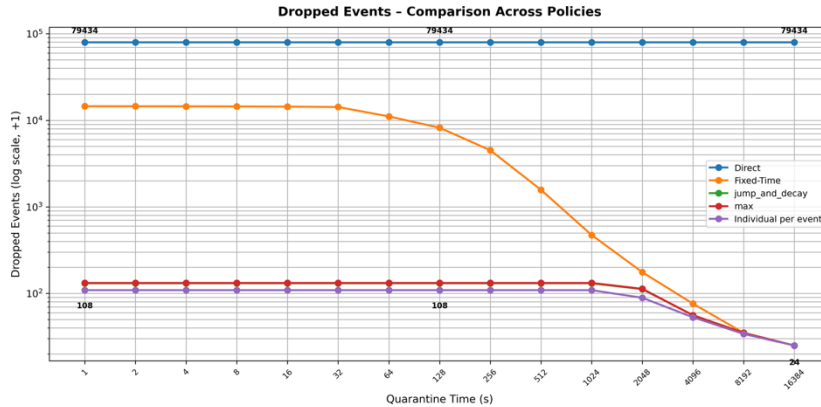


Figure 5.15: Dropped Events - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.15 presents the number of dropped events for the complex Fires query across all quarantine policies. The Direct policy drops the highest number of events, remaining constant at approximately 79.434 across all window sizes. The Fixed-Time policy shows a clear decreasing trend: it begins with around 10^4 dropped events and steadily reduces as the quarantine window increases. In contrast, the Max, Jump-and-Decay, and Individual Per Event policies maintain consistently low dropped event counts around 10^2 independent of the window size. Their stable behavior indicates that these dynamic strategies retain enough events to reconstruct most valid sequences even under disorder. Overall, the figure highlights the strong dependence of dropped events on the buffering

strategy, with dynamic policies exhibiting significantly fewer drops compared with the baseline policies.

5.2.3.2 Complex Events

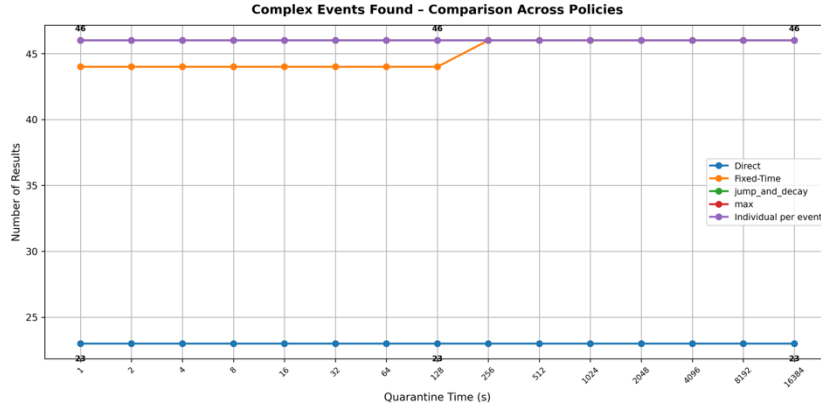


Figure 5.16: Complex Events - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.16 presents the number of complex events detected across all quarantine policies. The Direct policy consistently produces the lowest number of results, remaining at 23 complex events regardless of the quarantine window. The Fixed-Time policy identifies around 44 results for most window sizes and increases to 46 as the window becomes sufficiently large. The Individual Per Event policy achieves the maximum number of results 46 across all windows, demonstrating its robustness under disorder. The Max and Jump-and-Decay policies also converge to the same upper bound of 46 results, indicating that these dynamic strategies successfully retain the necessary events to construct all valid matches. Overall, the figure shows that adaptive policies consistently recover the full set of complex event sequences, while baseline policies, especially Direct, fail to do so due to their limited buffering capabilities.

5.2.3.3 Execution Time

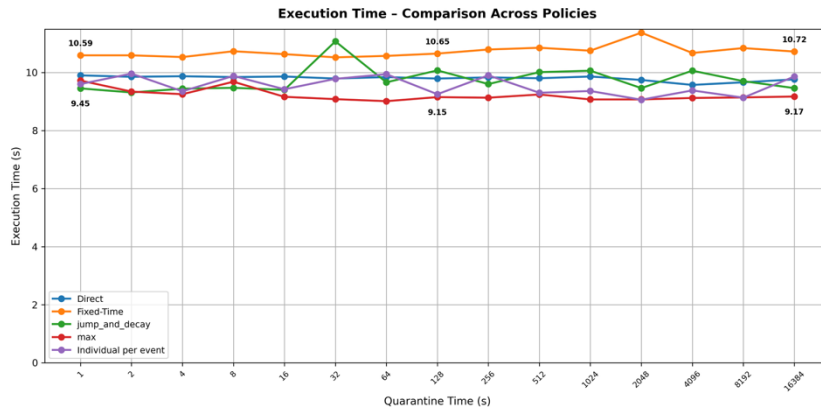


Figure 5.17: Execution Time - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.17 presents the execution time of the complex Fires query across all quarantine policies. The Fixed-Time policy consistently exhibits the highest execution times, reaching values above 10 seconds for larger windows due to its need to buffer increasingly more events of the stream. In contrast, the Max policy achieves the lowest execution times, around 9 seconds, demonstrating that its adaptive strategy efficiently limits unnecessary buffering. The Jump-and-Decay and Individual Per Event policies follow a similar trend, maintaining execution times close to the lower bound while scaling smoothly with the quarantine window. The Direct policy remains stable across all window sizes, as it performs no buffering at all. Overall, the figure shows that dynamic policies not only preserve more events but also maintain competitive execution times, whereas Fixed-Time exhibits the highest overhead as the quarantine window grows.

5.2.3.4 Detection Delay

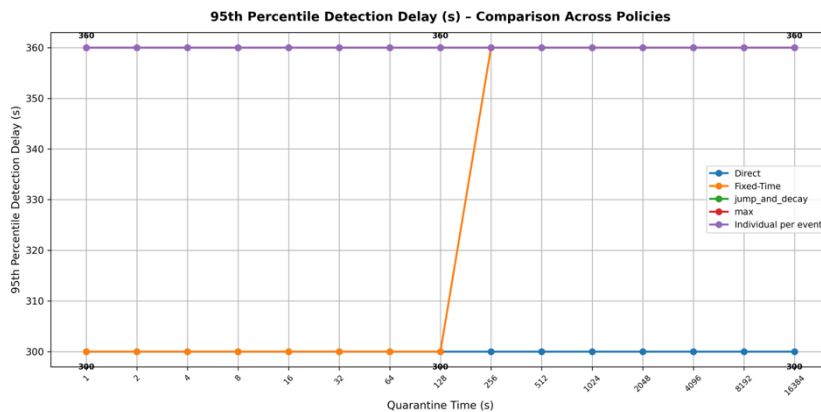


Figure 5.18: 95th Percentile Detection Delay - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.18 presents the 95th percentile detection delay for the complex Fires query across all quarantine policies. The Direct policy maintains a constant detection delay of 300 seconds, reflecting its lack of buffering. The Fixed-Time policy initially matches this delay for small windows but increases to 360 seconds once the quarantine window exceeds 128 seconds, after which it remains stable. The Individual per Event consistently exhibits the maximum delay of 360 seconds across all window sizes, and the other dynamic policies follow the same pattern, matching this upper bound due to their similar buffering behavior. Overall, the figure shows that detection delay is primarily determined by how much reordering each policy performs, with more extensive buffering naturally resulting in higher delays.

5.2.3.5 Comparison with ideal results

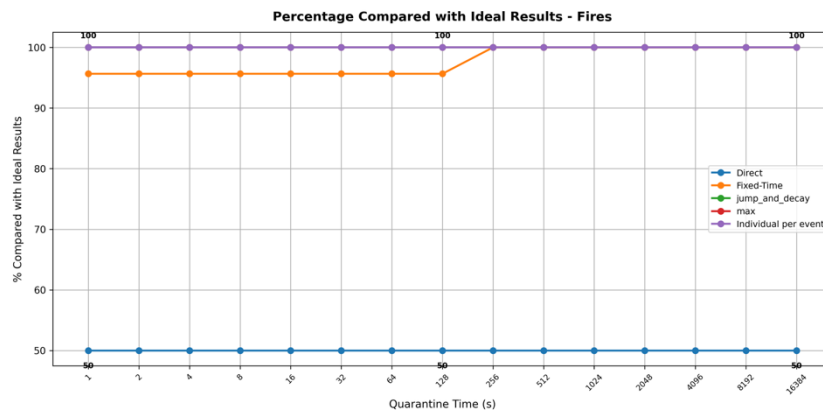


Figure 5.19: Percentage Compared with Ideal Results - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.19 presents the percentage of detected complex events relative to the ideal results across all quarantine policies. The Direct policy consistently achieves only 50% of the ideal output, reflecting its inability to handle disorder. The Fixed-Time policy performs substantially better, maintaining around 95% for most window sizes and reaching 100% once the quarantine window becomes sufficiently large. The Individual Per Event policy achieves perfect performance (100%) across all windows, demonstrating its robustness under varying disorder levels. The dynamic policies, Max and Jump-and-Decay, align with this upper bound as well, matching the ideal results due to their similar buffering behavior. Overall, the figure shows that adaptive strategies are capable of fully reconstructing the correct set of complex events, while baseline approaches, especially Direct, fall significantly short.

5.3 Detailed Analysis of the Aviation Dataset

5.3.1 Aviation Dataset: Medium-Complexity Query Analysis

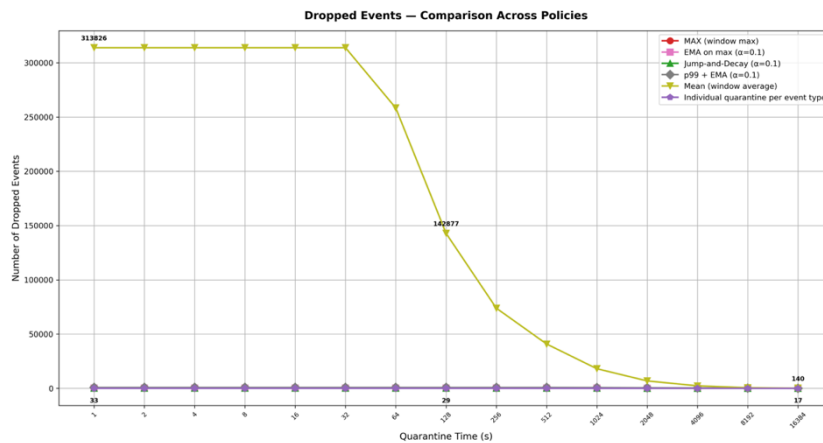
The medium complexity query for the Aviation dataset detects pairs of events where a flight event from Alaska Airlines is followed by a flight event from JetBlue Airways within 50 seconds, based on their event timestamps. The evaluation is partitioned by the ORIGIN airport, and the query uses CONSUME BY NONE, which means that events remain available for multiple matches, allowing overlapping pairs to be detected.

```
SELECT * FROM S
WHERE (Alaska_Airlines_Inc as alaska; JetBlue_Airways as jetblue)
PARTITION BY [ORIGIN]
WITHIN 50 [Event_Timestamp]
CONSUME BY NONE
```

Query 5.4: Medium complexity Aviation query

The following figures present the full set of performance metrics for this query. Each graph highlights a different aspect of system behavior under the selected quarantine policies. These graphs provide a complete view of how the policies perform when processing this medium complexity query on the Aviation dataset.

5.3.1.1 Dropped Events



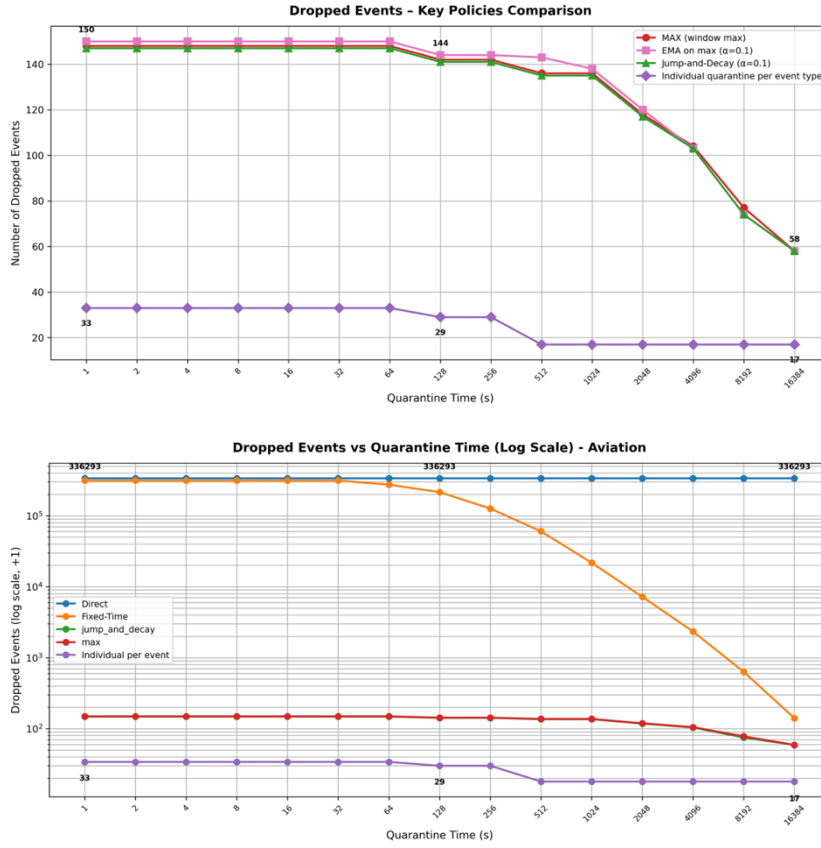


Figure 5.20: Dropped events for the medium complexity Aviation query. (a) All dynamic policies. (b) Key dynamic policies. (c) Comparison between the top dynamic policies and the existing baseline policies.

The dropped events for the Aviation query reveal clear differences in how each quarantine policy handles disorder in the dataset. The Direct policy performs the worst, consistently dropping more than 330,000 events regardless of quarantine time, since it performs no buffering and therefore cannot recover out-of-order events. Fixed-Time begins at the same level but steadily improves as the quarantine window increases, showing that larger windows allow it to recover a significant number of previously lost events. Dynamic policies behave far more efficiently. Max, EMA on max, and Jump-and-Decay all start with around 150 dropped events and gradually reduce this number as the quarantine window grows, demonstrating that they adapt well to the disorder characteristics of the dataset. The Individual per event policy performs the best overall, consistently dropping the fewest events (around 30) and improving slightly with larger windows. This is because it maintains separate disorder buffers per event type (airline), allowing it to recover out-of-order events more effectively in this partitioned workload. Overall, the Aviation results show that dynamic policies outperform baseline policies, and that Individual per event provides the highest correctness by minimizing dropped events across all quarantine times.

5.3.1.2 Complex Events

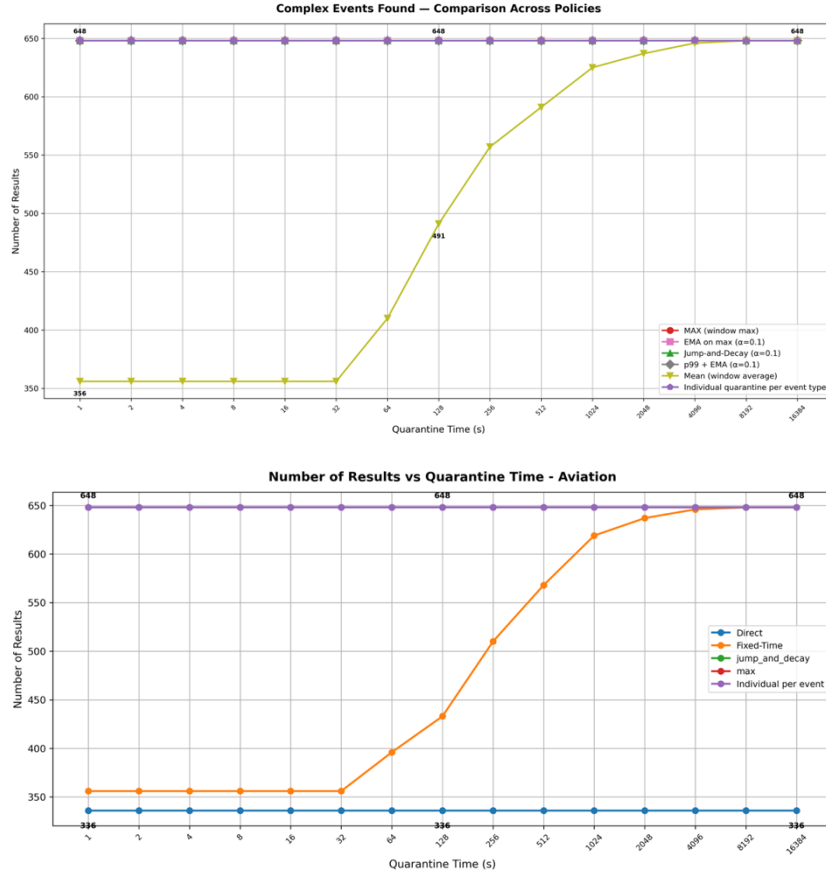


Figure 5.21: Results for the medium-complexity Aviation query. (a) All dynamic policies. (b) Comparison between the top dynamic policies and the existing baseline policies.

The Complex Events Found results for the Aviation dataset show that most dynamic policies consistently recover the full set of approximately 648 results across all quarantine times. These policies successfully reorder the out-of-order events even with very small windows, demonstrating strong correctness. In contrast, the baseline policies perform noticeably worse. Direct finds only 336 complex events, since it performs no buffering at all. Fixed-Time improves steadily with larger windows, starting near 350 results and reaching the 648 only when the quarantine window becomes sufficiently large. Overall, these results show that dynamic policies are highly effective at recovering all complex events, while the Mean policy and baseline strategies require large windows or fail entirely to achieve ideal correctness.

5.3.1.3 Execution Time

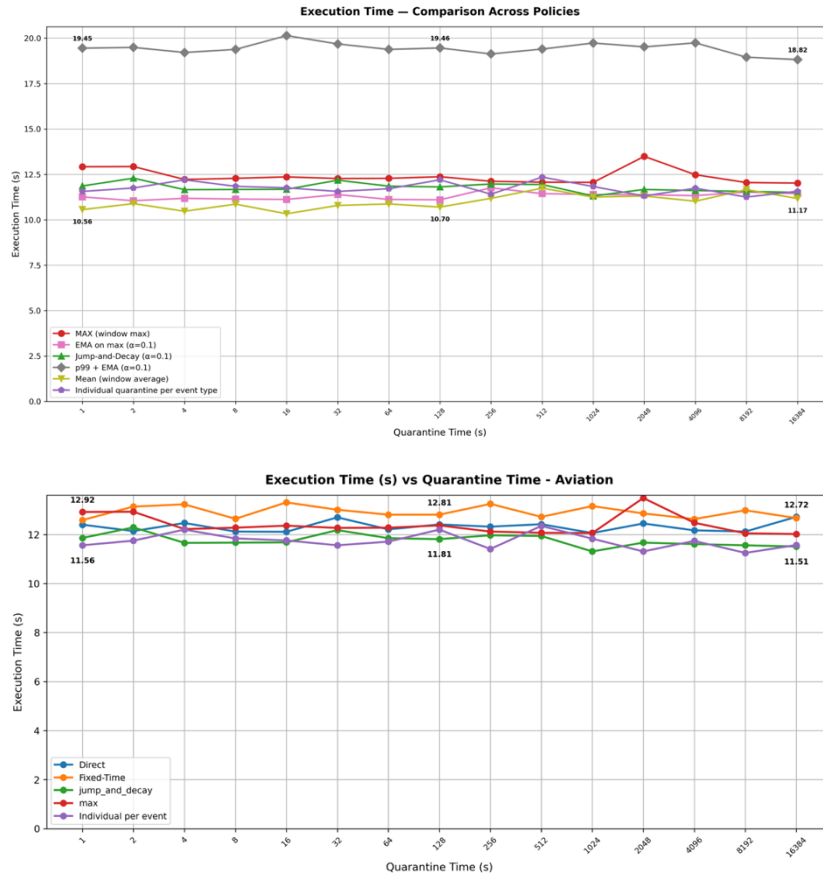


Figure 5.22: Execution time for the medium complexity Aviation query. (a) All dynamic policies. (b) Top dynamic policies versus existing baseline policies.

The execution time results for the Aviation query show that all policies exhibit stable and nearly constant performance across the entire range of quarantine times. The dynamic policies (Max, Jump-and-Decay, Mean, and Individual per event) cluster tightly between 10 and 12 seconds, indicating that their computational overhead is independent of the quarantine window. The only exception is the p99 + EMA policy, which consistently shows the highest execution times, around 18–20 seconds, due to the additional time it requires for the percentile estimation. Direct and Fixed-Time policies remain stable between 11.5 and 13 seconds, with no significant differences as the quarantine window increases. This demonstrates that the Aviation workload does not add additional computational time as buffering grows, and that the system's execution cost is affected by event-processing logic rather than quarantine size. Overall, the results show that execution time is highly stable for all policies, both baseline and dynamic.

5.3.1.4 Detection Delay

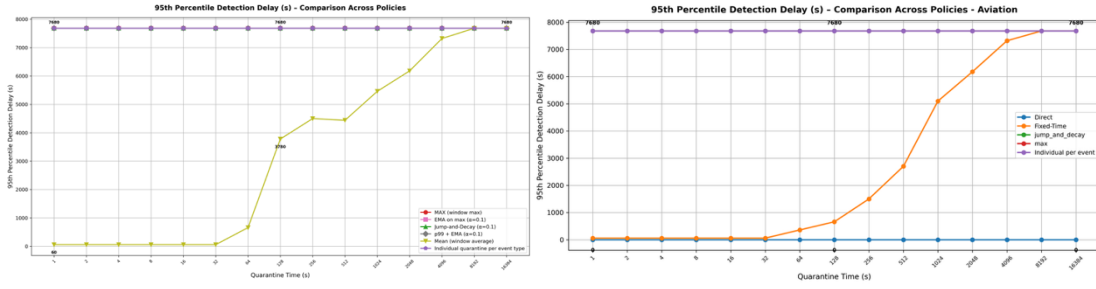


Figure 5.23: 95th-percentile detection delay for the medium complexity Aviation query. (a) All dynamic policies. (b) Key dynamic policies versus baseline policies.

The 95th-percentile detection delay for the Aviation query shows two distinct behavioral groups among the policies. The Direct policy maintains a zero delay across all quarantine times, since it performs no buffering. In contrast, almost all dynamic policies, exhibit a constant detection delay, approximately 7680 seconds. These policies always wait long enough to reorder events, even the most delayed, resulting a stable detection delay that does not depend on the quarantine window. The Mean (window average) policy begins with a low delay (around 60 seconds) for small quarantine windows but increases steadily as the window grows, eventually converging to 7680 seconds of detection delay. Among the baseline policies, Fixed-Time shows a sharp increase in detection delay once the quarantine window exceeds a small threshold. For small windows, it maintains low detection delay, but as the window grows, it eventually reaches the same 7680-second detection delay as the dynamic policies. This demonstrates that Fixed-Time is directly affected by the size of the quarantine window, since larger windows force events to remain buffered for longer.

5.3.1.5 Comparison with ideal results

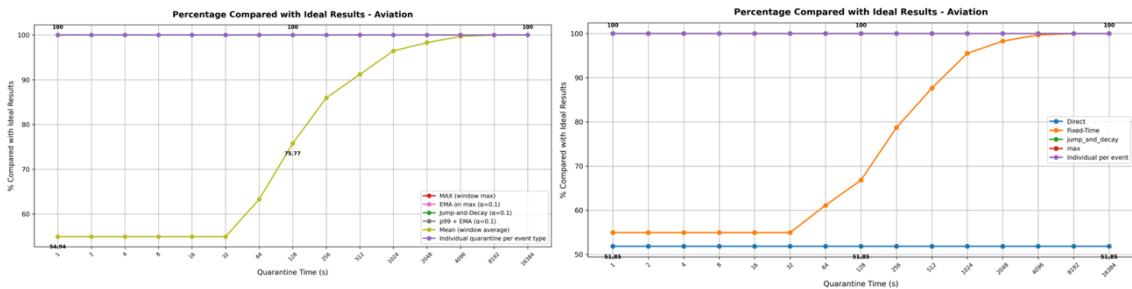


Figure 5.24: Percentage of results recovered compared with the ideal output for the medium complexity Aviation query. (a) All dynamic policies. (b) Key dynamic policies versus baseline policies.

The Percentage Compared with Ideal Results graphs for the Aviation dataset show that most dynamic policies achieve 100% correctness across all quarantine times. These policies successfully reorder all relevant events even with minimal buffering, demonstrating strong robustness to the dataset's disorder. Only the Mean (window average) policy performs significantly worse for small windows. It begins at approximately 54.94% correctness and increases gradually as the quarantine window grows. Only at large windows does it converge toward 100% correctness, indicating that the Mean policy requires more buffering to avoid dropping events. Among the baseline policies, Direct remains at ~50% correctness, since it performs no buffering and cannot recover out-of-order events. Fixed-Time starts near the same level but steadily improves as the quarantine window increases, eventually reaching 100% correctness for sufficiently large windows. Overall, the results show that dynamic policies provide ideal correctness across all windows, while Fixed-Time policy require larger windows to achieve that.

5.3.1.6 Maximum Events in Quarantine and Quarantine Size

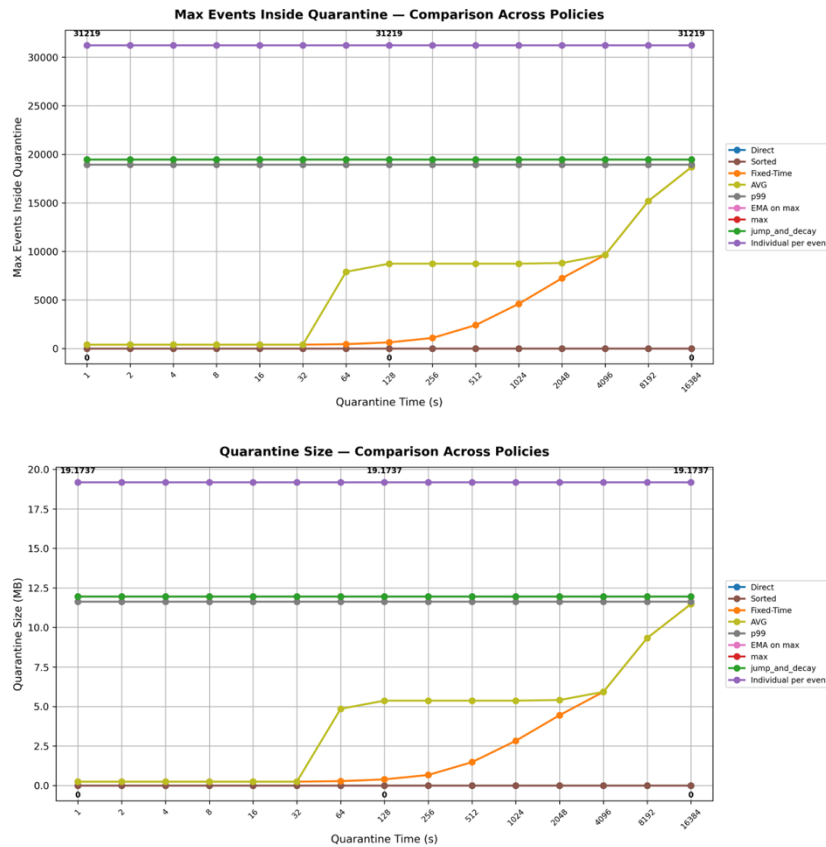


Figure 5.25: (a) Max events inside quarantine across all policies. (b) Quarantine size (MB) across all policies.

The Max Events Inside Quarantine and Quarantine Size results for the Aviation dataset reveal clear differences between policies. The Individual per event policy has the highest memory usage, maintaining a constant buffer of approximately 31.219 events (≈ 19.17 MB) regardless of the quarantine window, because it maintains a separate quarantine buffer for each event type. The Jump-and-Decay and the p99 policy also behave the same, each having 18.000–20.000 events in quarantine across all windows. The Average-Mean policy begins with very small buffers (similar to Direct) but grows steadily with the quarantine window and eventually reaches the ~ 18.000 events (≈ 12 MB) for large windows. In contrast, the Fixed-Time policy shows moderate quarantine sizes, starting near zero and rising to a few megabytes as the window expands, while Direct remains at zero for all quarantine times, since it performs no buffering.

5.3.2 Aviation Dataset: Simple Query Analysis

The simple query for the Aviation dataset detects pairs of events where a flight from *Alaska Airlines* is followed by a flight from *JetBlue Airways* within a 50-second window, based on their event timestamps. Like the simple query of the Fires dataset, this query does not use any partitioning. This means that the entire flight stream is evaluated sequentially as a single timeline. The query uses CONSUME BY NONE, which means that all events remain available for matching, allowing multiple flight pairs to be found.

```
SELECT * FROM S
WHERE (Alaska_Airlines_Inc as alaska; JetBlue_Airways as jetblue)
WITHIN 50 [Event_Timestamp]
CONSUME BY NONE
```

Query 5.5: Simple complexity Aviation query

The following figures present the performance metrics for this query under the selected quarantine policies. Each graph highlights a different aspect of the system's behavior, providing a clear view of how the policies perform when processing this simple aviation stream.

5.3.2.1 Dropped Events

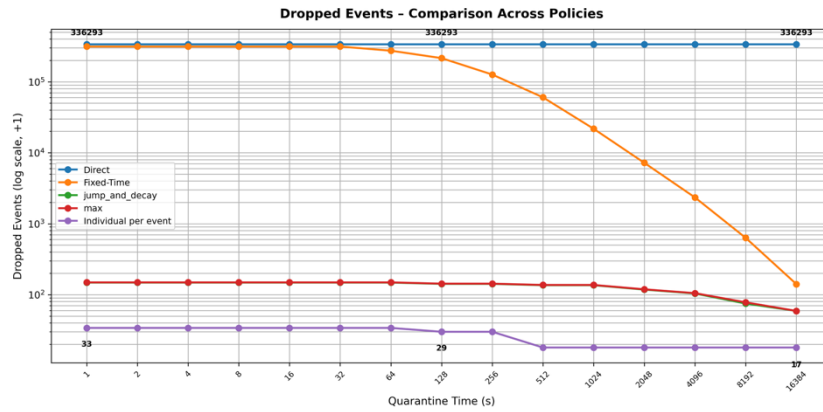


Figure 5.26: Dropped Events - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

The graph shows a performance gap between baseline and adaptive policies. The Direct policy drops a constant 336,293 events, while Fixed-Time performs equally poorly at small windows, requiring very large quarantine times to reduce its high data loss. Conversely, all dynamic mechanisms keep dropped events extremely low. The MAX and Jump-and-Decay policies overlap near the bottom, keeping data loss under 150 events. The Individual per Event policy achieves the best results, starting at just 33 drops at 1 second, decreasing to 29 drops at 128 seconds, and reaching a minimum of 17 drops. This demonstrates that per-carrier thresholds adapt perfectly to flight stream characteristics without relying on long, static delays.

5.3.2.2 Complex Events

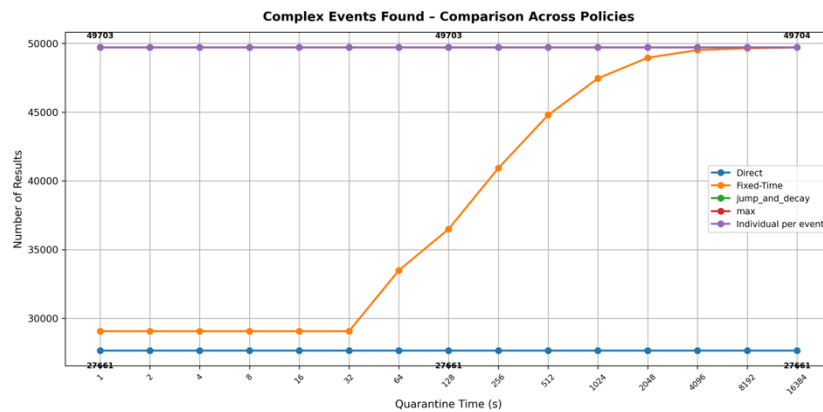


Figure 5.27: Complex Events - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

The graph reflects the direct impact of event drops on query accuracy. The Direct policy shows a flat and low number of complex events, capturing 27,661 results across all time

windows due to its high data loss. The Fixed-Time policy starts with a similarly low output of around 29,000 results for small quarantine windows, requiring significantly larger windows above 1024 seconds to finally approach peak accuracy. In contrast, all dynamic policies maximize accuracy immediately from the very first second. The MAX, Jump-and-Decay, and Individual per Event mechanisms overlap completely at the top of the graph, capturing 49,703 complex events from a 1-second quarantine window and reaching a maximum of 49,704 results.

5.3.2.3 Execution Time

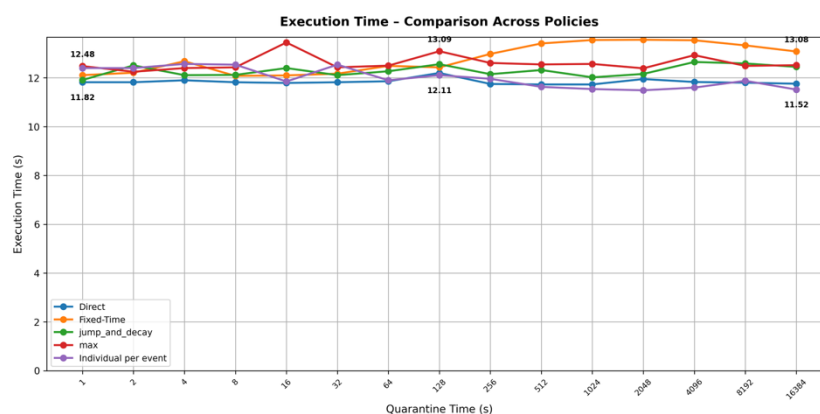


Figure 5.28: Execution Time - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

The graph shows that all quarantine mechanisms exhibit highly stable and uniform execution time for this query. Across the entire range of quarantine time windows, the total processing time for all five policies remains tightly clustered between 11.52 and 13.09 seconds. This performance confirms that the advanced tracking logic of the dynamic approaches including the Individual per Event policy does not introduce any meaningful computational or execution overhead compared to the simplest Direct baseline.

5.3.2.4 Detection Delay

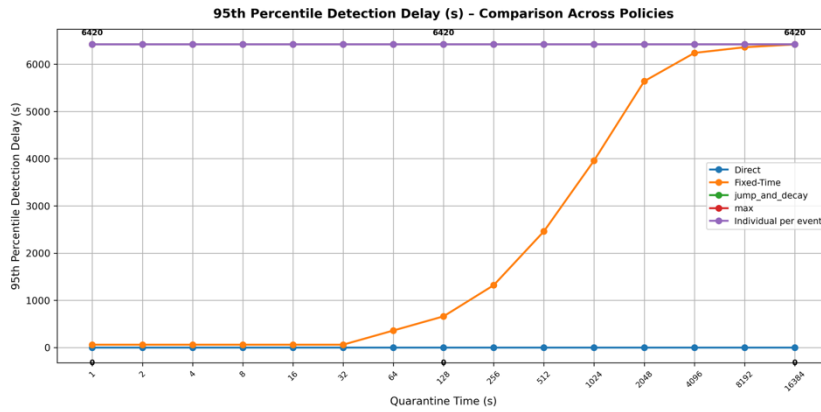


Figure 5.29: 95th Percentile Detection Delay - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

The graph illustrates a clear trade-off between processing delay and output quality. The Direct policy shows a 0-second delay because it processes events immediately, which results in severe data loss, while the Fixed-Time policy scales its latency linearly up to 6,420 seconds as the quarantine window expands. Conversely, all dynamic approaches immediately stabilize at a constant 6,420-second delay across all points. Although this latency is higher than Fixed-Time at smaller windows, it represents the exact out-of-order interval required by the system to prevent data loss and ensure maximum query accuracy from the very beginning.

5.3.2.5 Comparison with ideal results

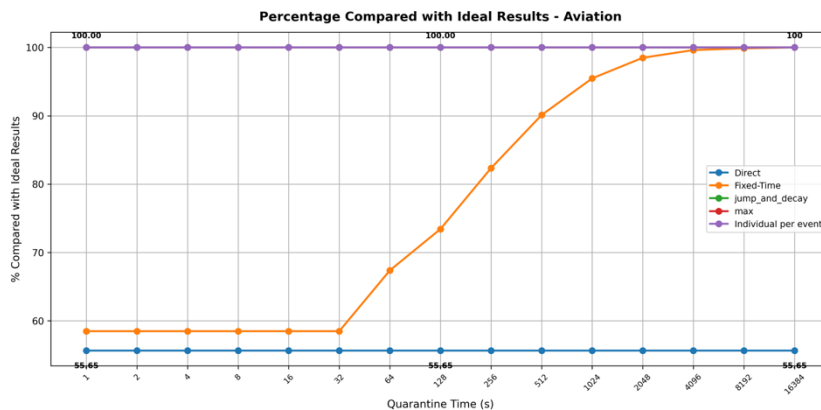


Figure 5.30: Percentage Compared with Ideal Results - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

The graph demonstrates that the Direct policy remains stuck at a low 55.65% accuracy across all windows due to its severe data loss. The Fixed-Time policy starts at a low 58.50% for small windows up to 32 seconds and requires large quarantine times to gradually approach peak accuracy. In contrast, dynamic approaches delivering 100.00% accuracy from the very first second. This proves that dynamic thresholds successfully eliminate the need for manual window configuration while consistently ensuring flawless event sequence matching.

5.3.3 Aviation Dataset: Complex Query Analysis

The complex query for the Aviation dataset correlates filtered events from Alaska Airlines and JetBlue Airways under a 50-second temporal constraint. By applying filters (`AIR_TIME > 120` for Alaska and `DISTANCE > 1000` for JetBlue), the query increases selectivity and focuses on longer flights and long-distance segments, thereby increasing the overall complexity of the query. Partitioning by `ORIGIN` isolates the evaluation per airport, ensuring that disorder is handled independently across locations. The `CONSUME BY NONE` allows events to participate in multiple matches, enabling the detection of overlapping correlations. This structure makes the query representative of realistic event filtering and correlation workloads, providing a strong basis for evaluating correctness, detection delay, and memory behavior across quarantine policies.

```
SELECT * FROM S
WHERE (Alaska_Airlines_Inc as alaska; JetBlue_Airways as jetblue)
FILTER alaska [AIR_TIME>120.0] AND jetblue [DISTANCE>1000]
PARTITION BY [ORIGIN]
WITHIN 50 [Event_Timestamp]
CONSUME BY NONE
```

Query 5.6: Complex Aviation Query

The following figures present the full set of performance metrics for this complex query. Each graph highlights a different aspect of system behavior under the selected quarantine policies. These visualizations provide a complete view of how the policies perform when processing this complex query on the Aviation dataset.

5.3.3.1 Dropped Events

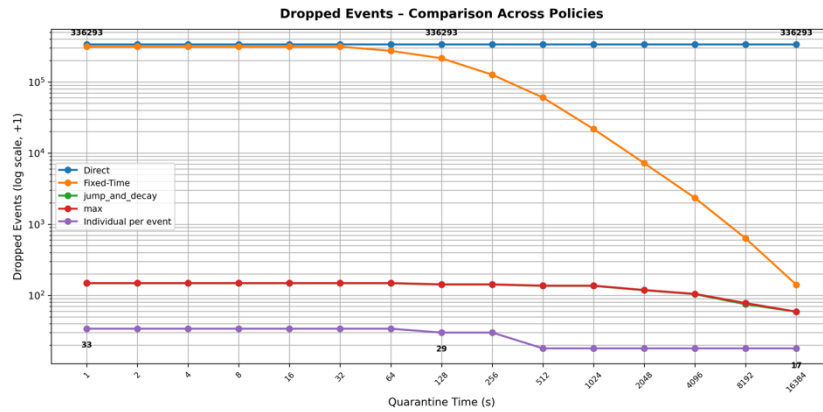


Figure 5.31: Dropped Events - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.31 presents the number of dropped events across all quarantine policies for the complex Aviation query. The Direct policy consistently drops the largest number of events over 330.000 regardless of the quarantine window, reflecting its inability to buffer and reorder the stream. The Fixed-Time policy starts at the same high drop count but steadily reduces dropped events as the quarantine window increases, demonstrating the benefit of additional buffering capacity. The dynamic policies Max and Jump-and-Decay maintain lower drop counts, with Max stabilizing around a few hundred events and Jump-and-Decay decreasing slightly as the window grows. The Individual Per Event policy achieves the lowest number of dropped events overall, remaining near 30 drops and improving further at larger windows. Overall, the figure shows that the number of dropped events depends on the buffering strategy, with adaptive policies dropping far fewer events than the baselines.

5.3.3.2 Complex Events

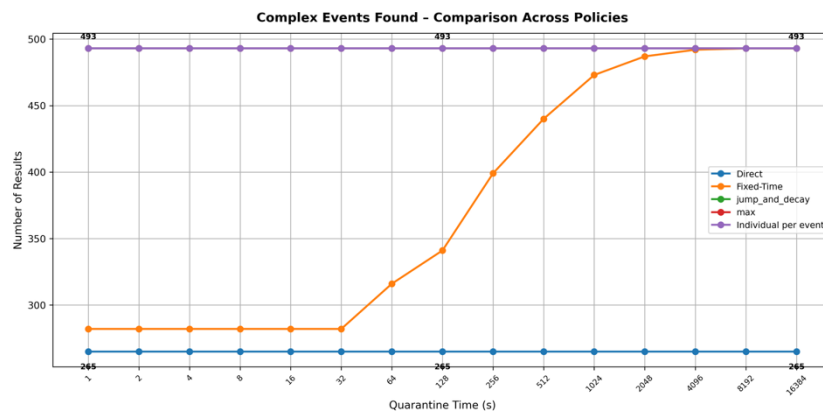
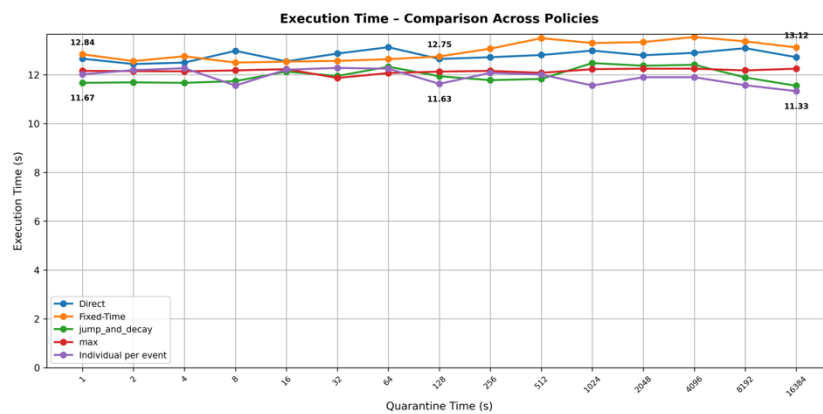


Figure 5.32: Complex Events - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.32 presents the number of complex events detected across all quarantine policies for the complex Aviation query. The Direct policy remains constant at 265 detected events, reflecting its inability to recover results lost due to disorder. The Fixed-Time policy starts slightly higher, at 285 results, and steadily improves as the quarantine window increases, eventually reaching the upper bound of 493 results. The Individual Per Event policy consistently achieves this upper bound across all window sizes, demonstrating its robustness and ability to fully reconstruct the correct set of complex events. The dynamic policies Max and Jump-and-Decay match this ideal performance as well, aligning with the upper bound due to their similar buffering behavior.

5.3.3.3 Execution Time



5.3.3.4 Detection Delay

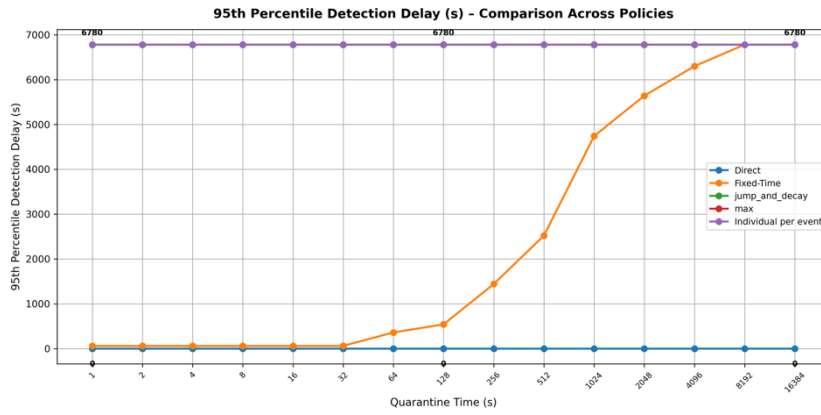


Figure 5.34: 95th Percentile Detection Delay - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.34 presents the 95th percentile detection delay across all quarantine policies for the complex Aviation query. The Direct policy has zero detection delay across all window sizes, as it performs no buffering and reordering. The Fixed-Time policy shows an increase in detection delay as the quarantine window grows, reflecting the cost of holding events for longer periods before evaluation. The Individual per Event policy remains at the upper bound 6780 seconds across all window sizes, as it fully reorders the stream before producing results. The dynamic policies align with this upper bound as well due to their similar buffering behavior. Overall, the figure shows that detection delay is affected by how each policy reorders the stream, with more extensive buffering leading to significantly higher detection delays, while at the same time allowing the system to detect more complex events and improve its overall accuracy.

5.3.3.5 Comparison with ideal results

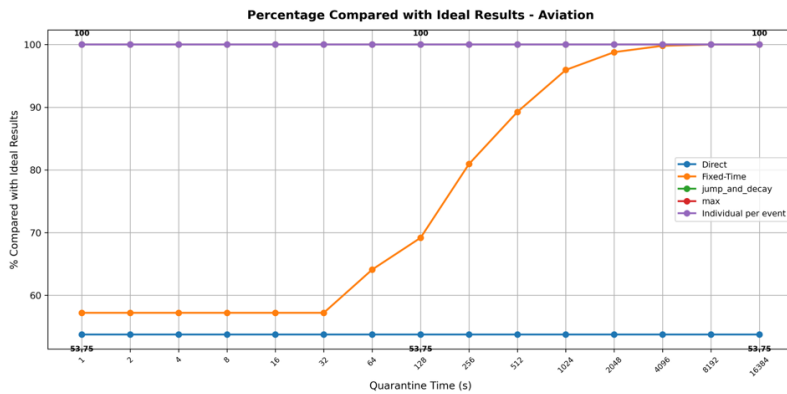


Figure 5.35: Percentage Compared with Ideal Results - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.35 presents the percentage of detected complex events compared with the ideal results across all quarantine policies for the complex Aviation query. The Direct policy remains stable at approximately 54%, reflecting its inability to recover results lost due to disorder. The Fixed-Time policy starts slightly higher and gradually improves as the quarantine window increases, eventually approaching 100% once sufficient buffering is available. The Individual Per Event policy consistently achieves 100% across all window sizes, demonstrating its robustness and ability to fully construct the correct set of complex events. The other dynamic policies match this ideal performance as well.

5.4 Detailed Analysis of the Crypto Dataset

5.4.1 Crypto Dataset: Medium-Complexity Query Analysis

The medium complexity query for the Crypto dataset detects triples of events where an event for BNB is followed by an event for DOGE, and then by a NEAR event, all occurring within a 10-second window based on their event timestamps. The evaluation is partitioned by Exchange. The query uses CONSUME BY ANY, meaning that once an event participates in a valid BNB→DOGE→NEAR sequence, it is consumed and cannot be reused for additional matches.

```
SELECT * FROM S
WHERE (BNB as bnb; DOGE as doge; NEAR as near)
PARTITION BY [Exchange]
WITHIN 10 [Event_Timestamp]
CONSUME BY ANY
```

Query 5.7: Medium complexity Crypto query

The following figures present the full set of performance metrics for this query. Each graph highlights a different aspect of system behavior under the selected quarantine policies. These graphs provide a complete view of how the policies perform when processing this medium complexity query on the Crypto dataset.

5.4.1.1 Dropped Events

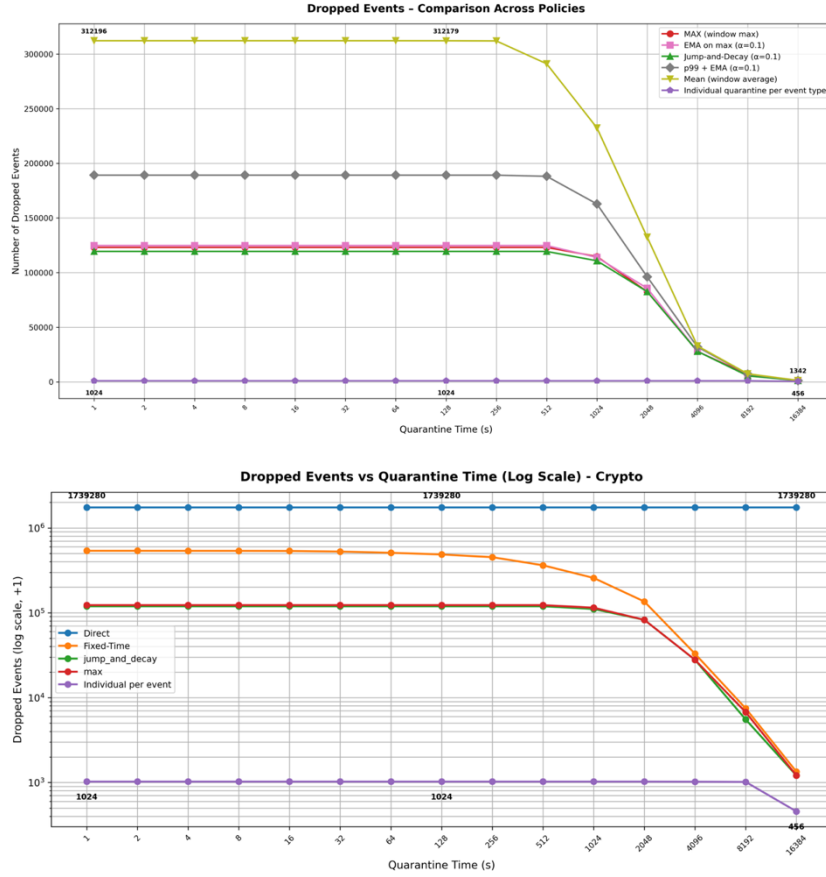


Figure 5.36: Dropped events for the medium complexity Crypto query. (a) All dynamic policies. (b) Comparison between the top dynamic policies and the existing baseline policies.

The Dropped Events for the Crypto dataset show substantial variation across policies as the quarantine window increases. The Direct policy performs the worst, consistently dropping approximately 1.74 million events regardless of the quarantine time, since it performs no buffering and therefore cannot tolerate any disorder. The Fixed-Time, max and Jump-and-Decay policies both start with similarly high drops at small quarantine times but gradually reduce their losses as the quarantine window grows, reflecting their ability to absorb more out-of-order events. The Individual per event policy achieves the best performance, maintaining a consistently low number of dropped events, around 1.024 for most window sizes and decreasing even more at the largest windows.

5.4.1.2 Complex Events

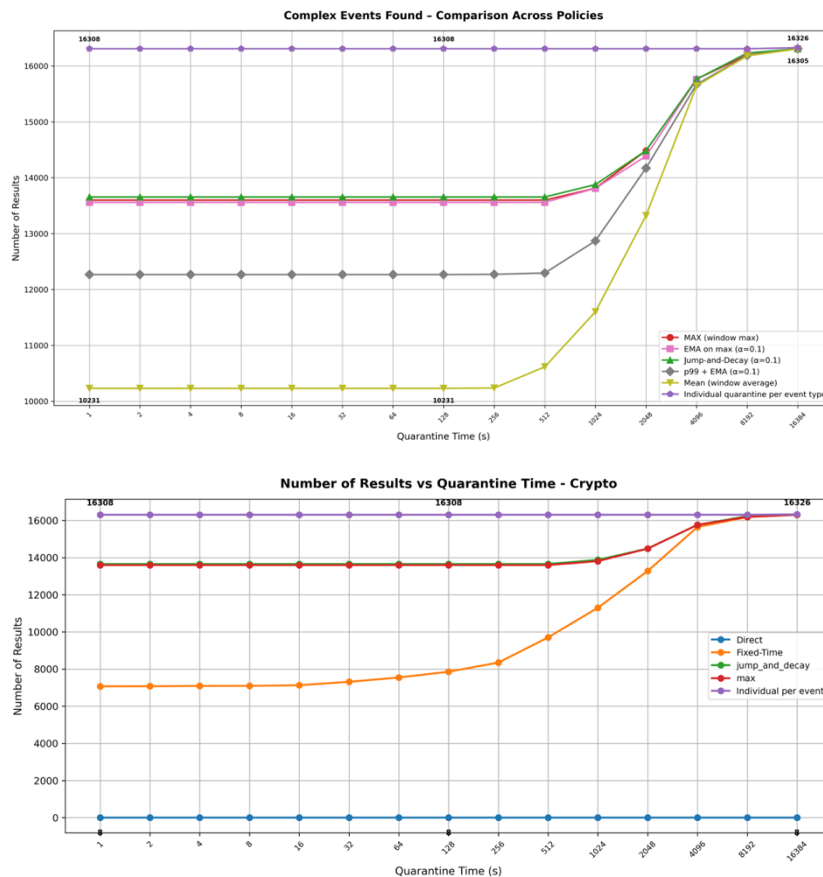


Figure 5.37: Complex events found for the medium complexity Crypto query. (a) All dynamic policies. (b) Comparison between the top dynamic policies and the existing baseline policies.

The Complex Events for the Crypto dataset show how each buffering policy approaches the ideal number of detected events as the quarantine window increases. The Direct policy finds zero results across all window sizes, since it performs no buffering and therefore misses all valid ordered triples. The Fixed-Time policy improves steadily with larger windows, starting at 7.000 results and eventually reaching the ideal number of results near 16.000. The Jump-and-Decay and Max policies begin near 13,000 complex events and steadily approach the ideal number, as the quarantine time increases. The Individual per event policy consistently achieves the best performance, maintaining 16.308 results across nearly all windows and slightly increasing to 16.326 at the largest window, effectively matching the ideal output.

5.4.1.3 Execution Time

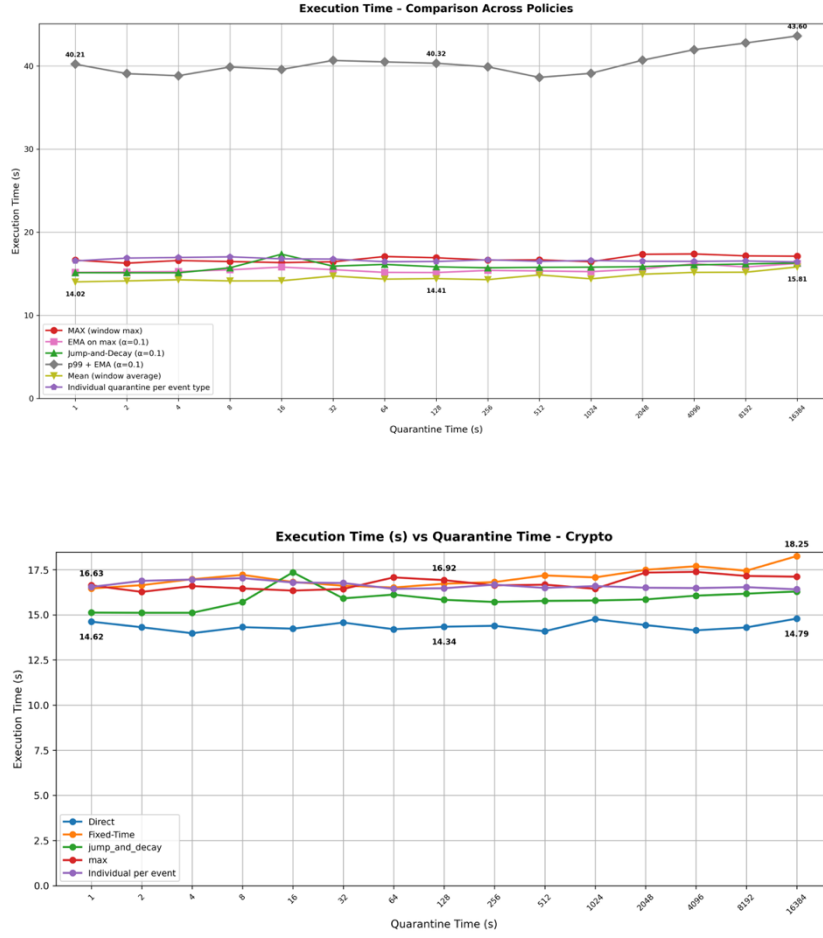


Figure 5.38: Execution time for the medium complexity Crypto query. (a) All dynamic policies. (b) Top dynamic policies versus existing baseline policies.

The Execution Time results for the Crypto dataset show that all buffering policies introduce moderate overhead compared to the Direct policy but remain within a narrow performance range. The Direct policy is consistently the fastest, with execution times between 14.3 s and 14.8 s, since it performs no buffering or disorder handling. The Fixed-Time policy, Jump-and-Decay, and Individual per event all exhibit slightly higher execution times, ranging from 16 s to 17 s. The p99 EMA policy incurs the highest overhead, reaching up to 43 s at the largest quarantine windows because it continuously computes the 99th percentile, which adds additional computational overhead. Despite these differences, all dynamic policies remain within a range of 14–18 s, indicating that even the more complex buffering strategies add only modest computational cost relative to the baseline.

5.4.1.4 Detection Delay

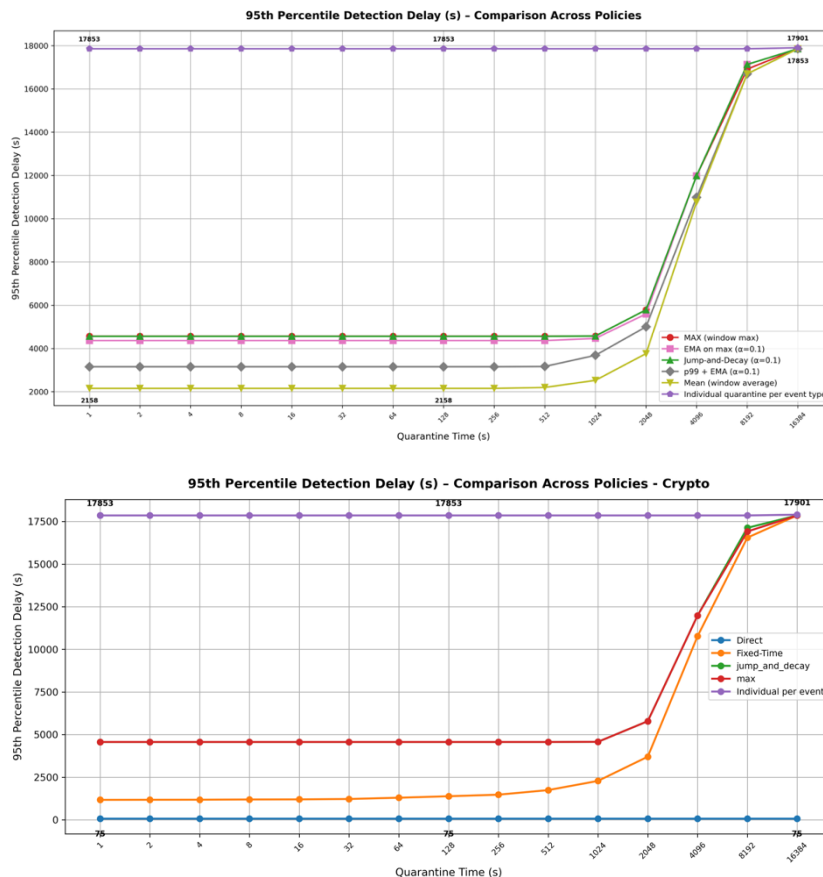


Figure 5.39: 95th-percentile detection delay for the medium complexity Aviation query. (a) All dynamic policies. (b) Key dynamic policies versus baseline policies.

The 95th Percentile Detection Delay for the Crypto dataset reveals two distinct groups of policies. The Fixed-Time, max, and jump-and-decay policies all show increasing detection delay as the quarantine window grows, since their overall time is directly affected by the size of the quarantine. As a result, these policies exhibit higher detection delay at larger quarantine windows. In contrast, the Individual per event maintain a constant detection delay of approximately 18.252 s across all window sizes. Because larger quarantine windows allow these policies to detect more patterns with minimal drops, the number of detected complex events increases, and consequently their detection delay also increases. Finally, the Direct policy consistently achieves the lowest detection delay, since it processes events immediately without buffering or disorder handling.

5.4.1.5 Comparison with ideal results

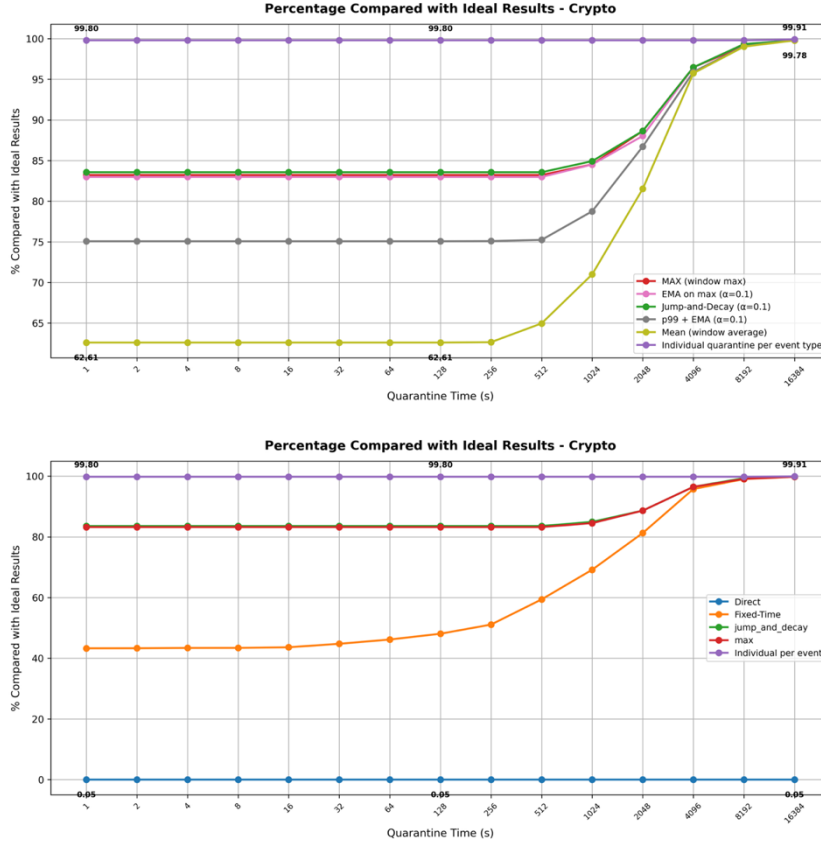


Figure 5.40: Percentage of results recovered compared with the ideal output for the medium complexity Crypto query. (a) All dynamic policies. (b) Key dynamic policies versus baseline policies.

The Percentage Compared with Ideal Results for the Crypto dataset highlights clear differences in how quickly each policy approaches the ideal output as the quarantine window increases. The Direct policy remains near 0% (0.05) across all window sizes, since it performs no buffering and therefore misses almost all valid BNB→DOGE→NEAR sequences. The Fixed-Time policy starts at 45% and increases steadily with larger windows, eventually converging toward the ideal output results at high quarantine times. The Max and Jump-and-Decay policies start at 84% but, as the quarantine window increases, they progressively approach the ideal result. In contrast, the Individual per event policy consistently has the best performance, remaining around 99.8–99.9% for all windows. Finally, p99 EMA and Mean policies begin at lower percentages (approximately 62% and 75%, respectively) but rise sharply once the quarantine window exceeds 1024 s, converging with the top-performing policies. Overall, the Crypto dataset demonstrates that the Individual Per Event Type policy is the only one that consistently reaches ideal accuracy at all window sizes, whereas all other policies need substantially larger windows to converge toward full result recovery.

5.4.1.6 Maximum Events in Quarantine and Quarantine Size

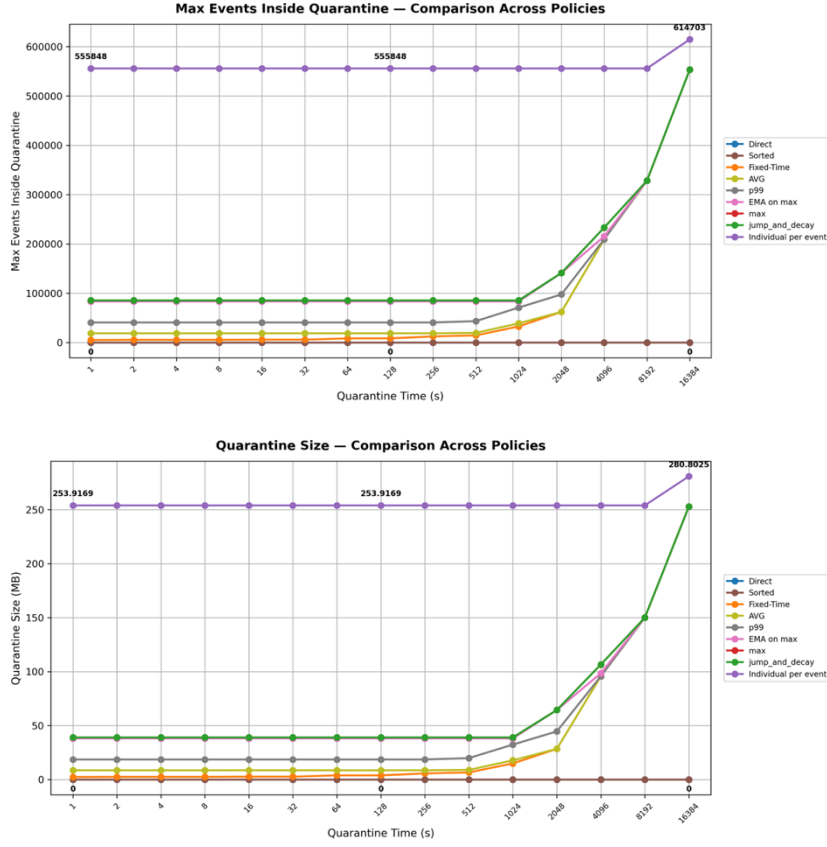


Figure 5.41: (a) Max events inside quarantine across all policies. (b) Quarantine size (MB) across all policies.

The Quarantine Size and Max Events Inside Quarantine results for the Crypto dataset show substantial differences in memory requirements across policies as the quarantine window increases. The Direct policy maintains zero quarantine size, since it performs no buffering. The Fixed-Time policy and all the dynamic policies, except the Individual Per Event policy, follow the same general pattern: they maintain a low and stable quarantine size at small window durations and only begin to increase once the quarantine window becomes sufficiently large. Finally, the Individual per event policy consistently maintains the highest quarantine size, remaining near 254 MB across most windows and increasing to 280 MB at the largest window. This behavior reflects its design: maintaining separate per event type buffers leads to high memory consumption but achieves the best accuracy. Overall, the Crypto dataset demonstrates that adaptive and per event type buffering policies achieve higher correctness at the cost of increased memory usage, whereas simpler policies remain lightweight but achieve lower accuracy.

5.4.2 Crypto Dataset: Simple Query Analysis

The low-complexity query for the Crypto dataset detects sequences of events where a transaction update in BNB is followed sequentially by updates in DOGE and NEAR within a tight 10-second window, based on their event timestamps. Unlike the previous datasets, this query uses CONSUME BY ANY, meaning that an event is randomly consumed only upon successfully participating in the detection of a complex event, after which it cannot be reused. The execution processes the entire event stream sequentially without applying any temporal partitioning.

```
SELECT * FROM S
WHERE (BNB as bnb; DOGE as doge; NEAR as near)
WITHIN 10 [Event_Timestamp]
CONSUME BY ANY
```

Query 5.8: Simple complexity Crypto query

The following figures present the performance metrics for this query for the top three quarantine policies compared with the existing quarantine policies. Each graph highlights a different aspect of system behavior under the selected quarantine policies. These graphs provide a complete view of how the policies perform when processing this simple-complexity query on the Crypto dataset.

5.4.2.1 Dropped Events

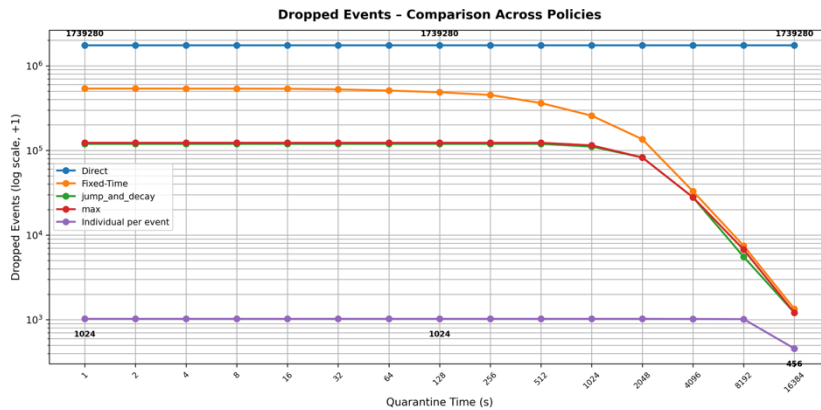


Figure 5.42: Dropped Events - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

As demonstrated in the graph, there is a difference between the baselines policies and the dynamic approaches. The Direct policy suffers from maximum data loss, discarding a constant 1,739,280 events regardless of the configuration, whereas the Fixed-Time policy maintains highly elevated drop rates for small windows before scaling down at larger intervals. The adaptive MAX and Jump-and-Decay mechanisms show a strong improvement, keeping dropped events stable at around 120,000 for the majority of quarantine times before reaching close to 1,000 at the largest quarantine window. The Individual per Event policy secures the top performance, maintaining an impressive low of just 1,024 drops across almost all quarantine times and hitting an absolute minimum of 456 drops at 16,384 seconds.

5.4.2.2 Complex Events

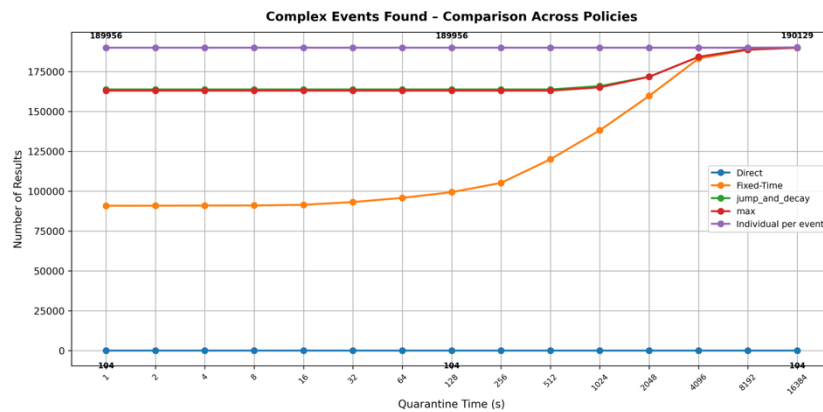


Figure 5.43: Complex Events - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

The graph shows how event drops directly impact query accuracy. The Direct policy produces a flat and low output of complex events due to its data loss. The Fixed-Time policy performs slightly better at small windows but remains low until the quarantine time increases significantly to capture the missed sequences. All adaptive policies maximize accuracy from the very first second. The MAX, Jump-and-Decay, and Individual per Event mechanisms overlap completely at the top of the graph, capturing the maximum number of complex events instantly.

5.4.2.3 Execution Time



Figure 5.44: Execution Time – Comparison between the Top Dynamic Policies and the Existing Baseline Policies

The graph demonstrates that execution time remain highly uniform and stable across all evaluated configurations. The Direct baseline maintains the lowest and most stable execution time, averaging around 14.50 seconds since it completely avoids holding events. The dynamic policies introduce a manageable overhead due to their state-tracking mechanisms, with their processing times hovering steadily between 19.50 and 22.50 seconds across most windows. Notably, the Fixed-Time policy matches the performance of the dynamic quarantines at smaller intervals but climbs significantly at larger windows, peaking at 24.84 seconds.

5.4.2.4 Detection Delay

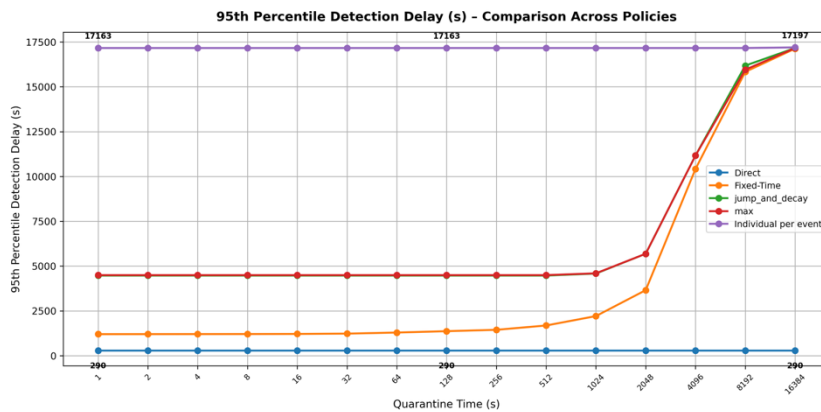


Figure 5.45: 95th Percentile Detection Delay – Comparison between the Top Dynamic Policies and the Existing Baseline Policies

The graph shows a clear separation in latency metrics among the tested quarantine configurations, following a very similar trade-off pattern to the one observed in both the

Fires and Aviation datasets. The Direct baseline maintains a 290-second detection delay because it processes incoming data immediately, sacrificing accuracy for speed. The Fixed-Time policy scales up gradually before spiking at higher windows, while the dynamic MAX and Jump-and-Decay policies track closely together, holding a stable delay of 4,490 seconds across smaller windows before clamps upward. Crucially, much like its behavior in the previous datasets, the Individual per Event policy exhibits a flat 17,163 second delay across nearly the entire range, peaking at 17,197 seconds. Although this indicates higher latency at smaller windows, this sustained buffer is precisely what enables the policy to achieve near-zero event drops from the very start.

5.4.2.5 Comparison with ideal results

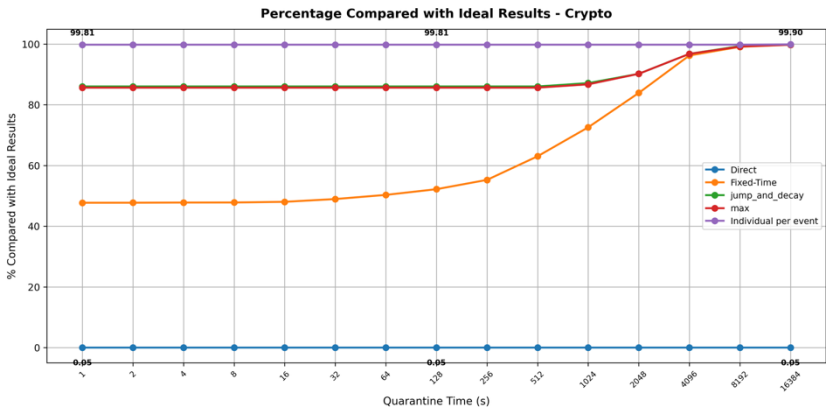


Figure 5.46: Percentage Compared with Ideal Results - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

The graph shows how closely each policy matches the ideal results, mirroring the trends seen in the Fires and Aviation datasets. The Direct baseline stays flat at a near-zero 0.05% due to severe data loss, while Fixed-Time starts at a low 47.80% and only climbs as the window expands. The dynamic MAX and Jump-and-Decay policies offer a strong improvement at 85.90% before peaking at 99.90%. The Individual per Event policy achieves great accuracy instantly, maintaining a flat 99.81% across smaller windows and hitting 99.90% at the final point. This proves that dynamic, per-event quarantine consistently delivers optimal results across all datasets without requiring manual window configuration.

5.4.3 Crypto Dataset: Complex Query Analysis

The complex query for the Crypto dataset detects ordered sequences of three cryptocurrency events, BNB followed by DOGE followed by NEAR, within a 10-second window, after applying some filters that increase the selectivity and complexity of the pattern. Specifically, the query requires BNB events with TxFee < 10.000, DOGE events with TxSize < 250, and NEAR events with TxFee > 500. The evaluation is partitioned by Exchange. Because the query uses CONSUME BY ANY, each event can participate in at most one match, meaning that once an event is used in a BNB→DOGE→NEAR sequence, it is consumed and cannot be reused for overlapping sequences. The combination of temporal ordering, per exchange partitioning, and selective attribute filters makes this a representative complex workload for evaluating correctness, buffering behavior, and detection delay across quarantine policies.

```
SELECT * FROM S
WHERE (BNB as bnb; DOGE as doge; NEAR as near)
FILTER bnb [TxFee<10000] AND doge [TxSize<250] AND near [TxFee>500]
PARTITION BY [Exchange]
WITHIN 10 [Event_Timestamp]
CONSUME BY ANY
```

Query 5.9: Complex Crypto Query

The following figures present the full set of performance metrics for this complex query. Each graph highlights a different aspect of system behavior under the selected quarantine policies. These visualizations provide a complete view of how the policies perform when processing this complex query on the Crypto dataset.

5.4.3.1 Dropped Events

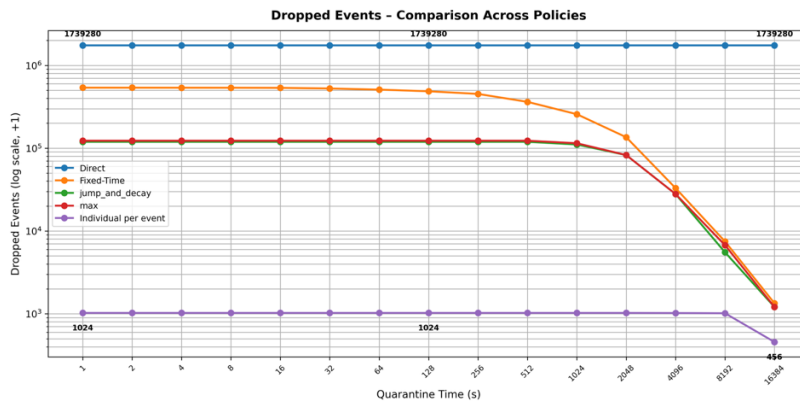


Figure 5.47: Dropped Events - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.47 presents the number of dropped events across all quarantine policies for the complex Crypto query. The Direct policy consistently produces the highest number of dropped events around 1.7 million because it performs no buffering or reordering. The Fixed-Time policy starts at similarly high levels but gradually reduces dropped events as the quarantine window increases, reflecting the benefit of additional buffering capacity. The dynamic policies Max and Jump-and-Decay maintain substantially lower drop counts, decreasing further as the window grows. The Individual Per Event policy achieves the lowest number of dropped events overall, remaining close to a few hundred and improving steadily with larger windows.

5.4.3.2 Complex Events

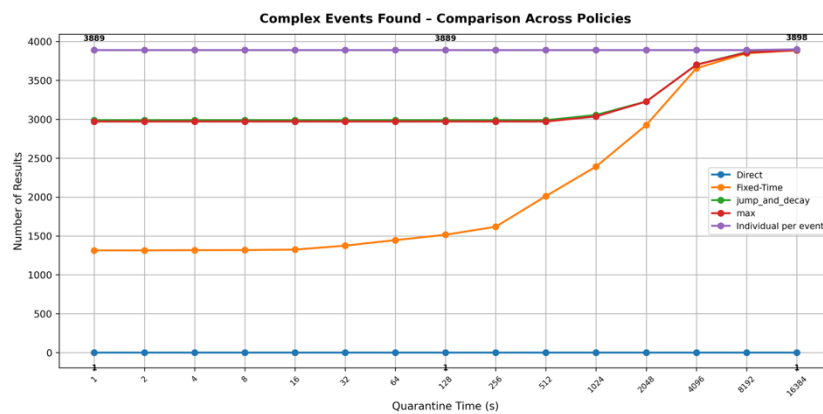


Figure 5.48: Complex Events - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.48 presents the number of complex events detected across all quarantine policies for the complex Crypto query. The Direct policy remains near zero across all window sizes, as it cannot recover events lost due to disorder. The Fixed-Time policy starts at around 1300 detected events and steadily improves as the quarantine window increases, eventually approaching the upper bound of approximately 3900 results. The dynamic policies Max and Jump-and-Decay maintain intermediate performance for smaller windows, detecting around 3000 events, and then increase at larger windows to match the ideal upper bound. The Individual Per Event policy consistently achieves this upper bound across all window sizes, demonstrating its robustness and ability to fully construct the correct set of complex events.

5.4.3.3 Execution Time

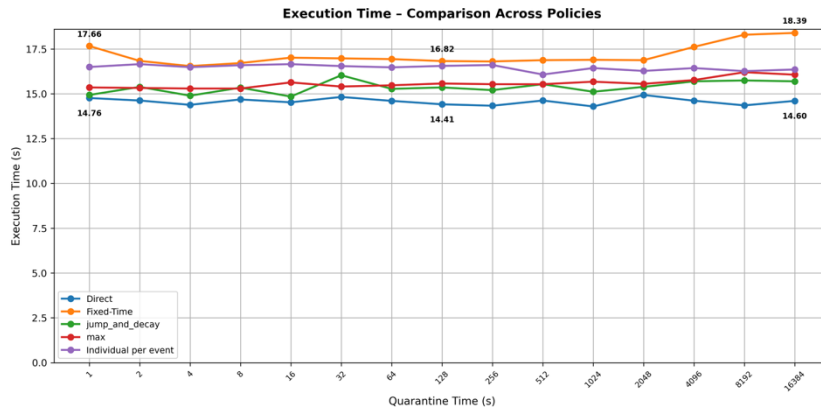


Figure 5.49: Execution Time - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.49 presents the execution time of the complex Crypto query across all quarantine policies. The Fixed-Time policy consistently exhibits the highest execution times, ranging from approximately 17.7 to 18.4 seconds, due to the overhead of maintaining a large, fixed buffer. The Direct policy achieves the lowest execution times around 14.6 to 14.8 seconds since it performs no buffering or reordering. The dynamic policies fall between these two extremes, maintaining stable execution times between 15 and 17 seconds across all window sizes. Overall, the figure shows that execution time remains stable as the quarantine window increases, with adaptive policies adding moderate overhead compared with Direct and Fixed-Time.

5.4.3.4 Detection Delay

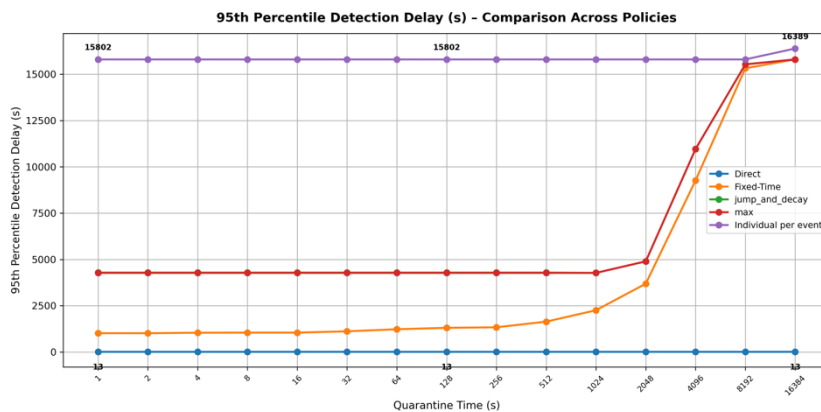


Figure 5.50: 95th Percentile Detection Delay - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.50 presents the 95th percentile detection delay across all quarantine policies for the complex Crypto query. The Direct policy maintains near-zero detection delay across all window sizes, as it performs no buffering or reordering. The Fixed-Time and Max policies show an increase in detection delay once the quarantine window exceeds 2048 seconds, reflecting the cost of holding events for extended periods before evaluation. The Individual Per Event remain at the upper bound across all window sizes, as it fully reorders the stream before producing results. The Jump-and-Decay policy aligns with this upper bound as well, matching the behavior of the more aggressive buffering strategies. Overall, the figure shows that detection delay is strongly affected by how each policy reorders the stream, with more aggressive buffering leading to significantly higher detection delays but also enabling more complete and accurate detection of complex events.

5.4.3.5 Comparison with ideal results

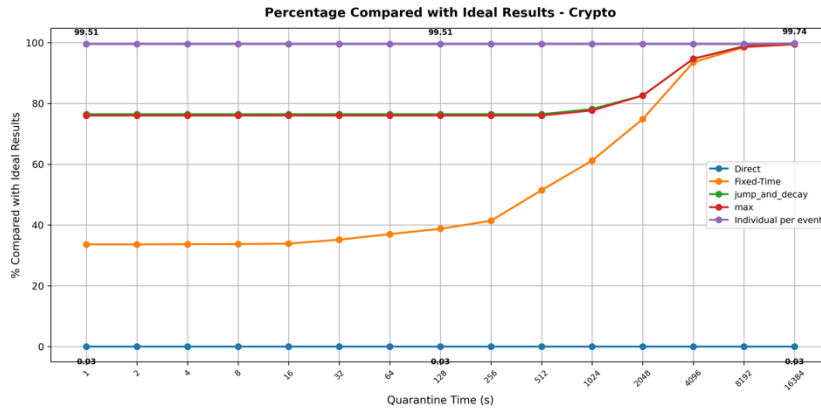


Figure 5.51: Percentage Compared with Ideal Results - Comparison between the Top Dynamic Policies and the Existing Baseline Policies

Figure 5.51 presents the percentage of detected complex events compared with the ideal results across all quarantine policies for the complex Crypto query. The Direct policy remains near zero across all window sizes, as it performs no buffering and therefore fails to recover almost all valid sequences. The Fixed-Time policy starts at one-third of the ideal results and steadily improves as the quarantine window increases, eventually approaching 100% once sufficient buffering is available. The dynamic policies Max and Jump-and-Decay maintain intermediate accuracy for smaller windows, around 75%, and then increase at larger windows to match the ideal upper bound. The Individual Per Event policy consistently achieves perfect accuracy across all window sizes, remaining close to 100% throughout.

CHAPTER 6

CONCLUSION

This chapter summarizes the main contributions of the thesis, presents the overall conclusions derived from the experimental evaluation, and outlines potential directions for future work.

6.1 Summary of the Project

The objective of this thesis was to enhance CORE, a Complex Event Recognition system designed to process high-volume streams containing late and out-of-order events. To handle the disorder of the stream, the system was extended with additional temporal attributes (Event Time, Arrival Time, System Arrival Delay, Detection Delay), allowing a clearer distinction between when an event occurred, when it was observed, and how delayed it was.

Based on these concepts, we redesigned the Fixed-Time policy and introduced six dynamic quarantine mechanisms AVG, p99-EMA, MAX-EMA, MAX, Jump-and-Decay, and Per-Event each adapting its buffering behavior to the observed lateness patterns of the stream. To evaluate these policies under diverse conditions, we conducted experiments on three heterogeneous datasets (fires, aviation, cryptocurrency), allowing a comprehensive evaluation of correctness, memory behavior, and runtime performance across datasets with different distributions and disorder characteristics.

6.2 Overall Conclusion and Policy Characteristics

Across all datasets and query complexities, the results consistently show that dynamic quarantine policies outperform static approaches, with the Per-Event policy achieving the most balanced and robust performance.

The DIRECT policy offers minimal correctness because it performs no buffering, whereas the Fixed-Time policy improves accuracy but remains unable to adapt to fluctuations in stream disorder.

Among dynamic strategies, MAX and Jump-and-Decay recover many late events but suffer from large buffer sizes under extreme disorder. AVG maintains small buffers but fails to capture large delays, reducing correctness. The p99-EMA policy offers stronger statistical adaptation but incurs the highest execution time due to percentile estimation overhead.

The Per-Event policy overcomes these limitations by handling lateness independently for each event type. It consistently minimizes dropped events, achieves near-ideal correctness even with small quarantine windows, and maintains competitive execution time. Although it requires more memory due to per-type buffering, its stability, adaptability, and correctness make it the most effective and predictable policy for real-time streaming environments.

6.3 Future Work & Extensions

Several directions can further improve the system. First, the memory usage of the Per-Event policy could be reduced through compression or more compact per-type buffering structures, enabling high correctness under stricter resource constraints.

The quarantine layer could be enhanced with predictive capabilities, using machine-learning models to forecast disorder spikes and proactively adjust buffering windows.

Beyond quarantine management, optimizing the internal mechanics of the Complex Event Automaton, such as refining the compact graph update logic or improving indexing structures, could yield additional performance gains under high event throughput.

Finally, exploring distributed and parallel execution models would allow both the quarantine layer and the CEA engine to scale across multi-core or multi-node environments, addressing challenges such as synchronization and partitioning while significantly increasing throughput.

REFERENCES

- [1] Bossonney, K., Buzeta, N., Calisto, V., López, J.-E., Riveros, C., & Vansummeren, S. (2025). *CORE+: A Complex Event Recognition Engine in C++*. (SIGMOD/PODS '25)
- [2] Wu, E., Diao, Y., & Rizvi, S. (2006). *High-Performance Complex Event Processing over Streams*. ACM SIGMOD.
- [3] Agrawal, J., Diao, Y., Gyllstrom, D., & Immerman, N. (2008). *Efficient Pattern Matching over Event Streams*. ACM SIGMOD.
- [4] Cugola, G., & Margara, A. (2012). *Processing Flows of Information: From Data Stream to Complex Event Processing*. ACM Computing Surveys.
- [5] Apache Flink Documentation – Complex Event Processing (CEP). Apache Software Foundation. Available at: <https://nightlies.apache.org/flink/flink-docs-stable/docs/libs/cep/>
- [6] Katsifodimos, A., & Markl, V. (2018). *Distributed Event-Time Stream Processing*. IEEE Data Engineering Bulletin.
- [7] Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., & Tzoumas, K. (2015). *Apache Flink™: Stream and Batch Processing in a Single Engine*. IEEE Data Engineering Bulletin.
- [8] EsperTech Inc. (2023). *Esper Reference Documentation*.
- [9] Eckert, M., Bry, F., & Brodt, S. (2010). *Incremental Evaluation of Event Patterns over Event Streams*.
- [10] Cugola, G., & Margara, A. (2012). *Processing Flows of Information: From Data Stream to Complex Event Processing*. ACM Computing Surveys.
- [11] OpenCEP Project. (2023). *Architecture of Logic-Based Event Processing Engines*.
- [12] https://data.ct.gov/Public-Safety/Connecticut-Fire-Department-Incidents-FiresOnly/b6pg-k63s/about_data
- [13] https://www.transtats.bts.gov/Fields.asp?gnoyr_VQ=FGK
- [14] <https://www.kaggle.com/datasets/itshasebsaeed/bitcoin-mempool-state-data>
- [15] Margara, A., Cugola, G., & Tamburrelli, G. (2014). *OpenCEP: An Open-Source Complex Event Processing Engine*. *Proceedings of the 8th ACM International Conference on Distributed Event-Based Systems (DEBS)*, 318–321. <https://doi.org/10.1145/2611286.2611313>
- [16] Demers, A., Gehrke, J., Panda, B., Riedewald, M., Sharma, V., & White, W. (2007). *Cayuga: A SQL-like language for complex event processing over data streams*. *Proceedings of the 3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 156-172.

- [17] Mei, Y., & Madden, S. (2009). ZStream: a cost-based query processor for adaptively detecting composite events. *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data*, 193–206. <https://doi.org/10.1145/1559845.1559867>
- [18] Artikis, A., Sergot, M., & Paliouras, G. (2015). An Event Calculus for Event Recognition. *IEEE Transactions on Knowledge and Data Engineering*, 27(4), 895-108. <https://doi.org/10.1109/TKDE.2014.2356476>
- [19] Alevizos, E., Artikis, A., & Paliouras, G. (2022). Wayeb: a CEP engine for spatio-temporal patterns. *Proceedings of the 16th ACM International Conference on Distributed and Event-Based Systems (DEBS)*, 174–177. <https://doi.org/10.1145/3524860.3533246>